POSTGRADUATE MASTER NOTES · COMPUTER NETWORKS

Chapter 3 — Transport Layer

Condensed, de-duplicated study notes for understanding and examination

What this is. A single, coherent re-write of a 186-page set of study notes on Kurose & Ross, Chapter 3. All unique content — definitions, mechanisms, derivations, worked examples, edge cases, class-note conventions and Wireshark-lab facts — is kept; material that the source stated two, three or four times (main text, exam-focus boxes, connection map, confusion list, formula sheet, revision sheet and question bank) is stated *once*, in the place where it is first needed, and cross-referenced elsewhere.

Sources used by the original notes (and therefore by these)

[TB] J. F. Kurose, K. W. Ross, *Computer Networking: A Top-Down Approach*, 8th ed., Ch. 3 (§3.1–§3.9). Primary detailed reference.

[SL] Kurose & Ross, Chapter 3 lecture slides, 9th ed. (2025), cited by footer label, e.g. SL 3-86. Used for lecture emphasis and 9th-edition-only material.

[CN] Professor's handwritten class notes. Where they set a convention for this course, it is flagged.

[AS] Assignment 3 — Wireshark Labs: TCP and UDP. Gathered into Section 10.

How to read the boxes

Definition	Precise statement, usually in the wording of [TB]/[SL]; learn nearly verbatim.
Σ Formula	Formula with variables, units, conditions, when to use it and the usual mistake.
Worked example	Given → Required → Formula → Substitution → Calculation → Answer → Interpretation.
Insight	Why the design is as it is; derivations and edge cases.
Source note	Edition differences, class conventions, or where the sources disagree — and which to use.
Exam trap	Used *only* where the warning is needed to understand the mechanism in hand; all other traps are in Section 11.
Supplementary	Outside both sources; included only to clarify. Results derived here from source statements are marked [S].
Exam Focus	One compact box per topic: memorise / understand / formulas / question patterns.

Source & convention note: Notation fixed for the whole document

The source reused several symbols; they are made consistent here.

- L = packet length in *bits* (Section 4); R = link rate (bit/s); $d_{\text{trans}} = L/R$.
- ℓ = number of *data bytes* in a TCP segment (the source wrote L here too).
- p = segment loss probability in the Mathis throughput relation (the source and SL 3-154 write L).
- N = window in packets (GBN/SR); *cwnd*, *rwnd*, *ssthresh* in bytes or in *MSS* units as stated; W = window at which loss occurs.
- k = number of bits in a sequence-number field; sequence space $\{0, \ldots, 2^k - 1\}$.
- λ_{in} = original-data rate, λ'_{in} = offered load (incl. retransmissions), λ_{out} = delivered throughput.

Conventions used in all numerical work (each is justified where it arises): slow start → congestion avoidance when *cwnd* ≥ *ssthresh* (§8.3.4); Reno deflates *cwnd* to *ssthresh* when fast recovery ends — the class convention (§8.4); *DevRTT* is updated *after* *EstimatedRTT*, using the new estimate (§6.5); congestion-window tables list the *cwnd* used *during* each round, and a loss "in round n" changes the window from round $n+1$.

Contents

1 Transport-Layer Services and Foundations **5**
1.1 The problem the transport layer solves . . . 5
1.2 Transport layer versus network layer . . . 6
1.3 The Internet's two transport protocols . . . 6
1.4 The end-to-end principle . . . 7
1.5 Terminology and chapter structure . . . 7

2 Multiplexing and Demultiplexing **9**
2.1 Problem and intuition . . . 9
2.2 Ports . . . 9
2.3 Connectionless (UDP) demultiplexing . . . 10
2.4 Connection-oriented (TCP) demultiplexing . . . 10
2.5 Worked demultiplexing examples . . . 11
2.6 Port scanning . . . 12

3 Connectionless Transport: UDP **13**
3.1 What UDP is . . . 13
3.2 Why choose UDP? . . . 13
3.3 UDP segment structure . . . 14
3.4 The Internet checksum . . . 14

4 Principles of Reliable Data Transfer **17**
4.1 The problem . . . 17
4.2 rdt1.0: a perfectly reliable channel . . . 17
4.3 rdt2.0: a channel with bit errors . . . 17
4.4 rdt2.1: sequence numbers for garbled ACK/NAKs . . . 18
4.5 rdt2.2: a NAK-free protocol . . . 19
4.6 rdt3.0: a channel with errors and loss . . . 19
4.7 Performance of stop-and-wait . . . 21
4.8 Pipelining . . . 22

5 Go-Back-N and Selective Repeat **25**
5.1 The shared framework . . . 25
5.2 Go-Back-N (GBN) . . . 25
5.3 Selective Repeat (SR) . . . 26
5.4 The SR window-size dilemma . . . 28
5.5 Packet reordering and sequence-number reuse . . . 28
5.6 Comparing GBN and SR . . . 29

6 Connection-Oriented Transport: TCP **31**
6.1 TCP at a glance . . . 31
6.2 The TCP connection, MSS and MTU . . . 31
6.3 TCP segment structure . . . 32
6.4 Sequence numbers and acknowledgment numbers . . . 33
6.5 Round-trip time estimation and timeout . . . 35
6.6 TCP reliable data transfer . . . 37

6.7 TCP flow control . . . 39
6.8 TCP connection management . . . 41

7 Principles of Congestion Control 45
7.1 What congestion is . . . 45
7.2 Causes and costs: three scenarios . . . 45
7.3 Approaches to congestion control . . . 47

8 TCP Congestion Control 48
8.1 Three questions, three answers . . . 48
8.2 AIMD . . . 49
8.3 Classic TCP congestion control . . . 49
8.4 Evolution of the congestion window . . . 51
8.5 Macroscopic throughput and "long, fat pipes" . . . 53
8.6 TCP CUBIC . . . 54
8.7 The bottleneck link: keep the pipe just full, but no fuller . . . 54
8.8 Network-assisted control: ECN . . . 55
8.9 Delay-based congestion control . . . 56
8.10 TCP fairness . . . 57

9 Evolution of Transport-Layer Functionality 59
9.1 Why the two "work horses" were not enough . . . 59
9.2 QUIC: Quick UDP Internet Connections . . . 59
9.3 Head-of-line blocking . . . 60

10 Reading Real Traces: the Wireshark Labs 62
10.1 Tools and capture . . . 62
10.2 The UDP capture . . . 62
10.3 The TCP capture: setup and segment structure . . . 62
10.4 Sequence numbers, ACKs and retransmissions in the trace . . . 63
10.5 RTT, timeout and throughput from a trace . . . 63
10.6 Flow control and window scaling in the trace . . . 64
10.7 Congestion control in a Stevens time–sequence graph . . . 64

11 Common Confusions & Exam Traps 65

12 Formula Sheet 67

13 Curated Question Bank 70
13.1 Level 1 — Basic recall . . . 70
13.2 Level 2 — Conceptual understanding . . . 70
13.3 Level 3 — Numerical . . . 71
13.4 Level 4 — Protocol tracing . . . 72
13.5 Level 5 — Analytical: compare, justify, debug . . . 73
13.6 Level 6 — Challenging (connecting several ideas) . . . 74

14 Final Revision 75
14.1 The chapter on one page . . . 75
14.2 The formulas you must be able to write from memory . . . 75
14.3 Protocol comparison . . . 76

14.4 Diagrams to be able to draw 76
14.5 High-value definitions 77
14.6 Last-minute checklist 77

1 Transport-Layer Services and Foundations

[TB §3.1; SL 3-2 to 3-9]

1.1 The problem the transport layer solves

A laptop runs a browser, a video call and a music app at once. The network layer (IP) can move a datagram from the laptop to a server's *host*; it neither knows which *program* the data is for nor promises that the data arrives, arrives intact, or arrives in order. Applications need the opposite: *process-to-process* delivery, often *reliable*. The transport layer is the software in the end systems that closes this gap.

- **Why not the network layer?** Routers are deliberately simple: they forward datagrams and keep no per-application state. "Intermediate routers neither act on, nor recognize, any information that the transport layer may have added to the application messages" [TB §3.1.1].
- **Why not each application?** It can (QUIC does, Section 9), but then every application re-implements process addressing, error detection, retransmission and pacing. Placing that machinery once, in the OS behind the socket interface, is the layered-design answer.

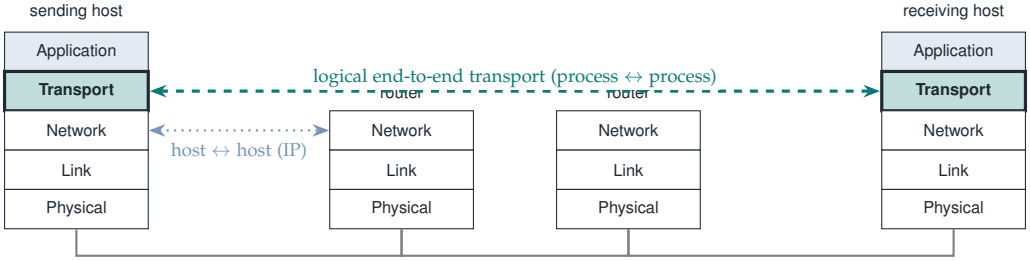


Figure 1: The five-layer Internet stack. Transport runs only in the end systems; routers implement the bottom three layers and never inspect transport headers. After TB Fig. 3.1 / SL 3-4.

Definition: Transport layer; segment

A transport-layer protocol provides **logical communication between application processes** running on different hosts: from the application's point of view it is as if the hosts were directly connected, although they may be separated by many routers and link types. [TB §3.1; SL 3-4]

On the sending side the transport layer converts application messages into transport-layer packets called **segments**, by (possibly) breaking each message into chunks and adding a transport header to each; the segment is handed to the network layer, which encapsulates it in a network-layer packet, a **datagram**.

Source & convention note: Segment vs datagram

The RFCs call a TCP packet a segment but often call a UDP packet a "datagram" (hence *User Datagram Protocol*), while also using "datagram" for IP packets. [TB] and [SL] avoid the clash by calling *both* TCP and UDP packets *segments* and reserving *datagram* for IP. These notes do the same.

What the transport layer does at each end. [SL 3-7, 3-8]

Sender	Receiver
1. is passed an application message through a socket	1. receives the segment from IP
2. determines the header field values	2. checks header values (e.g. checksum)
3. creates the segment (header + chunk)	3. extracts the application message
4. passes the segment to IP	4. demultiplexes it up to the right socket

Path: app $\xrightarrow{\text{socket}}$ transport (+header) $\xrightarrow{\text{segment}}$ network (+IP header) $\xrightarrow{\text{datagram}}$ routers $\cdots \rightarrow$ network $\rightarrow$ transport (check, demultiplex) $\xrightarrow{\text{socket}}$ app.

1.2 Transport layer versus network layer

[TB §3.1.1; SL 3-5, 3-6]

The household analogy. Two houses (East and West Coast), each with a dozen cousins who each write to every cousin in the other house every week. In each house one child — Ann (West), Bill (East) — collects and distributes the mail; the postal service carries it between houses.

Analogy	Network	Analogy	Network
letters in envelopes	application messages	houses	hosts (end systems)
cousins	processes	Ann and Bill	transport-layer protocol
		postal service	network-layer protocol

Four lessons, each with a technical counterpart:

1. Ann and Bill work only inside their homes ⇒ transport protocols live in end systems and have no say in how the core moves messages.
2. Substitutes Susan and Harvey give a worse service ⇒ a network may offer several transport protocols with different service models (TCP, UDP).
3. If the post office gives no delivery-time bound, Ann and Bill cannot promise one ⇒ transport services are *constrained* by the network-layer service model: no delay or bandwidth guarantee is possible over IP.
4. But some services can be *added*: reliable delivery over a lossy network, and confidentiality via encryption.

Table 1: Network layer versus transport layer.

	Network layer	Transport layer
Logical communication between	hosts	processes (running on hosts)
Addressing unit	IP address (a host interface)	IP address + port number (a socket)
Runs in	every host and every router	end systems only
Unit of data	datagram	segment
Internet protocol(s)	IP: best effort, unreliable	UDP (unreliable) or TCP (reliable)
Relationship	provides the underlying service	relies on, and enhances, it (SL 3-6)

Definition: IP's best-effort service

IP provides logical communication between hosts. Its service model is **best-effort delivery**: it makes its "best effort" but **does not guarantee delivery, does not guarantee orderly delivery, and does not guarantee the integrity** of the data. IP is therefore an *unreliable* service. Every host has at least one IP address. [TB §3.1.2]

1.3 The Internet's two transport protocols

[TB §3.1.2; SL 3-9]

The most fundamental responsibility of UDP and TCP is to **extend IP's host-to-host delivery to process-to-process delivery** — this is transport-layer multiplexing and demultiplexing (Section 2). Both also provide integrity checking through an error-detection field in their headers.

Three remarks from TB §3.1.2 worth quoting:

- Process-to-process delivery and error checking "are the only two services that UDP provides!"
- TCP "converts IP's unreliable service between end systems into a reliable data transport service between processes."
- Congestion control is "a service for the general good" rather than for the invoking application: it stops one connection from swamping the links and strives to give each connection an equal share, whereas a UDP application "can send at any rate it pleases, for as long as it pleases."

Table 2: Services provided and not provided (the only full UDP/TCP service table in these notes; the multi-protocol summary is Table 19).

Service	UDP	TCP
Process-to-process delivery	yes	yes
Error detection (checksum)	yes	yes
Reliable data transfer	no — segments may be lost or reordered	yes — correct, in order (sequence numbers, ACKs, timers, retransmission)
Connection setup	no (connectionless)	yes (three-way handshake)
Flow control	no	yes
Congestion control	no — "UDP traffic is unregulated"	yes
Delay / bandwidth guarantees	**no**	**no** — not available over IP
SL 3-9 summary	"unreliable, unordered delivery; no-frills extension of best-effort IP"	"reliable, in-order delivery; congestion control; flow control; connection setup"

Insight: Reliability and congestion control have different beneficiaries

Reliability benefits *this* application; congestion control benefits *everyone else* and often makes a single connection slower than it could be. That is why an application can rationally prefer UDP (§3.2), and why fairness becomes a social problem once UDP and parallel TCP connections are involved (§8.10).

1.4 The end-to-end principle

Definition: End-end principle (Saltzer 1984, via TB §3.3.2)

Since certain functionality must be implemented on an end-end basis, "functions placed at the lower levels may be redundant or of little value when compared to the cost of providing them at the higher level."

It is the design logic behind the whole chapter: routers stay simple (best effort), and anything that must hold end to end — error detection (§3.4), reliability, congestion response — is implemented in the end systems.

1.5 Terminology and chapter structure

Layer	Unit of data	What it contributes to this chapter
Application	message	processes, sockets (HTTP, DNS, SMTP . . .)
Transport	segment	ports; UDP or TCP; runs only in end systems
Network	datagram	host-to-host, best effort; IP addresses; Protocol field selects UDP/TCP
Link	frame	MTU limits the frame size (hence $MSS = \text{MTU} - 40$, §6.2)
Physical	bits	where bit errors originate

A *host* or *end system* runs all five layers; a *router* runs the bottom three. A *process* communicates through one or more *sockets*; a socket is identified using *port numbers* (Section 2).

The chapter alternates **principles** and **Internet protocols** (SL 3-2): four principles — multiplexing/demultiplexing, reliable data transfer, flow control, congestion control — and their instantiations UDP and TCP, followed by the newer evolution (CUBIC, ECN, BBR, QUIC). An essay on "the transport layer" should be organised around exactly these.

Exam Focus

Memorise. Definition of the transport layer (logical communication between *processes*); segment vs datagram; IP's three non-guarantees; UDP's two services; TCP's added services; services neither provides (delay, bandwidth).

Understand. Why transport runs only in end systems; how transport services are constrained by, yet can exceed, network services; why congestion control is "for the general good"; the end-end principle.

Diagram. Fig. 1: two five-layer hosts, three-layer routers, logical end-to-end arrow.

Typical questions. "Map the household analogy to the network." "Give a service transport can add and one it cannot."

"List the services of UDP and of TCP."

2 Multiplexing and Demultiplexing

[TB §3.2, Figs. 3.2–3.5; SL 3-11 to 3-25]

2.1 Problem and intuition

The postal service delivers a bundle of letters to the house; Ann reads the name on each envelope and hands it to the right cousin (*demultiplexing*). Going the other way she collects her siblings' letters, makes sure each carries a return name, and gives the pile to the carrier (*multiplexing*). In a host the "names" are **port numbers** and the "cousins" are **sockets**. SL 3-13 asks: how does the transport layer know to deliver a message to the Firefox process rather than Netflix or Skype? (TB's version: Web browsing plus one FTP and two Telnet sessions — four processes to choose between.)

Definition: Socket; multiplexing; demultiplexing

A process has one or more **sockets** — doors through which data passes between the network and the process. The receiving transport layer delivers data to a socket, not directly to a process; each socket has a unique identifier whose format depends on whether it is a UDP or a TCP socket.

Demultiplexing: delivering the data in a transport-layer segment to the correct socket (receiver).
Multiplexing: gathering data chunks at the source host from different sockets, encapsulating each with header information (later used for demultiplexing) to create segments, and passing them to the network layer (sender). [TB §3.2; SL 3-11]

Every host does both, because every host sends and receives; for any one segment, one end multiplexes and the other demultiplexes. SL 3-25 adds that mux/demux happens at *all* layers (based on header fields at each).

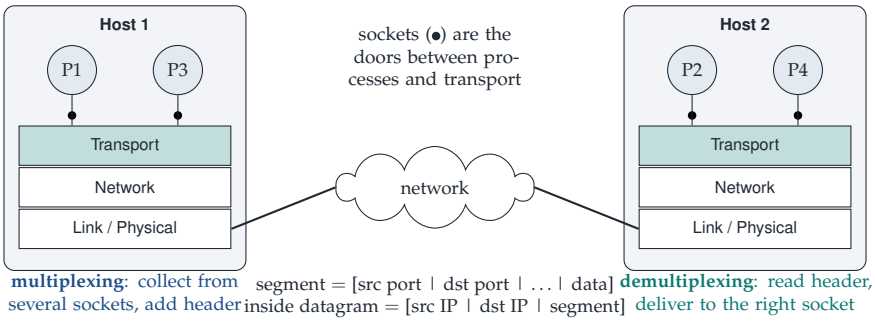


Figure 2: Transport-layer multiplexing and demultiplexing. After TB Fig. 3.2 / SL 3-11.

2.2 Ports

Multiplexing requires (1) unique socket identifiers and (2) fields in each segment naming the socket to deliver to: the **source port** and **destination port** fields, at the top of every UDP and TCP header (Figs. 5 and 19). [TB §3.2, Fig. 3.3]

Σ Formula: Port space

$$\text{port} \in \{0, 1, \ldots, 2^{16} - 1\} = \{0, \ldots, 65\,535\}, \qquad \text{well-known ports} = \{0, \ldots, 1023\}.$$

Meaning. 16-bit unsigned integers. Well-known ports are restricted and reserved for well-known application protocols (HTTP 80, FTP 21; list in RFC 1700 / `www.iana.org`). A new application must be assigned a port.

How ports are chosen. A *server* socket is bound to a specific (often well-known) port so clients can find it; a *client* socket is normally given one automatically — "a port number in the range 1024 to 65535 that is currently not being used" — because nobody needs to know it in advance.

Common mistake. "65 535 ports": there are $2^{16} = 65\,536$ values, 0 to 65 535 inclusive.

How demultiplexing works. [SL 3-20] The host receives IP datagrams; each has source and destination IP addresses and carries one segment; each segment has source and destination ports; the host uses *IP addresses and port numbers* to direct the segment to the appropriate socket. Which fields are used is exactly what distinguishes

UDP from TCP.

2.3 Connectionless (UDP) demultiplexing

[TB §3.2, Fig. 3.4; SL 3-21, 3-22]

A UDP socket is created and bound to a port (Python `clientSocket.bind(('', 19157))`; Java `new DatagramSocket(12534)`). To send, the application must name the destination IP address and port. TB's example: a process on Host A with port 19157 sends to port 46428 on Host B. A's transport layer builds a segment with source port 19157, destination port 46428 and two more fields (length, checksum) — multiplexing; IP makes a best-effort delivery; B's transport layer reads destination port 46428 and delivers to that socket — demultiplexing.

Definition: UDP socket identification

A UDP socket is fully identified by the **two-tuple (destination IP address, destination port number)**. Consequently, two UDP segments with different source IP addresses and/or source ports but the same destination IP and port are delivered to the **same** socket. [TB §3.2; SL 3-21]

What the source port is for. It is part of the **return address**: when B replies, the destination port of the B→A segment is taken from the source port of the A→B segment (in `UDPServer.py` the server obtains it from `recvfrom()`). SL 3-22 shows the same rule with three hosts: two clients (ports 9157, 5775) both send to server port 6428, land on the one server socket, and receive replies addressed to 9157 and 5775.

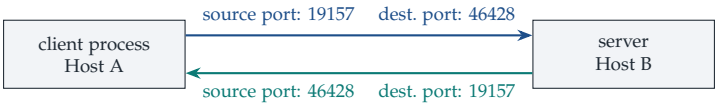


Figure 3: Inversion of source and destination port numbers in a reply. After TB Fig. 3.4.

Source & convention note: "UDP demultiplexes on the destination port only" vs "two-tuple"

SL 3-25 says UDP demultiplexes using the destination port number *only*; TB says a UDP socket is identified by (destination IP, destination port). They agree: the destination IP has already been used by the network layer to reach this host (and selects among a multihomed host's addresses), so inside the transport layer the only discriminating field is the destination port. *"What identifies a UDP socket?"* → the two-tuple. *"Which field does UDP use to demultiplex?"* → the destination port.

2.4 Connection-oriented (TCP) demultiplexing

[TB §3.2, Fig. 3.5; SL 3-23, 3-24]

Definition: TCP socket identification — the 4-tuple

A TCP socket is identified by the **four-tuple (source IP, source port, destination IP, destination port)**, and the host uses all four values to demultiplex. In contrast with UDP, two arriving TCP segments with different source IP addresses or source ports are directed to two *different* sockets — with the exception of a segment carrying the original connection-establishment request.

How the 4-tuple socket comes into existence.

1. The server has a **welcoming socket** waiting for connection requests on a port, e.g. 12000.
2. The client creates a socket and sends a connection-establishment request: `clientSocket = socket(AF_INET, SOCK_STREAM); clientSocket.connect((serverName,12000))`. The request is just a TCP segment with destination port 12000, a client-chosen source port, and the SYN bit set (§6.8.1).
3. The server OS finds the process waiting on 12000, which creates a new **connection socket**: `connectionSocket, addr = serverSocket.accept()`.
4. The new socket is identified by the four values in the request; all later segments whose four values match are demultiplexed to it.

A server may therefore hold many simultaneous connection sockets, "each associated with a different connecting

client" (SL 3-23).

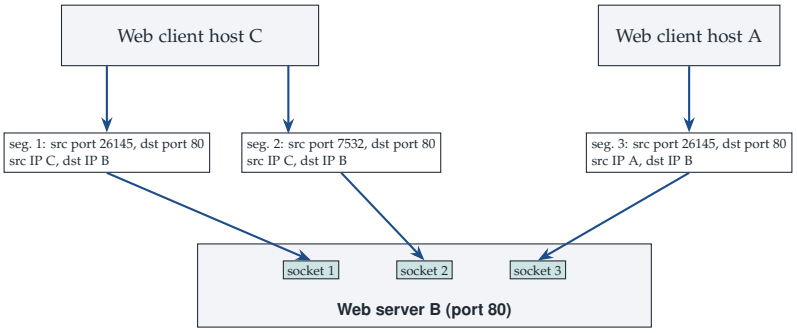


Figure 4: Two clients use destination port 80 on the same server; three connection sockets are distinguished by their 4-tuples. Host A and host C happen to choose the same source port 26145. After TB Fig. 3.5 / SL 3-24.

Insight: Why thousands of clients can share port 80 (or 443)

Every segment reaching a busy Web server has destination (B, 80). Demultiplexing on those two values alone would put every client's bytes into one socket and leave the server unable to separate interleaved byte streams. The 4-tuple makes each connection its own socket; two 4-tuples need differ in only one component — which is why A and C in Fig. 4 may both use source port 26145: "the two connections have different source IP addresses."

Web servers and TCP. [TB §3.2]

- All segments to a Web server — connection requests and HTTP requests alike — carry destination port 80; clients are told apart by source IP and source port.
- Sockets and processes need not correspond one-to-one: high-performance servers often use one process and create a new *thread* with a new connection socket per client, so many sockets may belong to one process.
- Persistent HTTP uses one connection socket for the life of the connection; non-persistent HTTP creates and closes a new connection (and socket) per request/response, which can severely load a busy server.

2.5 Worked demultiplexing examples

Worked example: Which UDP socket receives the segment?

Given. Server S (10.0.0.5) has UDP socket U1 on port 53 and U2 on port 5000. Arrivals: (i) 192.168.1.10:40000 → 10.0.0.5:53; (ii) 172.16.4.4:40000 → 10.0.0.5:53; (iii) 192.168.1.10:40001 → 10.0.0.5:5000.

Required. Receiving socket for each; the address of S's reply to (ii).

Rule. UDP uses (destination IP, destination port) only.

Answer. (i) U1; (ii) U1 — the different source IP is irrelevant; (iii) U2. The reply to (ii) goes to 172.16.4.4 port 40000 with source port 53.

Interpretation. (i) and (ii) come from different hosts yet share one socket; the application learns who sent what only from the source address returned by `recvfrom()`.

Worked example: Which TCP socket receives the segment?

Given. Server B has a welcoming socket on port 80 and connection sockets S1 $= (C, 26145, B, 80)$, S2 $= (C, 7532, B, 80)$, S3 $= (A, 26145, B, 80)$ written as (src IP, src port, dst IP, dst port). Arrivals: (a) A:26145→B:80, HTTP request; (b) C:7532→B:80, HTTP request; (c) D:26145→B:80 with SYN set; (d) C:26145→B:80, data.

Rule. Match all four values; a SYN with no matching connection goes to the welcoming socket, and `accept()` creates a new socket.

Answer. (a) S3; (b) S2; (c) welcoming socket, which creates S4 $= (D, 26145, B, 80)$; (d) S1.

Interpretation. (a) and (d) share source port and destination but reach different sockets because their source IPs differ. Under UDP's rule all four would have reached one socket.

Worked example: Same port number over TCP and over UDP

Given. Client C opens two TCP connections to B:443 from ports 50001 and 50002, and one UDP flow from port 50001 to B's UDP port 443 (HTTP/3 over QUIC runs on UDP).

Answer. At B: two TCP connection sockets $(C, 50001, B, 443)$ and $(C, 50002, B, 443)$ plus the TCP welcoming socket, and one UDP socket $(B, 443)$. No clash: TCP and UDP have *separate port spaces*, and the IP header's Protocol field (6 = TCP, 17 = UDP) tells the host which transport protocol receives the payload.

Label. The separate-port-space and Protocol-field facts are network-layer details (TB Ch. 4) — *supplementary*.

2.6 Port scanning

[TB §3.2, Focus on Security] A server waits on an open port; knowing which ports are open reveals which applications run — useful to administrators and to attackers "casing the joint". Some ports are used by convention (Microsoft SQL server listens on UDP 1434, whose buffer overflow the Slammer worm exploited). Scanners such as **nmap** probe TCP ports for accepted connections and UDP ports for responses, reporting open, closed or unreachable; how nmap reads TCP responses is in §6.8.3.

Exam Focus

Memorise. Mux = sender, demux = receiver; ports 16 bits, 0–65 535, well-known 0–1023, clients 1024–65 535; UDP socket = 2-tuple; TCP socket = 4-tuple; welcoming vs connection socket.

Understand. Source port as return address; why UDP segments from different sources share a socket; why TCP needs the 4-tuple; how `accept()` creates the connection socket; why one process may own many sockets.

Diagrams. Figs. 2 to 4.

Question patterns. "Which socket receives each of these segments?" "Two clients use the same source port to the same server port — can the server tell them apart?" "Destination port of the reply?" "Why does UDP need a source port if demultiplexing ignores it?"

3 Connectionless Transport: UDP

[TB §3.3; SL 3-27 to 3-38; RFC 768]

3.1 What UDP is

Design the thinnest possible transport protocol: it must at least reach the right process (otherwise it would be IP); everything else is optional. UDP is exactly that — "aside from the multiplexing/demultiplexing function and some light error checking, it adds nothing to IP"; an application using UDP "is almost directly talking with IP" [TB §3.3].

Definition: UDP

UDP takes messages from the application, attaches source and destination port fields for multiplexing/demultiplexing, adds two other small fields (length, checksum), and passes the segment to the network layer, which makes a best-effort attempt to deliver it. On arrival UDP uses the destination port to deliver the data to the right process. There is **no handshaking** before a segment is sent, so UDP is **connectionless**; each segment is handled independently. SL 3-27/3-38: "no frills", "bare bones", "best effort — send and hope for the best".

Consequences of what UDP omits (services in Table 2): segments may be lost or reordered and are never retransmitted; the first segment can carry data at once; no per-client state is kept; a slow reader can overflow the socket buffer and lose segments at the receiver (§6.7); UDP sends at whatever rate the application chooses.

3.2 Why choose UDP?

[TB §3.3]

1. **Finer application control over what is sent and when.** UDP hands data to IP immediately. TCP throttles the sender under congestion and keeps resending until acknowledged "regardless of how long reliable delivery takes". Real-time applications want a minimum rate, little delay, and can tolerate some loss.
2. **No connection establishment.** No three-way handshake delay — probably the principal reason DNS uses UDP. (HTTP uses TCP because reliability matters for Web pages; QUIC, in Chrome, uses UDP and builds reliability above it.)
3. **No connection state.** No send/receive buffers, congestion parameters or sequence/ACK numbers per client, so a server can support many more active clients.
4. **Small header overhead**: 8 bytes against TCP's 20.

Table 3: Popular applications and their transport protocols (TB Fig. 3.6).

Application	Application-layer protocol	Transport
E-mail; remote terminal; secure terminal	SMTP; Telnet; SSH	TCP
Web	HTTP, HTTP/3	TCP (HTTP), UDP (HTTP/3)
File transfer	FTP	TCP
Remote file server	NFS	typically UDP
Streaming multimedia	DASH	TCP
Internet telephony	typically proprietary	UDP or TCP
Network management; name translation	SNMP; DNS	typically UDP

- **Reliability over UDP** (SL 3-28, 3-38): if an application such as HTTP/3 needs reliable transfer over UDP, it adds reliability *and* congestion control at the application layer.
- **SNMP over UDP**: network management "must often run when the network is in a stressed state — precisely when reliable, congestion-controlled data transfer is difficult to achieve" (SL 3-38: UDP "can function when network service is compromised"). TCP would throttle exactly the traffic needed to diagnose congestion.
- **Streaming is not automatically UDP.** DASH runs over TCP; telephony over UDP *or* TCP. With low loss rates, and with some organisations blocking UDP for security, TCP is increasingly attractive for media. Choose by service requirements, not by application category.

- **The social cost of no congestion control.** If everyone streamed high-rate video over UDP without congestion control, router overflow would let few packets through, and the high loss would make TCP senders slash their rates: UDP can cause high loss and *crowd out* TCP sessions (continued in §8.10).

3.3 UDP segment structure

[TB §3.3.1, Fig. 3.7; SL 3-33]

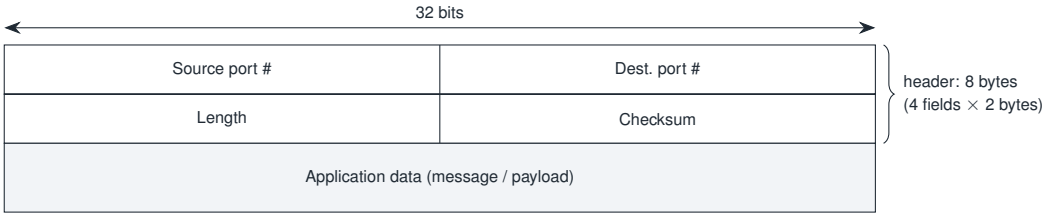


Figure 5: UDP segment format. Length counts header *plus* data. After TB Fig. 3.7 / SL 3-33.

Field	Size	Purpose (sender / receiver)
Source port	16 bits	Return address. Receiver ignores it for demultiplexing but passes it to the application (`recvfrom()`).
Destination port	16 bits	Demultiplexing: receiver chooses the socket from it.
Length	16 bits	Bytes in the UDP segment, header plus data — needed because data size varies per segment.
Checksum	16 bits	Error detection over the header, the data and (TB) a few IP-header fields including the IP addresses. Receiver recomputes and compares.
Data	variable	e.g. a DNS query/response, or audio samples.

Σ Formula: UDP length and header overhead

$$\text{Length} = 8 + D, \qquad D_{\max} = (2^{16} - 1) - 8 = 65\,527 \text{ bytes}^{[S]},$$

$$\text{overhead} = \frac{H}{H + D}, \qquad H_{\text{UDP}} = 8, \quad H_{\text{TCP}} = 20 \text{ bytes (no options)}.$$

Variables. D = application-data bytes; Length in bytes. The 16-bit field's maximum is 65 535 bytes, 8 of which are header (in practice IP's own length limit is tighter — TB Ch. 4).

Instances. 512-byte DNS message ⇒ Length = 520. Overhead[S]: 40-byte payload — UDP $8/48 = 16.7\%$, TCP $20/60 = 33.3\%$; 1460-byte payload — UDP 0.5%, TCP 1.4%. The header advantage matters for small messages (DNS, SNMP, voice samples) and is negligible for bulk transfer.

Common mistake. Treating Length as the payload length; quoting 65 535 as the payload limit.

3.4 The Internet checksum

[TB §3.3.2; SL 3-34 to 3-37]

Intuition. Bits flip in transit (link noise) or while a packet sits in router memory. The sender sends a short summary of the contents; the receiver recomputes it and compares. SL 3-34 with ordinary numbers: send 5, 6 and their sum 11; receive 4, 6, 11; $4 + 6 = 10 \neq 11 \Rightarrow$ error detected. The Internet checksum does the same with 16-bit words and one's-complement arithmetic.

Definition: Internet checksum

Sender: treat the segment contents — including the UDP header and the IP addresses — as a sequence of 16-bit integers; compute their **one's-complement sum** (a carry out of the most significant bit is added back into the least significant bit, "wrapped around"); the checksum is the **one's complement** of that sum (all bits inverted), placed in the checksum field.
Receiver: add all 16-bit words *including the checksum*. If there are no errors the result is `1111111111111111`; any 0 bit ⇒ error detected. Equal values mean *no error detected* — "but maybe errors nonetheless?" [SL 3-35]

Σ Formula: Checksum computation

With 16-bit words $w_1, \ldots, w_n$ (checksum field taken as 0 while computing), define

$$S_0 = 0, \qquad S_k = (S_{k-1} + w_k) \bmod 2^{16} + \left\lfloor \frac{S_{k-1} + w_k}{2^{16}} \right\rfloor, \qquad \text{checksum} = \overline{S_n} = (2^{16} - 1) - S_n.$$

Receiver test: $w_1 \oplus_1 \cdots \oplus_1 w_n \oplus_1 \text{checksum} = 1111111111111111$ ($\oplus_1$ = one's-complement addition).

Why all 1s. For any 16-bit S, $S + \overline{S} = \texttt{0xFFFF}$: every bit position holds exactly one 1, so no carries arise.

Common mistakes. (1) Dropping the carry instead of wrapping it. (2) Sending the sum instead of its complement. (3) Thinking word order matters — one's-complement addition is commutative and associative.

Worked example: Computing and verifying the checksum (TB §3.3.2)

Given. $w_1 = 0110011001100000$, $w_2 = 0101010101010101$, $w_3 = 1000111100001100$.

Required. The transmitted checksum and the receiver's check.

Calculation.

0110011001100000	
+ 0101010101010101	
1011101110110101	(no carry)
+ 1000111100001100	
1 0100101011000001	carry out of the MSB
wrap: 0100101011000001 + 1 = 0100101011000010	= sum
complement: 1011010100111101	= checksum

Verification. $0100101011000010 \oplus_1 1011010100111101 = 1111111111111111$ ✓.

Interpretation. The carry in step 2 would be silently lost in ordinary 16-bit arithmetic; wrapping it is what makes the result independent of word order (and, per RFC 1071, of host byte order — supplementary). SL 3-36 works the same procedure on two words: $1110011001100110 + 1010101010101010$ gives sum 1011101110111100 after wraparound and checksum 0100010001000011.

Worked example: What the checksum catches and what it misses

Single-bit error. Flip the LSB of w_1 above ($0 \to 1$): the true sum rises by exactly 1, so the receiver's total becomes $\texttt{0xFFFF} + 1$, which overflows and wraps to 0000000000000001 — contains 0s, error detected. Any single-bit error changes the total by $\pm 2^j$ and is always detected.

Undetectable double-bit error. Bit 0 is 0 in w_1 and 1 in w_2. Flip both: w_1' gains 1, w_2' loses 1; the sum is unchanged and the receiver again gets all 1s — corrupted data accepted.

Interpretation. Any two errors in the same bit position of two words, in opposite directions, cancel; so do swaps of whole 16-bit words. SL 3-37's title: "Internet checksum: weak protection!" The checksum trades strength for speed and simplicity; link-layer CRCs (TB Ch. 6) catch far more. What happens to a detected error is implementation-dependent: some UDP implementations discard the segment, others pass it up with a warning.

Why UDP has a checksum at all. [TB §3.3.2] Link layers such as Ethernet check errors, but (i) not every link on the path is guaranteed to, and (ii) bits can be corrupted while a segment is stored in a router's memory. Since neither link-by-link reliability nor in-memory error detection is guaranteed, error detection must be provided end to end if the end-to-end service is to have it — an instance of the end-end principle (§1.4). Because IP runs over almost any layer 2, a transport-layer check is a sensible safety measure.

Detection, correction and reliability are different.

Capability	Meaning	UDP?
Error detection	noticing flipped bits	yes — but not all errors
Error correction	repairing bits without retransmission	no
Reliable transfer	every byte delivered, uncorrupted, once, in order, even after loss	no

"Although UDP provides error checking, it does not do anything to recover from an error" [TB]; and a checksum cannot detect a *lost* segment at all. Reliability needs sequence numbers, acknowledgments, timers and retransmission (Section 4); SL 3-38's "helps with reliability (checksum)" means it supplies the detection step a reliable protocol needs.

Exam Focus

Memorise. UDP = mux/demux + error checking; connectionless; four reasons to use it; four 16-bit header fields; Length includes the header; checksum covers header, data and IP addresses; sum with wraparound, complement, receiver expects all 1s.

Understand. Why DNS and SNMP use UDP; why HTTP/3 builds reliability above UDP; why the checksum is end-to-end; why it misses some multi-bit errors; detection $\neq$ correction $\neq$ reliability.

Numerical patterns. Compute a checksum (show every carry and wrap); verify a received segment; construct an undetectable error; Length for a D-byte payload.

Conceptual patterns. "Why choose UDP over TCP?" "Why a checksum if the link layer has one?" "Is UDP reliable because it has a checksum?"

4 Principles of Reliable Data Transfer

[TB §3.4.1–§3.4.2, Figs. 3.8–3.18; SL 3-40 to 3-69]

4.1 The problem

Reliable data transfer is a "top-ten" problem of networking that arises at the link, transport and application layers. The service abstraction: over a **reliable channel** no transferred bits are corrupted or lost, and all are delivered in the order sent. A reliable data transfer (rdt) protocol must implement this abstraction although the layer below may be unreliable — TCP is a reliable protocol on top of the unreliable IP. SL 3-43 stresses two facts that drive everything that follows: the protocol's complexity depends *strongly* on the channel's characteristics (can it lose, corrupt, reorder?), and sender and receiver do not know each other's state ("was my message received?") unless it is communicated in a message.

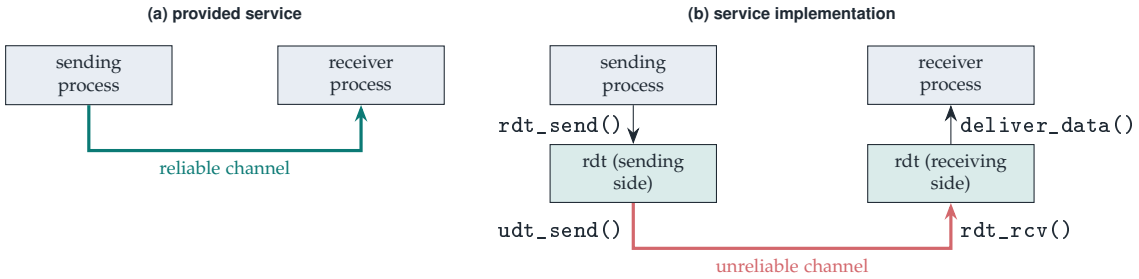


Figure 6: Reliable data transfer: service model and implementation, with the four interface primitives. After TB Fig. 3.8 / SL 3-44.

Interface primitives. `rdt_send()` is called from above to pass data for delivery; `udt_send()` is called by rdt to send a packet over the *u*nreliable channel; `rdt_rcv()` is called when a packet arrives at the receiving side; `deliver_data()` is called by rdt to pass data up.

Ground rules. Unidirectional *data* transfer, but control packets flow both ways (both sides call `udt_send()`); the channel does not reorder packets (relaxed in §5.5); "packet" is used instead of "segment" because the theory is general.

FSM notation. [TB §3.4.1; SL 3-45] Sender and receiver are finite-state machines. In a state, the next state is uniquely determined by the next event. Each transition arrow is labelled with the *event* above a horizontal line and the *actions* below it; Λ means no event or no action; a dashed arrow marks the initial state.

4.2 rdt1.0: a perfectly reliable channel

No bit errors, no loss, and the receiver is as fast as the sender. Each side has one state: the sender waits for `rdt_send(data)`, makes a packet and sends it; the receiver extracts and delivers. There is "no difference between a unit of data and a packet"; no feedback, sequence numbers or timers are needed because nothing can go wrong — on a channel that does not exist.

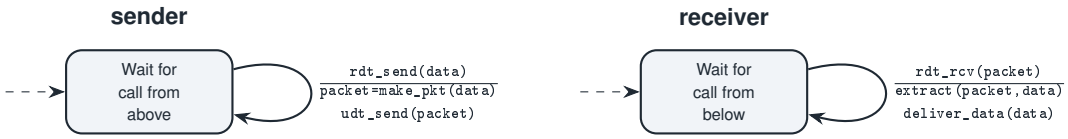


Figure 7: rdt1.0. After TB Fig. 3.9 / SL 3-46.

4.3 rdt2.0: a channel with bit errors

Intuition. New assumption: bits in a packet may flip (in transmission, propagation or buffering); packets still arrive, in order. Dictating a message over the phone, the listener says "OK" after each understood sentence and "Please repeat that" after a garbled one — positive and negative acknowledgments.

Definition: ARQ (Automatic Repeat reQuest)

Reliable transfer protocols based on retransmission are **ARQ protocols**. Handling bit errors requires three capabilities [TB §3.4.1]:

1. **Error detection** — e.g. the Internet checksum, extra bits carried in the packet;
2. **Receiver feedback** — the only way the sender learns the receiver's view: positive (ACK) and negative (NAK) acknowledgments, in principle one bit each;
3. **Retransmission** of a packet received in error.

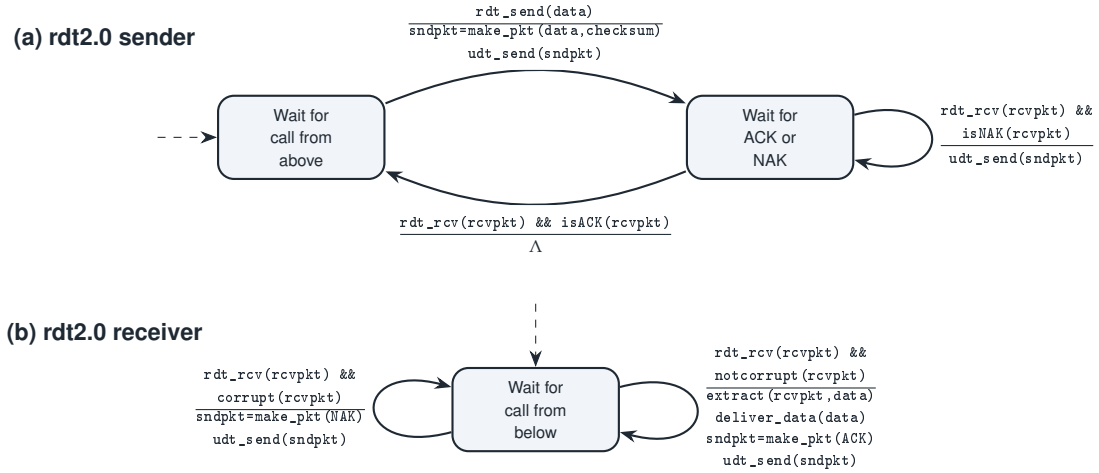


Figure 8: rdt2.0 for a channel with bit errors. After TB Fig. 3.10 / SL 3-49 to 3-52.

Mechanism. The sender makes a packet with a checksum, sends it, and moves to "Wait for ACK or NAK". An ACK returns it to waiting for data; a NAK causes retransmission. Crucially, in the waiting state `rdt_send()` cannot occur: the sender will not accept new data until it is sure the current packet was received correctly. Protocols with this behaviour are **stop-and-wait** protocols. The receiver (one state) replies ACK or NAK according to the checksum. SL 3-50: the receiver's state "isn't known to sender unless somehow communicated . . . that's why we need a protocol!"

The fatal flaw. [TB §3.4.1; SL 3-53] What if the ACK or NAK is corrupted? Checksum bits on ACK/NAKs detect it, but the sender still cannot know whether the receiver got the data. Three candidate fixes:

1. Ask "What did you say?" — which can itself be garbled; "we're heading down a difficult path."
2. Add enough checksum bits to *correct* errors — works only for a channel that corrupts but never loses.
3. Resend the current packet on a garbled ACK/NAK — but this introduces **duplicates**, and the receiver cannot tell new data from a retransmission.

Solution, "adopted in almost all existing data transfer protocols, including TCP": number the data packets with a **sequence number**. The receiver checks it to recognise retransmissions and discards (does not deliver) duplicates. For stop-and-wait a 1-bit number suffices, and because the channel does not lose packets, ACK/NAKs need not carry a number: they answer the most recent packet.

4.4 rdt2.1: sequence numbers for garbled ACK/NAKs

Mechanism. The protocol state must now record whether the packet being sent (or expected) is numbered 0 or 1, so each FSM doubles its states; the 0 and 1 halves are mirror images.

- Receiver, correct packet with the *expected* number: deliver, ACK, flip expected number.
- Receiver, correct packet with the *other* number: a retransmission of the packet just received — send an ACK (so the sender can move on) but do **not** deliver.
- Receiver, corrupted packet: NAK. Sender, corrupted ACK/NAK: treat like a NAK and retransmit.

SL 3-56 adds: the receiver cannot know whether its last ACK/NAK was received OK at the sender.

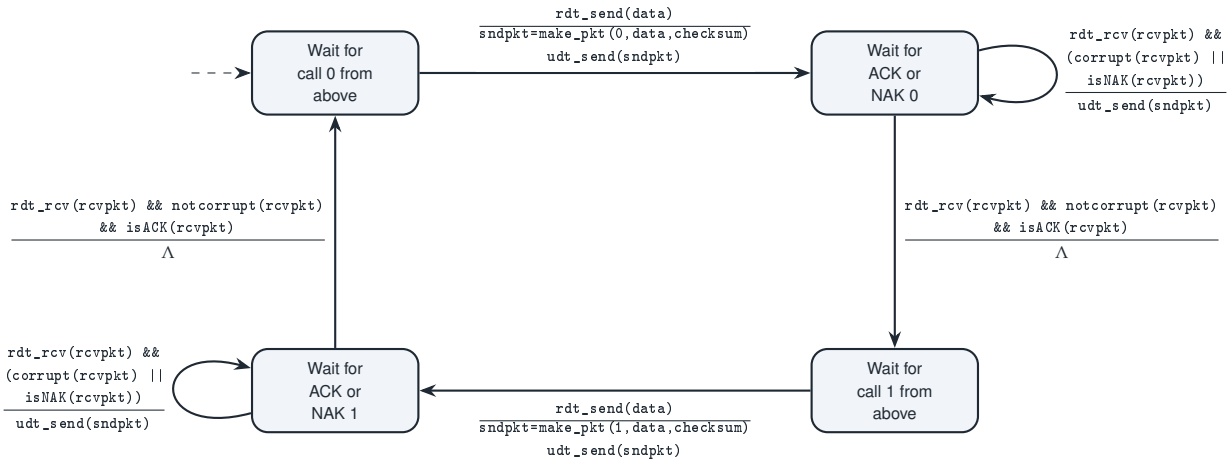


Figure 9: rdt2.1 sender: twice as many states, remembering whether the current packet is numbered 0 or 1. After TB Fig. 3.11 / SL 3-54.

Insight: Why two sequence numbers suffice for stop-and-wait

Only one packet is ever outstanding, so the receiver's only question is binary: "the packet I just got, or the next one?". Consecutive distinct packets always carry different bits and a retransmission always carries the same bit as its original. A larger space adds nothing for stop-and-wait over a non-reordering channel — with pipelining the answer changes completely (Section 5).

4.5 rdt2.2: a NAK-free protocol

Definition: rdt2.2

Same functionality as rdt2.1 using **ACKs only**. Instead of a NAK the receiver sends an ACK for the *last correctly received* packet, so the ACK must **carry the sequence number** it acknowledges. A sender receiving two ACKs for the same packet — a **duplicate ACK** — knows the following packet was not received correctly, and reacts exactly as to a NAK: retransmit the current packet. "As we will see, TCP uses this approach to be NAK-free" [SL 3-57].

The sender changes in two places relative to rdt2.1 (TB Fig. 3.13): in "Wait for ACK 0" the retransmit condition becomes `corrupt(rcvpkt) || isACK(rcvpkt,1)` and the move-on condition `notcorrupt(rcvpkt) && isACK(rcvpkt,0)`; symmetrically for "Wait for ACK 1". So a duplicate ACK is not a glitch but a deliberate *implicit NAK* — the idea behind TCP's fast retransmit (§6.6.1).

4.6 rdt3.0: a channel with errors and loss

Intuition. The channel may now also *lose* packets (data or ACK). Checksums, sequence numbers, ACKs and retransmission help "but not quite enough" (SL 3-59): two new questions are how to *detect* loss and what to do about it. In conversation: wait a while, and if there is no reply, say it again.

Definition: The rdt3.0 approach (TB §3.4.1; SL 3-60)

The **sender** is responsible for detecting and recovering from loss. It waits a "reasonable" time for an ACK and retransmits if none arrives.

- *How long?* At least an RTT plus receiver processing — a worst case that is hard to estimate and slow to wait for. So the sender chooses a value at which loss is *likely*, not guaranteed.
- A packet or ACK that was merely delayed produces a duplicate; sequence numbers (rdt2.2) already handle that. The receiver must state the number being ACKed.
- Implementation needs a **countdown timer** that the sender can start on every transmission (first or re-), respond to on interrupt, and stop.

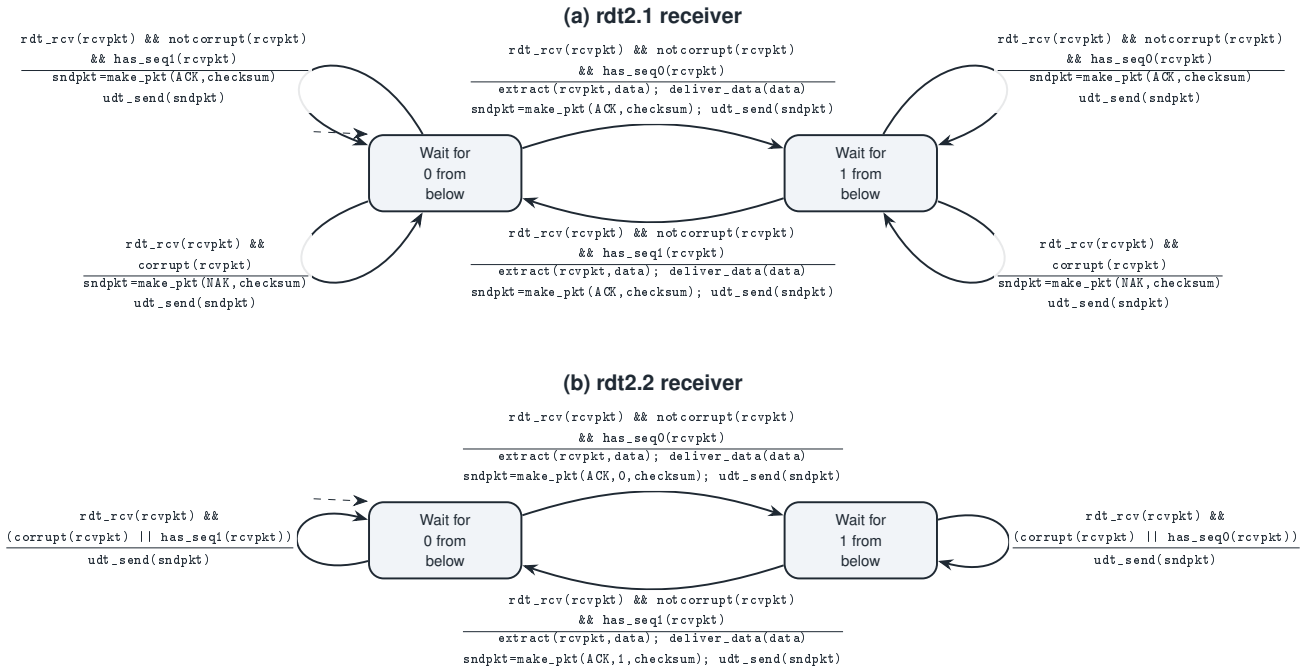


Figure 10: (a) rdt2.1 receiver (TB Fig. 3.12 / SL 3-55): corrupted ⇒ NAK; wrong-numbered (duplicate) ⇒ ACK, not delivered. (b) rdt2.2 receiver (TB Fig. 3.14 / SL 3-58): NAK-free; the self-loop `udt_send(sndpkt)` re-sends the last ACK, which names the last correctly received sequence number.

Insight: "From the sender's viewpoint, retransmission is a panacea"

The sender cannot tell whether a data packet was lost, its ACK was lost, or either was simply delayed. In every case the action is the same: **retransmit**. GBN, SR and TCP all inherit this; the price is that duplicates must be tolerated — which is why sequence numbers came one step earlier.

State	Event	Action and reason
Wait for call 0	`rdt_send(data)`	send pkt0, start timer
Wait for call 0	any `rdt_rcv`	Λ — a late/duplicate ACK when nothing is outstanding is ignored
Wait for ACK 0	corrupt ACK or ACK1	Λ — let the *timer* handle it (contrast rdt2.2, which retransmitted at once)
Wait for ACK 0	timeout	retransmit pkt0, restart timer
Wait for ACK 0	good ACK0	stop timer, go to Wait for call 1

The rdt3.0 receiver. TB leaves it as homework; the expected answer is that the **rdt2.2 receiver (Fig. 10b) works unchanged**: deliver and ACK a correct packet with the expected number; re-ACK the last good packet on corruption or duplicate. Loss needs no receiver action — the receiver cannot see a packet that never arrived. Because the numbers alternate 0,1,0,1, rdt3.0 is the **alternating-bit protocol**; TB: "We now have a working reliable data transfer protocol!"

Worked example: What does each side know? — the premature timeout, scenario (d)

Given. pkt1 arrives; ACK1 is merely delayed; the sender's timer expires first.

Trace.

- At the timeout the sender knows only that no ACK1 has arrived; it cannot distinguish "pkt1 lost", "ACK1 lost" and "ACK1 slow". It retransmits pkt1 and restarts the timer.
- The receiver has delivered pkt1 and expects 0. The retransmitted pkt1 has the wrong number ⇒ duplicate: not delivered, ACK1 re-sent.
- The delayed ACK1 arrives: the sender moves to "Wait for call 0", sends pkt0, starts the timer.
- The second ACK1 arrives in "Wait for ACK 0": wrong number ⇒ Λ.

Answer. Nothing is lost or duplicated at the application; one packet and one ACK were sent unnecessarily.

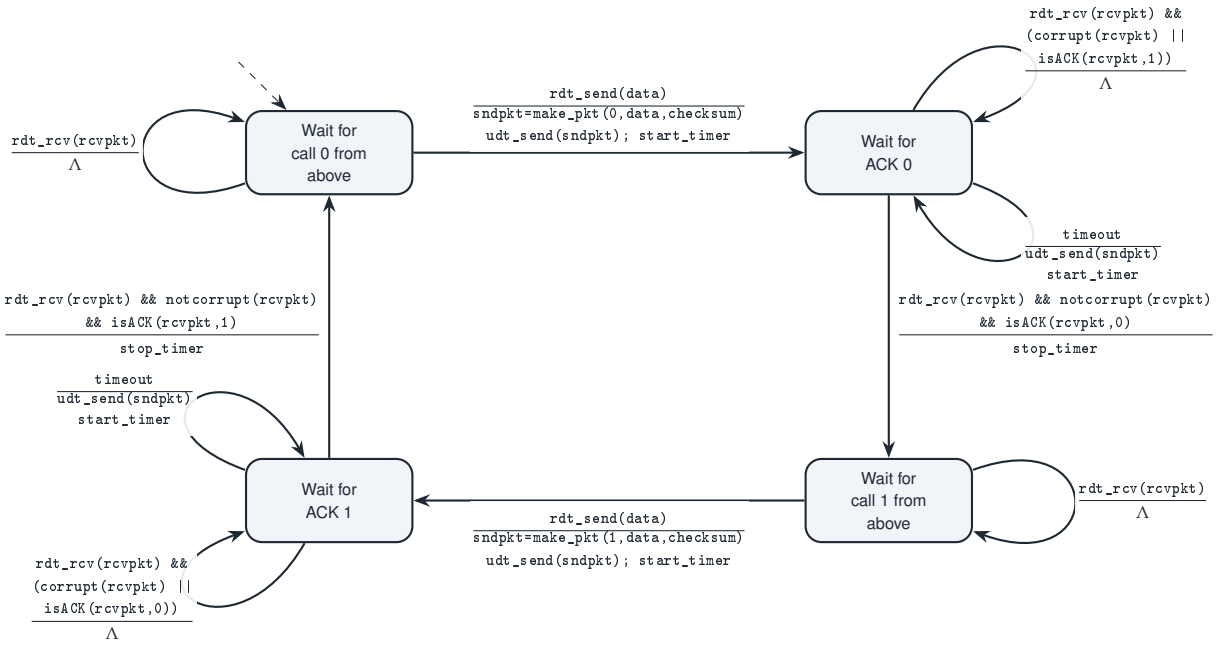


Figure 11: rdt3.0 sender. After TB Fig. 3.15 / SL 3-61, 3-62.

Interpretation. This is why the "ignore wrong-numbered ACKs" self-loops are essential: had the sender treated the duplicate ACK1 as a retransmit trigger (rdt2.2 behaviour), every premature timeout would cascade into further unnecessary retransmissions.

Table 4: The rdt family: each version fixes exactly one weakness of its predecessor (the rdt2.x vs rdt3.0 comparison).

	rdt2.0	**rdt2.1**	**rdt2.2**	**rdt3.0**
Channel	bit errors	bit errors, incl. on ACK/NAK	bit errors	bit errors **and loss**
New mechanism	checksum, ACK, NAK, retransmission	1-bit sequence number on data	ACK carries seq #; duplicate ACK = NAK	countdown **timer**
Sender / receiver states	2 / 1	4 / 2	4 / 2	4 / 2 (rdt2.2 receiver)
Handles duplicates?	no	yes	yes	yes
Handles loss?	no	no	no	yes
Remaining weakness	fails if ACK/NAK corrupted	uses NAKs	cannot detect loss	stop-and-wait: poor utilisation

4.7 Performance of stop-and-wait

rdt3.0 is correct, but its performance "is unlikely to be satisfactory" (TB); SL 3-67: "rdt 3.0 protocol performance stinks!" After pushing one packet onto the link the sender idles for a whole round trip.

Σ Formula: Transmission delay and stop-and-wait utilisation

$$d_{\text{trans}} = \frac{L}{R}, \qquad U_{\text{sender}} = \frac{L/R}{RTT + L/R}, \qquad \text{throughput} = \frac{L}{RTT + L/R}.$$

Variables & units. L = packet size (bits), R = link rate (bit/s), RTT in s (made of propagation delays, taken as given); U dimensionless.

Definition. Utilisation = fraction of time the sender (channel) is actually busy sending bits.

Derivation. From the first bit at $t = 0$, the last bit leaves at L/R and reaches the receiver $RTT/2$ later; the tiny ACK returns after another $RTT/2$. The next packet starts at $RTT + L/R$ and the sender was busy for L/R of that cycle.

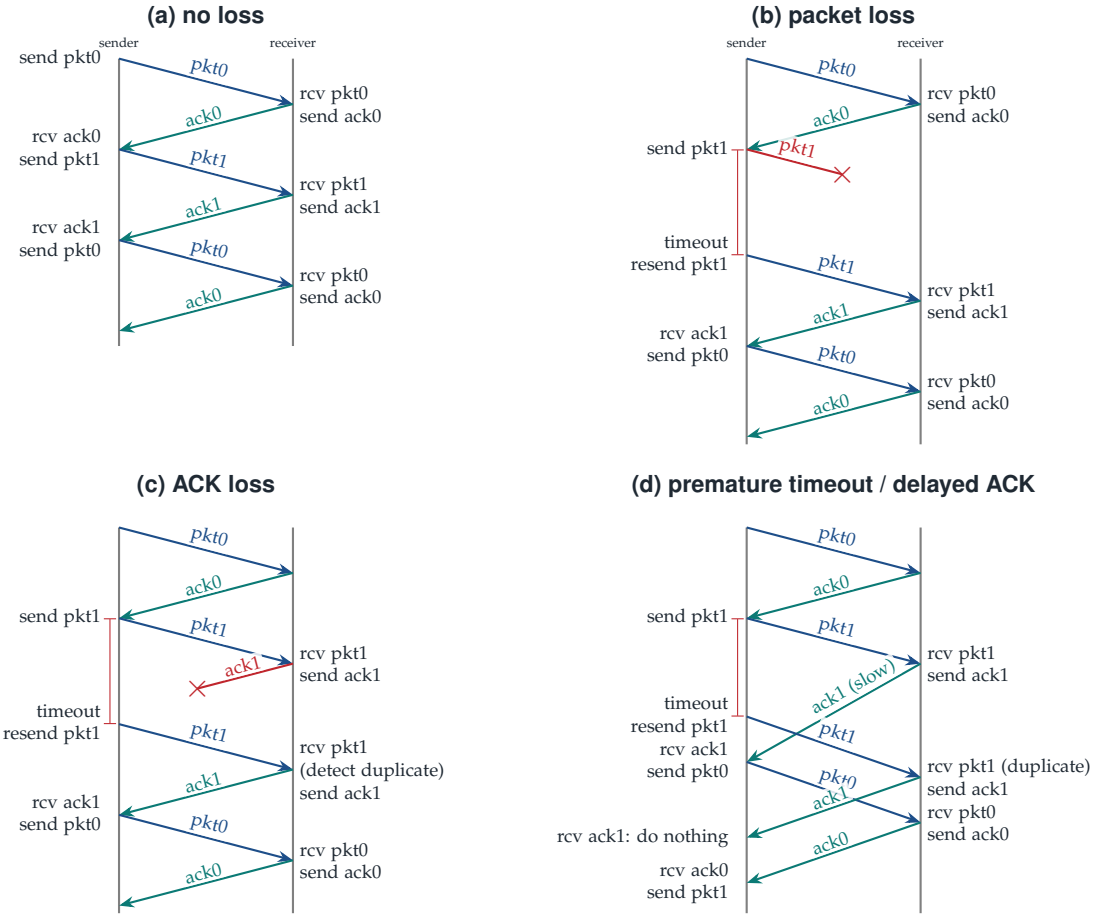


Figure 12: rdt3.0 (alternating-bit protocol) in operation. Red bars mark the timer. After TB Fig. 3.16 / SL 3-63, 3-64.

Assumptions. ACK transmission time ignored; ACK sent as soon as the last bit arrives; no loss, processing or queueing delay (including them makes things worse).

Common mistake. Omitting L/R from the denominator (negligible here, not in general); mixing bytes and bits; confusing transmission with propagation delay.

Worked example: Stop-and-wait coast to coast (TB §3.4.2; SL 3-65)

Given. $RTT \approx 30\,\text{ms}$ (15 ms one way); $R = 1\,\text{Gbps} = 10^9\,\text{bit/s}$; $L = 1000$ bytes $= 8000$ bits.

Required. (a) d_{trans}; (b) U; (c) throughput.

Calculation. $d_{\text{trans}} = 8000/10^9 = 8\,\mu\text{s} = 0.008\,\text{ms}$. The last bit arrives at $RTT/2 + L/R = 15.008\,\text{ms}$ and the ACK returns at 30.008 ms. $U = 0.008/30.008 = 0.00027$. Throughput $= 8000\,\text{bits}/30.008\,\text{ms} \approx 267\,\text{kbps}$.

Answer. $8\,\mu$s; $U \approx 0.00027$; ≈ 267 kbps.

Interpretation. The sender is busy 2.7 hundredths of one percent of the time "even though a 1 Gbps link was available!" — the protocol limits the performance of the underlying channel (SL 3-67).

4.8 Pipelining

Definition: Pipelining

Instead of stop-and-wait, the sender may send **multiple packets without waiting for acknowledgments**; the in-transit packets fill a "pipeline". SL 3-68: the sender allows multiple "in-flight", yet-to-be-acknowledged packets.

Consequences [TB §3.4.2; SL 3-68]:

1. The **range of sequence numbers must increase**: each in-transit packet (not counting retransmissions) needs

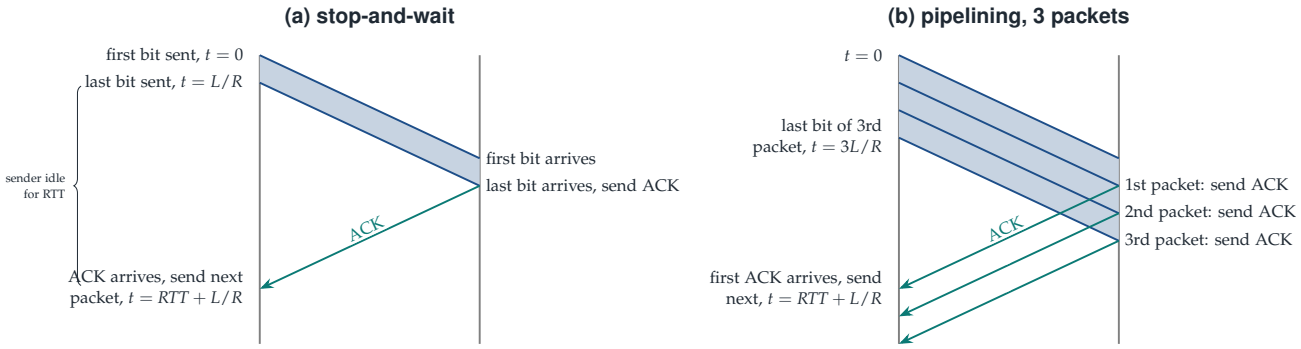


Figure 13: Stop-and-wait versus pipelined sending (transmission time exaggerated). After TB Fig. 3.18 / SL 3-66 to 3-69.

a unique number.

2. Sender and receiver may have to **buffer** more than one packet — at least the sent-but-unACKed packets at the sender; possibly correctly received packets at the receiver.
3. The required number range and buffering depend on how lost, corrupted and delayed packets are handled: **Go-Back-N** or **Selective Repeat** (Section 5).

Σ Formula: Utilisation with a window of N packets; window that fills the pipe[S]

$$U = \min\left\{1, \frac{N\,L/R}{RTT + L/R}\right\}, \qquad U = 1 \iff N\frac{L}{R} \geq RTT + \frac{L}{R} \iff N \geq 1 + \frac{R \cdot RTT}{L}.$$

Variables. N = packets the sender may have in flight (window, packets); $R \cdot RTT$ = **bandwidth–delay product (BDP)** in bits — the bits "in the pipe" during one round trip.

Derivation. N back-to-back packets take NL/R; the first ACK returns at $RTT + L/R$. If the window lasts at least that long, ACKs arrive before it is exhausted and the sender never idles.

Special cases. $N = 1$ gives stop-and-wait; $N = 3$ gives SL 3-69's $0.024/30.008 = 0.00081$.

Use. Round N up. The same BDP reappears as BBR's ideal in-flight data (§8.7).

Common mistakes. Reporting $U > 1$; forgetting the "+1".

Worked example: How large a window does the coast-to-coast link need?

Given. $R = 1\,\text{Gbps}$, $RTT = 30\,\text{ms}$, $L = 8000$ bits.

Required. (a) BDP; (b) minimum N for $U = 1$; (c) U with $N = 1000$.

Calculation. BDP $= 10^9 \times 0.030 = 3 \times 10^7$ bits $= 3.75\,\text{MB}$. $N \geq 1 + 3 \times 10^7/8000 = 3751$ packets. $U(1000) = 1000 \times 0.008/30.008 = 0.267$.

Answer. 3×10^7 bits; $N \geq 3751$; $U \approx 26.7\%$.

Interpretation. A thousandfold larger window gives exactly a thousandfold higher utilisation ($0.00027 \to 0.267$) until the pipe is full. Thousands of outstanding packets need a sequence space far larger than $\{0, 1\}$ — consequence (1).

	Stop-and-wait (rdt3.0)	Pipelining (GBN, SR, TCP)
Packets in flight	1	up to N (window)
Utilisation	$\frac{L/R}{RTT+L/R}$ — tiny on high-BDP paths	$\min\{1, \frac{NL/R}{RTT+L/R}\}$
Sequence numbers	1 bit (0/1)	k bits, window-limited (§5.4)
Buffering	one packet at the sender	up to N unACKed at the sender; possibly out-of-order at the receiver
Loss recovery	retransmit the one packet	policy choice: GBN or SR

Exam Focus

Memorise. Service model and the four primitives; FSM conventions; channel assumption and new mechanism of each rdt version (Table 4); ARQ's three capabilities; stop-and-wait; alternating-bit protocol; $U = (L/R)/(RTT + L/R)$ and

the 8 μs / 0.00027 / 267 kbps example; three consequences of pipelining.

Understand. Why rdt2.0 fails on corrupted ACK/NAKs; why one bit suffices; why a duplicate ACK can replace a NAK; why the sender alone handles loss; why rdt3.0 ignores wrong-numbered ACKs.

Diagrams. FSMs Figs. 8 to 11; the four timelines Fig. 12; Fig. 13.

Numerical patterns. U for stop-and-wait; window for $U = 1$; throughput; improvement factor with N packets.

Conceptual patterns. "Why was each rdt version needed?" "Draw the rdt3.0 receiver." "Trace rdt3.0 with a delayed ACK." "Why is stop-and-wait bad on high-BDP paths?"

5 Go-Back-N and Selective Repeat

[TB §3.4.3–§3.4.4, Figs. 3.19–3.27, Table 3.1; SL 3-70 to 3-78, 3-148, 3-149]

5.1 The shared framework

Both pipelined protocols allow up to N transmitted-but-unacknowledged packets. They differ in *error recovery*: Go-Back-N resends a whole window's worth; Selective Repeat resends only what is missing.

Definition: Sequence-number space

With a k-bit sequence-number field the range is $[0, 2^k - 1]$ and all sequence-number arithmetic is **modulo** 2^k: the space is a ring in which $2^k - 1$ is followed by 0. rdt3.0 had $k = 1$; TCP has a 32-bit field that counts *bytes*, not packets (§6.4). [TB §3.4.3]

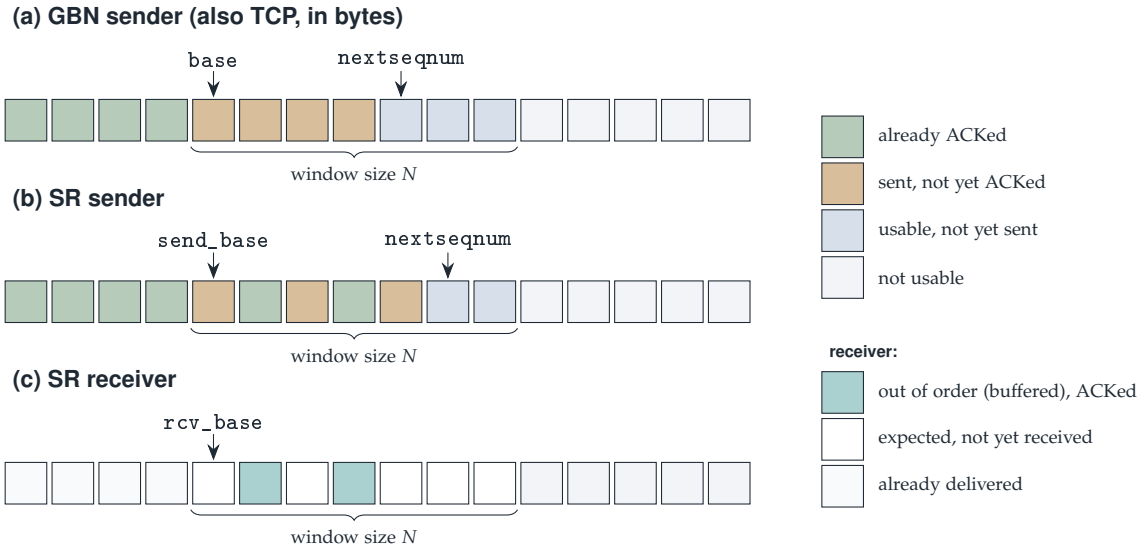


Figure 14: Sender's and receiver's views of sequence-number space. (a) GBN: every packet in the window from `base` up to `nextseqnum`−1 is outstanding. (b)–(c) SR: some packets inside the window are already individually ACKed/buffered, so sender and receiver windows need not coincide. After TB Figs. 3.19, 3.23 / SL 3-70, 3-74.

5.2 Go-Back-N (GBN)

Intuition. Keep the receiver as simple as possible: it accepts packets *only in order* and throws everything else away. The sender must therefore be ready to *go back* and resend everything from the first missing packet onward.

Definition: Go-Back-N (TB §3.4.3; SL 3-70)

The sender may transmit multiple packets without waiting for an acknowledgment, but may have no more than N unacknowledged packets in the pipeline (a window of up to N consecutive transmitted-but-unACKed packets; k-bit sequence numbers in the header).

Σ Formula: The four intervals of the GBN sender

With `base` = oldest unacknowledged sequence number and `nextseqnum` = smallest unused sequence number:

$[0,\ \text{base} - 1]$	transmitted and acknowledged
$[\text{base},\ \text{nextseqnum} - 1]$	sent, not yet acknowledged
$[\text{nextseqnum},\ \text{base} + N - 1]$	usable at once if data arrives from above
$\geq \text{base} + N$	not usable until packet base is ACKed

Invariant: $\text{nextseqnum} - \text{base} \leq N$ (packets in flight never exceed the window). The range of permissible outstanding numbers is a window of size N that slides forward: GBN is a **sliding-window protocol**. Why

limit N at all? Flow control and congestion control (§6.7, Section 8).

GBN sender event	Response [TB §3.4.3]
Invocation from above (`rdt_send`)	If the window is not full, create and send a packet and update variables. If full, return the data to the upper layer (an implicit "window full"); a real implementation would buffer it or use a semaphore/flag.
Receipt of ACK(n)	**Cumulative acknowledgment**: all packets up to *and including* n were correctly received. SL 3-70: move the window forward to begin at $n+1$.
Timeout	Resend **all** packets previously sent but not yet acknowledged — hence the name. A **single timer**, conceptually for the oldest unACKed packet: restarted when an ACK arrives and packets remain outstanding; stopped when none remain.

GBN sender (TB Fig. 3.20 / SL 3-148)

```
init:  base=1; nextseqnum=1
on rdt_send(data):
  if (nextseqnum < base+N) {
    sndpkt[nextseqnum]=make_pkt(nextseqnum,data,chksum)
    udt_send(sndpkt[nextseqnum])
    if (base==nextseqnum) start_timer
    nextseqnum++ }
  else refuse_data(data)
on timeout:
  start_timer
  udt_send(sndpkt[base]) ...
udt_send(sndpkt[nextseqnum-1])
on rdt_rcv(rcvpkt) && notcorrupt(rcvpkt):
  base=getacknum(rcvpkt)+1
  if (base==nextseqnum) stop_timer else start_timer
on rdt_rcv(rcvpkt) && corrupt(rcvpkt):  Λ
```

GBN receiver (TB Fig. 3.21 / SL 3-149)

```
init:  expectedseqnum=1
     sndpkt=make_pkt(0,ACK,chksum)
on rdt_rcv(rcvpkt) && notcorrupt(rcvpkt)
  && hasseqnum(rcvpkt,expectedseqnum):
  extract(rcvpkt,data)
  deliver_data(data)
  sndpkt=make_pkt(expectedseqnum,ACK,chksum)
  udt_send(sndpkt)
  expectedseqnum++
default (anything else):
  udt_send(sndpkt)   /* re-ACK last in-order packet */
```

Figure 15: GBN extended FSMs written as event handlers (each FSM has a single state "Wait"). An *extended* FSM adds variables, operations on them and conditional actions — close to a program; GBN is naturally implemented in event-based style.

Definition: GBN receiver (TB §3.4.3)

If packet n arrives correctly and **in order** (the last delivered packet was $n-1$), ACK n and deliver its data. In **all other cases** discard the packet and re-send an ACK for the most recently received in-order packet. Since delivery is one packet at a time and in order, receipt of k implies receipt of everything below it — so cumulative ACKs are natural. The receiver's only state is `expectedseqnum`.

Insight: Why throw away a correctly received packet?

The receiver must deliver in order. It could buffer $n+1$ while n is missing, but if n is lost both will be retransmitted anyway under the GBN timeout rule, so it simply discards $n+1$. Advantage: no receiver buffering and a one-variable receiver. Disadvantage: the discarded packet may itself be lost or garbled on retransmission, costing even more retransmissions.

Source & convention note: GBN receiver buffering: TB vs slides

TB §3.4.3 and SL 3-149 say the GBN receiver discards out-of-order packets ("no receiver buffering!") and remembers only `expectedseqnum`. SL 3-71 is looser: "can discard (don't buffer) or buffer: an implementation decision", naming the variable `rcv_base`. Both agree that the receiver re-ACKs the highest in-order packet and "may generate duplicate ACKs". **In answers, describe the TB receiver (discard) and mention buffering as an implementation option.**

5.3 Selective Repeat (SR)

Intuition. With large windows and bandwidth–delay products, a single error makes GBN resend many packets, most unnecessarily — "imagine, in our message-dictation scenario, that if every time a word was garbled,

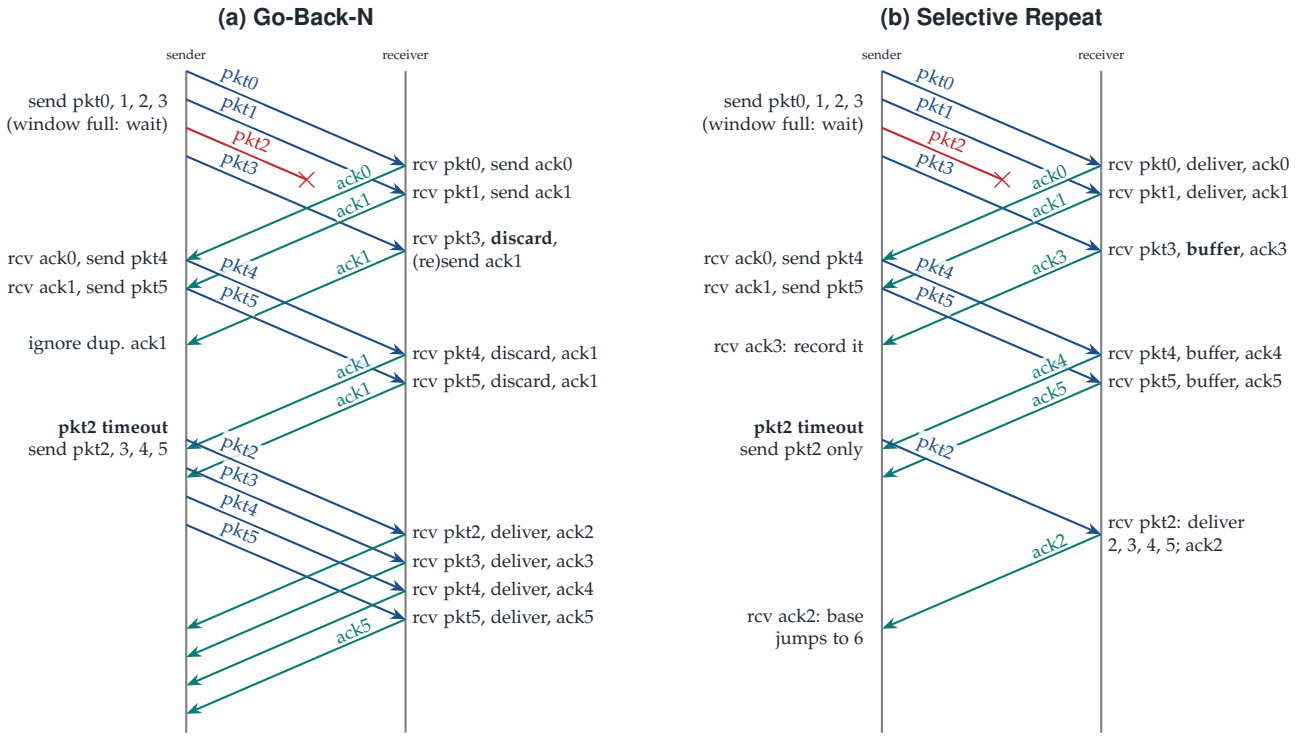


Figure 16: The same event — window $N = 4$, pkt2 lost — under GBN (TB Fig. 3.22 / SL 3-72) and SR (TB Fig. 3.26 / SL 3-76). GBN resends four packets; SR resends one.

the surrounding 1,000 words … had to be repeated." As the error rate rises the pipeline fills with needless retransmissions.

Definition: Selective Repeat (TB §3.4.4; SL 3-73)

The sender retransmits **only those packets it suspects were lost or corrupted**, which requires the receiver to **individually acknowledge** each correctly received packet and to **buffer** out-of-order packets for in-order delivery. A window of N still limits outstanding packets, but some packets inside it may already be ACKed. The sender keeps (conceptually) **one timer per unACKed packet**; on timeout it retransmits that single packet.

SR sender event	**Action** [TB Fig. 3.24; SL 3-75]
Data from above	If the next sequence number is inside the window, packetise and send; otherwise buffer or return it, as in GBN.
Timeout(n)	Resend packet n only and restart its timer. Each packet has its own *logical* timer; one hardware timer can mimic many.
ACK(n) in window	Mark n received. If $n =$ `send_base`, advance the base to the smallest unACKed number and transmit any untransmitted packets now inside the window.
SR receiver event	**Action** [TB Fig. 3.25; SL 3-75]
$n \in [\text{rcv_base},\ \text{rcv_base} + N - 1]$, correct	Send selective ACK(n); buffer if new. If $n =$ `rcv_base`, deliver it and all consecutively numbered buffered packets; advance the window by the number delivered.
$n \in [\text{rcv_base} - N,\ \text{rcv_base} - 1]$, correct	**ACK it anyway**, although it was ACKed before.
otherwise	ignore.

Insight: Why the receiver must re-ACK packets below its window

"The sender and receiver will not always have an identical view of what has been received correctly" — SR windows need not coincide. If the receiver ACKs `send_base` and that ACK is lost, the receiver's window moves on but the sender's does not; the sender times out and resends a packet now *below* the receiver's window. Without a fresh ACK the sender's window would never advance.

In Fig. 16b, SL 3-76 asks "what happens when ack2 arrives?": `send_base` is 2, packets 3, 4, 5 are already ACKed, so the base jumps straight to 6 and packets 6–9 become sendable.

5.4 The SR window-size dilemma

[TB §3.4.4, Fig. 3.27; SL 3-77, 3-78]

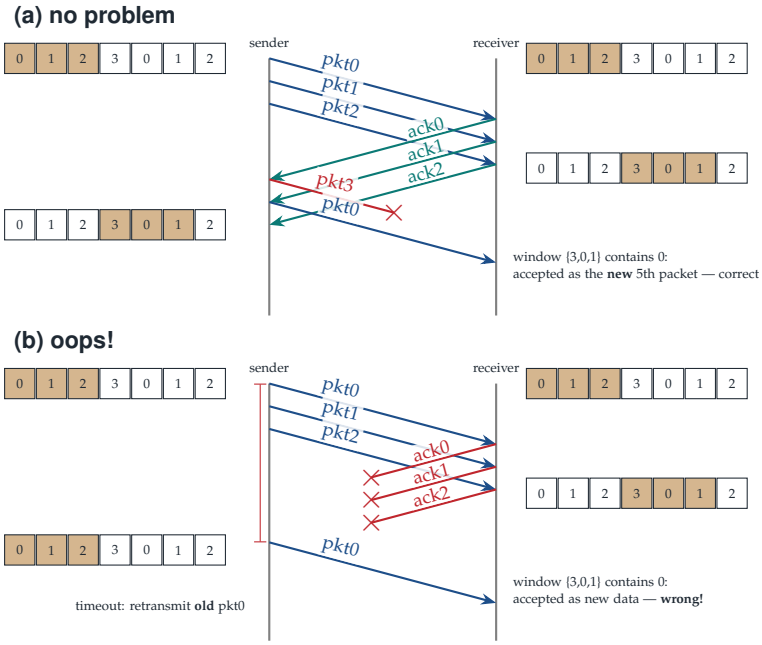


Figure 17: Sequence numbers $\{0, 1, 2, 3\}$ with an SR window of 3. The receiver sees identical behaviour in (a) and (b) and cannot tell a retransmission of the 1st packet from the 5th packet. Shaded cells are the current windows. After TB Fig. 3.27 / SL 3-77, 3-78.

Packets 0–2 are sent, received and ACKed, so the receiver's window moves over the 4th, 5th and 6th packets, numbered 3, 0, 1. In (a) the ACKs arrive, the sender sends packets 3 (lost), 4 (numbered 0) and 5; the new "0" is correctly accepted. In (b) all three ACKs are lost; the sender times out and resends the *old* packet 0, which the receiver — whose window contains 0 — accepts as the 5th packet. "There is no way of distinguishing the retransmission of the first packet from an original transmission of the fifth packet." SL 3-78: what relationship between sequence-number size and window size avoids this?

Σ Formula: Window-size bounds

$$N_{\text{SR}} \le \frac{2^k}{2} = 2^{k-1}, \qquad N_{\text{GBN}} \le 2^k - 1.$$

Source. TB §3.4.4: "the window size must be less than or equal to half the size of the sequence number space for SR protocols." The GBN bound is TB homework; the derivation below is the standard argument[S].

Derivation (SR). After the receiver has received and ACKed a full window, its window covers the *next* N numbers. If all those ACKs are lost, the sender's window still covers the *old* N. For the receiver to recognise any retransmission as old, old and new windows must not overlap on the ring of 2^k numbers: $N + N \le 2^k$.

Derivation (GBN). The GBN receiver's "window" is the single number `expectedseqnum`. After a full window of N packets it expects the number N positions ahead; confusion arises only if that equals the first number of the old window, i.e. $N \equiv 0 \pmod{2^k}$. So $N \le 2^k - 1$ suffices.

Instances. $k = 2$: $N_{\text{SR}} \le 2$ (so TB's $N = 3$ fails), $N_{\text{GBN}} \le 3$. $k = 3$: $N_{\text{SR}} \le 4$, $N_{\text{GBN}} \le 7$.

Common mistake. Quoting the GBN bound for SR, or vice versa.

5.5 Packet reordering and sequence-number reuse

When the "channel" is a network, packets can be reordered: old copies of a packet with number x can appear although neither window contains x — the network behaves as if it buffered packets and emitted them at any

later time. Since numbers are reused, a number is not reused until the sender is "sure" no earlier packet with that number remains in the network, by assuming a **maximum packet lifetime** — about **three minutes** in the TCP extensions for high-speed networks [RFC 7323]. [TB §3.4.4] (At high speed TCP's 32-bit space wraps much faster than that: Level-6 question 13.6.)

5.6 Comparing GBN and SR

Table 5: How GBN and SR respond to single faults (window $N = 4$, packets 0–5, everything else delivered).

Event	Go-Back-N	Selective Repeat
Packet 2 lost	Receiver discards 3, 4, 5 and re-sends ACK1 each time (sender ignores the duplicates). Single timer (oldest unACKed = 2) expires: resend 2, 3, 4, 5; delivered in turn.	Receiver buffers 3, 4, 5 and sends ACK3, 4, 5; sender records them. Pkt2's own timer expires: resend only 2; receiver delivers 2–5 together, ACK2.
ACK1 lost (pkts 0–3 arrive)	ACK2 arrives and, being cumulative, covers 0, 1, 2: base $\leftarrow 3$. Nothing retransmitted — the lost ACK is harmless.	ACK2 marks only pkt2; pkt1 still unACKed, `send_base` stays 1. Pkt1's timer expires $\Rightarrow$ resend; receiver (pkt1 now in $[\text{rcv_base} - N, \text{rcv_base} - 1]$) re-ACKs without delivering; the base then jumps past all ACKed packets.
Packet 2 corrupted	Checksum fails: treated as any unexpected packet — discard, re-send ACK1; then as row 1.	Checksum fails: ignored (no ACK); then as row 1 — only pkt2 resent.
Pkt3 arrives before pkt2 (reordering only)	Pkt3 discarded, ACK1 re-sent; pkt2 then delivered (ACK2), but pkt3 is gone: the sender times out on 3 and resends 3 and everything after it.	Pkt3 buffered and ACKed; pkt2 arrives, 2 and 3 delivered together; no retransmission.

Worked example: Counting transmissions after one loss

Given. 10 packets (0–9), $N = 4$; packet 3 lost on its first transmission; all else (including ACKs) delivered; the timeout for 3 fires when 3, 4, 5, 6 are outstanding.

Required. Total data-packet transmissions, GBN and SR.

Calculation. GBN: 10 originals + resend of 3, 4, 5, 6 (the receiver discarded 4–6) $= 14$. SR: 10 + resend of 3 only $= 11$.

Answer. GBN 14, SR 11.

Interpretation. One loss costs GBN up to N retransmissions and SR exactly one; the gap grows linearly with N — so GBN is most wasteful precisely on high-BDP paths, where large windows are needed.

Worked example: Choosing a window for a given sequence field

Given. A 3-bit sequence field; a path needing 6 packets in flight to fill the pipe.

Formula. $N_{\text{GBN}} \leq 2^k - 1$, $N_{\text{SR}} \leq 2^{k-1}$.

Calculation. $2^3 - 1 = 7$; $2^2 = 4$.

Answer. GBN up to 7, SR up to 4; only GBN can fill the 6-packet pipe; SR needs $k \geq 4$ ($N \leq 8$).

Interpretation. SR's economy in retransmissions is paid for in sequence-number space: one extra bit for the same window.

Exam Focus

Memorise. GBN's four intervals, `base/nextseqnum`, three sender events, receiver rule (deliver in order, else discard and re-ACK), cumulative ACK(n), single timer; SR's three sender and three receiver events, per-packet timers, `send_base/rcv_base`; the SR dilemma with $\{0, 1, 2, 3\}$, $N = 3$; $N_{\text{SR}} \leq 2^{k-1}$, $N_{\text{GBN}} \leq 2^k - 1$; maximum packet lifetime ≈ 3 min; Table 7.

Understand. Why GBN discards and resends a window; why SR re-ACKs below its window; why windows need not coincide; why the SR window is at most half the space; why cumulative ACKs make lost ACKs cheap.

Diagrams. Figs. 14, 16 and 17; GBN extended FSMs (Fig. 15).

Table 6: Go-Back-N versus Selective Repeat — the detailed comparison (referred to from elsewhere).

Feature	Go-Back-N	Selective Repeat
Sender window	up to N consecutive unACKed packets	up to N consecutive numbers; some may be ACKed
Receiver window	effectively 1 (`expectedseqnum`)	N (`rcv_base` ... `rcv_base`$+N-1$)
Sequence-space limit	$N \leq 2^k - 1$	$N \leq 2^{k-1}$
ACKs	cumulative: ACK(n) covers all $\leq n$	individual: ACK(n) covers n only
Duplicate ACKs	generated (re-ACK highest in-order)	not used for signalling; old packets below the window are re-ACKed
Timers	one, for the oldest unACKed packet	one logical timer per unACKed packet
On timeout	resend all sent-but-unACKed packets	resend only the timed-out packet
Out-of-order packet	discarded; re-ACK last in-order	buffered and ACKed
Receiver buffering	none	up to N packets
Delivery to application	one packet at a time, in order	in-order batches when gaps fill
Lost ACK	usually harmless (next cumulative ACK covers it)	costs a retransmission of that packet
Strength / weakness	simple receiver / many needless retransmissions when errors are frequent or windows large	few retransmissions / more complex sender and receiver, buffering, smaller window

Table 7: Summary of reliable-data-transfer mechanisms (TB Table 3.1).

Mechanism	Use, comments
Checksum	Detect bit errors in a transmitted packet.
Timer	Timeout/retransmit a packet, possibly because it or its ACK was lost. Because timeouts also occur when a packet is merely delayed (premature timeout) or its ACK was lost, the receiver may receive duplicates.
Sequence number	Number data packets in order. Gaps reveal a lost packet; repeated numbers reveal a duplicate.
Acknowledgment	Tell the sender a packet or set of packets arrived correctly; typically carries the sequence number(s); individual or cumulative.
Negative ACK	Tell the sender a packet was not received correctly; typically carries its sequence number.
Window, pipelining	Restrict the sender to a range of sequence numbers; allowing several unACKed packets raises utilisation over stop-and-wait. The window may be set by the receiver's ability to buffer, the network's congestion, or both.

Question patterns. Maximum window for given k; count transmissions after a loss; trace windows and ACKs; which packets are resent at the timeout; "compare GBN and SR"; "explain the SR dilemma"; "why must numbers not be reused too soon?"

6 Connection-Oriented Transport: TCP

[TB §3.5, Figs. 3.28–3.42; SL 3-79 to 3-106, 3-150 to 3-152; RFCs 793, 1122, 2018, 5681, 7323]

TCP "relies on many of the underlying principles discussed in the previous section, including error detection, retransmissions, cumulative acknowledgments, timers, and header fields for sequence and acknowledgment numbers" [TB §3.5]. This section shows how those principles are instantiated.

6.1 TCP at a glance

Table 8: TCP's defining properties (SL 3-80; TB §3.5; the class summary lists the same set). Almost every "describe TCP" question is marked against this list.

Property	Precise meaning
Point-to-point	One sender, one receiver; no multicast in a single send. "With TCP, two hosts are company and three are a crowd."
Connection-oriented	Handshaking (control messages) initialises sender and receiver state before data flows. The connection is *logical*: state lives only in the two end systems; it is not a TDM/FDM circuit, and routers "are completely oblivious to TCP connections; they see datagrams, not connections."
Reliable, in-order byte stream	What a process reads from its receive buffer is uncorrupted, without gaps, without duplication and in sequence. No message boundaries.
Full duplex	Data flows in both directions at once in the same connection.
Pipelined	Many unACKed segments outstanding; congestion and flow control set the window.
Cumulative ACKs	An ACK number acknowledges all bytes *before* it.
Flow- and congestion-controlled	The sender will not overwhelm the receiver (§6.7) or the network (Section 8).
MSS	The maximum segment size bounds the application data per segment.

"Connection-oriented" and "reliable" are *independent* properties (QUIC is both, over connectionless UDP); define them separately in answers.

6.2 The TCP connection, MSS and MTU

The **client** initiates, the **server** responds: `clientSocket.connect((serverName,serverPort))` triggers a three-way handshake (two segments without payload, a third that may carry payload; §6.8.1). Afterwards the client passes a byte stream through the socket into the connection's **send buffer**; TCP takes chunks from it "at its own convenience" (RFC 793) and passes segments to IP. A connection is therefore buffers, variables and a socket in each of two hosts — nothing in between.

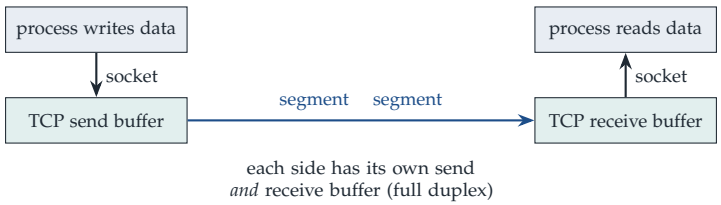


Figure 18: TCP send and receive buffers. After TB Fig. 3.28.

Definition: MSS and MTU

MSS (maximum segment size): the maximum amount of *application-layer data* in a segment — not the segment size including headers ("confusing, but we have to live with it").
MTU (maximum transmission unit): the length of the largest link-layer frame the local sending host can send. [TB §3.5.1]

Σ Formula: Setting the MSS

$$MSS = \text{MTU} - (\text{IP header} + \text{TCP header}) = \text{MTU} - 40 \text{ bytes (typically)}, \qquad \text{segments} = \lceil F/MSS \rceil.$$

Meaning. The segment plus TCP/IP headers (20+20) must fit one link-layer frame. Ethernet and PPP have MTU 1500 $\Rightarrow$ $MSS = 1460$. Path-MTU discovery [RFC 1191] can set it from the smallest MTU on the path.

Common mistakes. Counting headers in the MSS; subtracting only 20; confusing MSS (one segment) with the window (bytes in flight).

Worked example: Segmenting a file (TB Fig. 3.30)

Given. $F = 500\,000$ bytes, $MSS = 1000$, first byte numbered 0.

Calculation. $\lceil 500\,000/1000 \rceil = 500$ segments; segment i ($i = 0, 1, \ldots$) has sequence number ISN $+ i \cdot MSS$: 0, 1000, 2000, ...; the last carries bytes 499 000–499 999.

Interpretation. If F is not a multiple of the MSS the last chunk is smaller. Interactive applications send far less than an MSS: a Telnet/ssh segment is often 21 bytes (20-byte header + 1 data byte).

6.3 TCP segment structure

[TB §3.5.2, Fig. 3.29; SL 3-81]

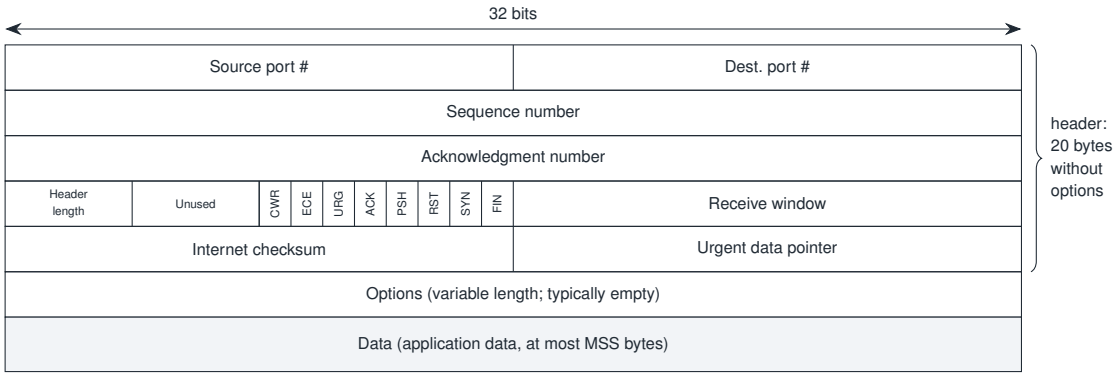


Figure 19: TCP segment structure (the class drawing of row 4 is "H LEN | NU | C | E | U | A | P | R | S | F | Receive window"). After TB Fig. 3.29 / SL 3-81.

Field	Size	Purpose; sender and receiver behaviour
Source / dest. port	16 + 16 bits	Multiplexing; two of the four demultiplexing values.
Sequence number	32 bits	Byte-stream number of the first data byte in the segment; receiver uses it to order data, detect gaps and discard duplicates.
Acknowledgment number	32 bits	Sequence number of the next byte expected from the other side (cumulative); advances the peer's `SendBase`.
Header length	4 bits	Header length in 32-bit words (options make it variable): typically 5 (20 bytes), at most 15 (60 bytes).
Unused	4 bits	Reserved ("not used").
CWR, ECE	1 bit each	Explicit congestion notification (§8.8): receiver sets ECE; sender halves *cwnd* and sets CWR.
URG, PSH	1 bit each	Urgent data present; pass data up immediately. "In practice, PSH, URG, and the urgent data pointer are not used."
ACK	1 bit	The acknowledgment field is valid (0 in the SYN; 1 in the SYNACK and afterwards).
RST, SYN, FIN	1 bit each	Connection setup/teardown (§6.8); SYN and FIN each consume one sequence number.
Receive window	16 bits	Flow control: bytes the receiver is willing to accept (§6.7).
Internet checksum	16 bits	Error detection, computed as for UDP.
Urgent data pointer	16 bits	Last byte of urgent data.
Options	variable	MSS negotiation; window-scaling factor for high-speed networks; time-stamping (RFC 854, 1323).

Source & convention note: How many flag bits?

TB's text says the flag field has 6 bits (URG, ACK, PSH, RST, SYN, FIN), while its own Fig. 3.29 and SL 3-81 also draw CWR and ECE. Source-faithful answer: **six classic flags plus the two ECN bits**.

Σ Formula: Header length and receive-window limits

$$\text{header bytes} = 4 \times (\text{header-length field}), \quad 5 \le \text{field} \le 15 \;\Rightarrow\; 20\text{–}60 \text{ bytes};$$

$$rwnd_{\max} = 2^{16} - 1 = 65\,535 \text{ bytes (no window scaling).}$$

Worked instance[S]. On a 1 Gbps path with $RTT = 100$ ms the BDP is $10^9 \times 0.1 = 10^8$ bits $= 12.5$ MB, but an unscaled window allows at most $65\,535 \times 8/0.1 \approx 5.24$ Mbit/s — 0.5% of the link. Hence the **window-scaling option**.

Common mistake. Reading the header-length field as a byte count.

6.4 Sequence numbers and acknowledgment numbers

[TB §3.5.2; SL 3-82, 3-83] — the part of TCP that generates the most exam marks.

Definition: Sequence and acknowledgment numbers

TCP views data as an unstructured, ordered **stream of bytes**; sequence numbers are over the bytes, not the segments. The **sequence number** of a segment is the byte-stream number of its *first data byte*.
The **acknowledgment number** Host A puts in its segment is the sequence number of the *next byte A expects* from B. Because TCP acknowledges only up to the first missing byte, TCP provides **cumulative acknowledgments**.

The sender's view is the sliding window of Fig. 14(a) with bytes instead of packets: `SendBase` (oldest unACKed byte) and `NextSeqNum`, window $= \min\{cwnd, rwnd\}$ (§8.1.1).

Insight: Why number bytes rather than segments?

(1) The service is a byte stream with no message boundaries; numbering segments would expose structure the application is promised not to see. (2) Re-segmentation is legal: a retransmission need not have the

original boundaries. (3) Segment sizes vary (1-byte Telnet, 1460-byte bulk) under one scheme. (4) Partial acknowledgment ("I have bytes 0–535") is expressible only with byte numbers. (An explanation assembled from the sources' statements.)

Σ Formula: Sequence/ACK arithmetic; SYN and FIN

For a segment with sequence number Seq carrying ℓ data bytes:

$$\text{bytes carried} = [\,Seq,\ Seq + \ell - 1\,], \qquad Seq_{\text{next}} = Seq + \ell,$$

$$\text{ACK} = Seq + \ell \quad \text{(if everything so far has arrived in order).}$$

In general ACK = the **first missing byte**. Arithmetic is modulo 2^{32}. Header bytes are not numbered; a pure ACK ($\ell = 0$) does not advance the sequence number.

SYN and FIN. Each **consumes exactly one sequence number** although it carries no data: the SYNACK's ACK is client_isn + 1; the third handshake segment's ACK is server_isn + 1; the ACK of a FIN with number x is $x + 1$. A control segment that occupied no sequence space could not be acknowledged or retransmitted reliably.

Common mistakes. ACK = $Seq + \ell - 1$ (the ACK is the next byte, not the last received); counting the 20 header bytes in ℓ; advancing Seq for a pure ACK; writing the SYNACK's ACK as client_isn.

Initial sequence numbers. Examples often start at 0, but both sides choose their **ISN randomly**, to minimise the chance that a segment still in the network from an earlier, terminated connection between the same hosts and ports is mistaken for a valid segment of a later one [Sunshine 1978]. (It also hinders ISN-guessing attacks.)

Two directions, two numberings. A full-duplex connection carries two independent byte streams, each with its own ISN. A segment's Seq belongs to the stream it travels in; its ACK belongs to the opposite stream. Draw one number line per direction before tracing.

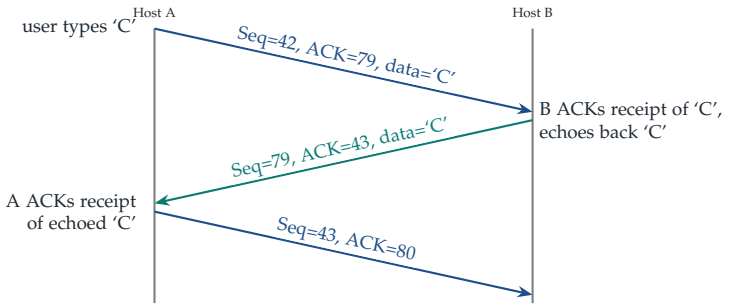


Figure 20: Telnet over TCP; initial sequence numbers 42 (client) and 79 (server). Also worked in class. After TB Fig. 3.31 / SL 3-83.

Worked example: Deriving every number in the Telnet figure

Segment 1 (A→B). Seq = 42 (A's first data byte); ACK = 79 (A has nothing from B yet and waits for byte 79); one data byte, 'C'.

Segment 2 (B→A). ACK = 42 + 1 = 43 (B has everything through 42); Seq = 79 (B's first data byte). The ACK rides on a data segment: it is **piggybacked**.

Segment 3 (A→B). ACK = 80 (A has received through 79). Seq = 43: the segment carries no data but the field must hold something, and 43 is the next byte A would send.

Answer. (42, 79), (79, 43), (43, 80).

Interpretation. A pure ACK's sequence number does not advance; piggybacking costs nothing.

Worked example: Cumulative ACK and the gap (TB §3.5.2)

Given. A holds bytes 0–535 and 900–1000 from B; 536–899 are missing.

Required. A's next ACK number.

Answer. ACK = 536, the first missing byte; bytes 900–1000 are invisible to a cumulative ACK.

Interpretation. What to do with out-of-order bytes is not specified by the RFCs ("up to implementor", SL 3-82): discard them (simpler receiver) or keep them and wait for the gap to fill (more bandwidth-efficient, and the approach taken in practice).

Worked example: Segments of different sizes, with a loss

Given. A sends back-to-back segments of 100, 200, 150, 250 bytes; the first has Seq 5000; the second is lost; B buffers out-of-order data.

Required. Each Seq; B's ACK after each arrival; A's recovery.

Calculation.

Seg	Seq	ℓ	Bytes	Fate	ACK from B
1	5000	100	5000–5099	arrives	5100
2	5100	200	5100–5299	**lost**	—
3	5300	150	5300–5449	arrives out of order	5100 (duplicate)
4	5450	250	5450–5699	arrives out of order	5100 (duplicate)

Answer. Two duplicate ACKs — not enough for fast retransmit (needs three, §6.6.1); A waits for the timeout on Seq 5100 and resends that segment only.

Interpretation. A fifth segment (Seq 5700, 100 bytes) would have produced a third duplicate and an immediate fast retransmit. When segment 2 arrives, B's ACK jumps to $5700 + 100 = 5800$: one cumulative ACK covers everything buffered.

6.5 Round-trip time estimation and timeout

[TB §3.5.3, Fig. 3.32; SL 3-84 to 3-86]

The problem. The timeout must exceed the RTT, but the RTT varies. Too short ⇒ premature timeouts and unnecessary retransmissions; too long ⇒ slow reaction to loss (SL 3-84). How large a margin? One timer per segment?

Definition: SampleRTT

The time from when a segment is sent (passed to IP) until its acknowledgment is received. Most implementations measure **one SampleRTT at a time** (for one outstanding segment), giving a new value about once per RTT, and **never for a retransmitted segment** [Karn 1987] (SL 3-84: "ignore retransmissions"). [TB §3.5.3]

Insight: Why retransmitted segments are never sampled (Karn)

A segment sent at t_0 times out, is resent at t_1, and an ACK arrives at t_2. The sender cannot tell whether the ACK answers the original ($t_2 - t_0$) or the retransmission ($t_2 - t_1$). Choosing $t_2 - t_1$ when the ACK was for the original makes the RTT look too short, shrinking the timeout and causing more spurious retransmissions exactly when the network is struggling. Refusing to sample removes the ambiguity.

Σ Formula: EstimatedRTT, DevRTT, TimeoutInterval

$$\begin{aligned} EstimatedRTT &\leftarrow (1-\alpha)\,EstimatedRTT + \alpha\,SampleRTT, && \alpha = 0.125 \ \text{[RFC 6298]} \\ DevRTT &\leftarrow (1-\beta)\,DevRTT + \beta\,\left|SampleRTT - EstimatedRTT\right|, && \beta = 0.25 \\ TimeoutInterval &= EstimatedRTT + 4\,DevRTT, && \text{initial } TimeoutInterval = 1\text{ s} \end{aligned}$$

Meaning. *EstimatedRTT* is an **exponential weighted moving average (EWMA)** of the samples; *DevRTT* is an EWMA of the *absolute deviation* (small when samples are steady, large when they fluctuate); the timeout is the estimate plus a safety margin that grows with variability (SL 3-86). Written as programming statements: new value from old.

Units. Seconds (ms in practice).

When. On each new SampleRTT (about once per RTT), never from a retransmitted segment.

Order of updates. TB lists *EstimatedRTT* first, then *DevRTT*; these notes use the *just-updated EstimatedRTT* inside $|\cdot|$. (*Supplementary*: RFC 6298 updates the variation first with the old estimate; the difference is small — state your convention.)

Common mistakes. Swapping the weights (old estimate gets 0.875); $\beta = 0.125$; dropping the absolute value; squaring (it is not a variance); a coefficient other than 4; multiplying instead of adding.

Insight: Why "exponential": deriving the weights

Write E_n for the estimate after sample S_n. Expanding the recursion,

$$E_n = \alpha S_n + (1-\alpha)E_{n-1} = \alpha S_n + \alpha(1-\alpha)S_{n-1} + (1-\alpha)^2 E_{n-2} = \cdots = \alpha \sum_{k=0}^{n-1} (1-\alpha)^k S_{n-k} + (1-\alpha)^n E_0.$$

The weight on the sample taken k updates ago is $\alpha(1-\alpha)^k$ — geometric, i.e. **exponentially decaying** (SL 3-85: "influence of past sample decreases exponentially fast"). With $\alpha = 0.125$: 0.125, 0.109, 0.096, 0.084, …; eight updates later $0.125 \times 0.875^8 \approx 0.043$. The weights sum to $\alpha \sum_k (1-\alpha)^k = 1$, so it is a true average. Small α: smooth but slow to track change; large α: jumpy, close to "the last sample". Recent samples get more weight because they better reflect current congestion.

Σ Formula: Exponential backoff: doubling the timeout

$$\text{on each timeout:} \quad \textit{TimeoutInterval} \leftarrow 2 \times \textit{TimeoutInterval}.$$

Rule (TB §3.5.4). On a timeout TCP retransmits the unACKed segment with the smallest sequence number and sets the next interval to *twice* the previous value instead of recomputing it; intervals grow exponentially over successive retransmissions (e.g. 0.75 → 1.5 → 3.0 s). When the timer is next started by new data or an ACK, *TimeoutInterval* is again computed from the latest *EstimatedRTT* and *DevRTT*.

Why. "A limited form of congestion control": a timeout is most likely caused by congestion, and persistent retransmission would worsen it; "TCP acts more politely, with each sender retransmitting after longer and longer intervals."

Worked example: How the three quantities evolve over several samples

Given. Initially *EstimatedRTT* = 100 ms, *DevRTT* = 10 ms; samples 120, 80, 200, 90, 100 ms; $\alpha = 0.125$, $\beta = 0.25$.

Formulas. Estimate, then deviation (with the new estimate), then timeout.

Step 1 in full. $\textit{EstimatedRTT} = 0.875(100) + 0.125(120) = 87.5 + 15 = 102.5$; $|120 - 102.5| = 17.5$; $\textit{DevRTT} = 0.75(10) + 0.25(17.5) = 7.5 + 4.375 = 11.875$; $\textit{TimeoutInterval} = 102.5 + 4(11.875) = 150.0$ ms.

Calculation. (ms)

Step	SampleRTT	EstimatedRTT	|Sample − Est.|	DevRTT	TimeoutInterval
0	—	100.00	—	10.00	140.00
1	120	102.50	17.50	11.88	150.00
2	80	99.69	19.69	13.83	155.00
3	200	112.23	87.77	32.31	241.48
4	90	109.45	19.45	29.10	225.84
5	100	108.27	8.27	23.89	203.83

Interpretation. (i) A 20 ms-high sample moved the estimate by only 2.5 ms — the filter is deliberately sluggish. (ii) The 200 ms outlier raised *EstimatedRTT* by ≈ 12.5 ms but *TimeoutInterval* by ≈ 86 ms: the *margin*, not the estimate, absorbs the shock. (iii) The timeout then falls only gradually (DevRTT decays by 0.75 per sample): TCP stays cautious for several RTTs. (iv) A *low* sample (step 2) also raises the timeout, because any deviation widens the margin. (v) At every step *TimeoutInterval* > *SampleRTT*: no spurious timeout.

Worked example: Why the timeout must not simply equal the latest RTT

Given. The same samples, but the naive rule *TimeoutInterval* = most recent *SampleRTT*.

Reasoning. After the 80 ms sample the timeout is 80 ms; the next segment takes 200 ms, so the timer fires prematurely. Consequences: (1) an unnecessary retransmission on a path already showing queueing; (2) that segment can no longer be sampled (Karn), starving the estimator; (3) the timeout is doubled; (4) in congestion control the timeout is a loss event: *ssthresh* halved and *cwnd* → 1 MSS (Section 8) — a heavy penalty for a segment that was never lost.

Interpretation. *EstimatedRTT* + 4*DevRTT* balances SL 3-84's two failure modes: *EstimatedRTT* tracks the centre (timeout not grossly long) and 4*DevRTT* the spread (not too short on a jittery path).

6.6 TCP reliable data transfer

[TB §3.5.4, Figs. 3.33–3.37, Table 3.2; SL 3-87 to 3-91, 3-150]

TCP builds reliable transfer on IP's best-effort service: the byte stream a process reads is "exactly the same byte stream that was sent". Simplifying assumptions for this section: no flow/congestion limit, data from above smaller than an MSS, one direction.

Table 9: The simplified TCP sender (TB Fig. 3.33; SL 3-87, 3-150).

Event	Action
Data from the application	Create a segment with sequence number `NextSeqNum`; if the timer is not running, start it (think of it as the timer for the *oldest* unACKed segment; interval *TimeoutInterval*); pass to IP; `NextSeqNum += length(data)`.
Timeout	Retransmit the not-yet-ACKed segment with the **smallest** sequence number; restart the timer.
ACK with value y	`if (y > SendBase) { SendBase = y; if (any unACKed segments) start timer }` — cumulative: all bytes below y are acknowledged. (Else: duplicate ACK, see fast retransmit.)

`SendBase` is the oldest unACKed byte, so `SendBase`-1 is the last byte known to be received correctly and in order; `NextSeqNum` is the next byte to send; both start at the ISN. It would be simplest to keep a timer per segment, but timer management is costly, so the recommended procedure [RFC 6298] uses a **single retransmission timer**; hence a TCP timeout resends only *one* segment, the oldest.

Table 10: TCP ACK-generation recommendation [RFC 5681] (TB Table 3.2; SL 3-88).

Event at the receiver	TCP receiver action
In-order segment with the expected number; all earlier data already ACKed	**Delayed ACK**: wait up to 500 ms for another in-order segment; if none arrives, send an ACK.
In-order segment with the expected number; one other in-order segment waiting for its ACK	Immediately send one cumulative ACK covering both.
Out-of-order segment with a higher-than-expected number (gap detected)	Immediately send a **duplicate ACK** indicating the next expected byte (the lower end of the gap).
Segment that partially or completely fills a gap	Immediately send an ACK, provided the segment starts at the lower end of the gap.

Insight: What delayed ACKs buy and cost

Rows 1–2 halve ACK traffic in bulk transfer (one ACK per two segments). The cost: *cwnd* grows per ACK, so fewer ACKs mean slower slow-start growth, and an isolated segment's ACK may wait up to 500 ms. Rows 3–4 switch the delay off: anything about a gap is urgent. (Inference from the sources' rules.)

Worked example: What does each side know? — the premature timeout, scenario (2)

Given. A sent Seq 92 (8 bytes) and Seq 100 (20 bytes); both arrived; B sent ACK 100 and ACK 120; neither has reached A when the timer for 92 expires.

Trace. B knows it holds 92–119 in order and has sent two ACKs, but not whether they arrived. A knows

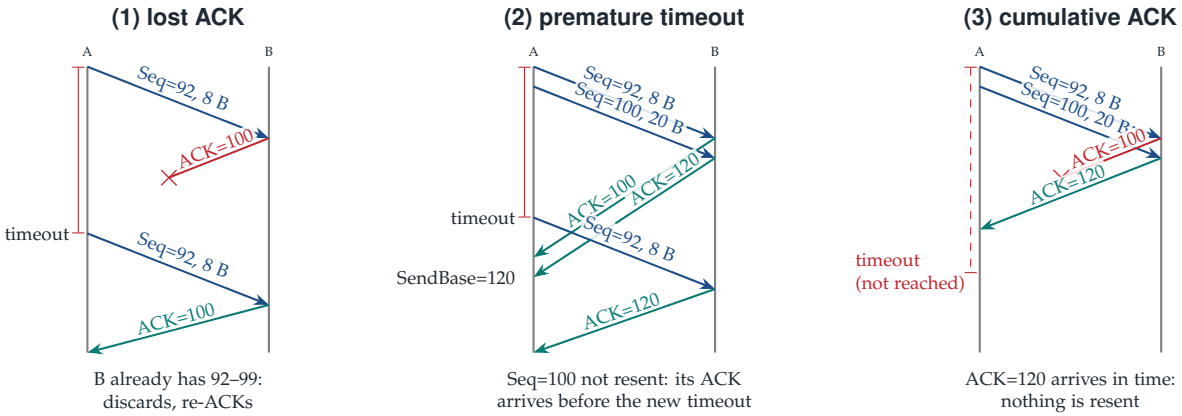


Figure 21: TCP retransmission scenarios with the canonical numbers. After TB Figs. 3.34–3.36 / SL 3-89, 3-90. In class the same scenarios were drawn with two 8-byte segments, so the second ACK is $100 + 8 = 108$ instead of 120 — same rule, ACK $= Seq + \ell$.

`SendBase` $= 92$, `NextSeqNum` $= 120$ and that the timer fired; it cannot distinguish segment lost, segment corrupted, ACK lost, ACK slow. A resends only the oldest segment (92) and restarts the timer; "as long as the ACK for the second segment arrives before the new timeout, the second segment will not be retransmitted." B recognises 92–99 as old, discards them and ACKs 120. ACK 120 $>$ `SendBase` $\Rightarrow$ `SendBase` $= 120$; nothing outstanding; timer stopped.

Answer. One unnecessary 8-byte retransmission; nothing delivered twice. TCP cannot tell loss from delay; the sender's response is always to retransmit the segment in question.

6.6.1 Fast retransmit

Definition: Duplicate ACK; fast retransmit (TB §3.5.4; SL 3-91)

A **duplicate ACK** re-acknowledges data for which the sender already has an ACK. Since TCP has no NAKs, a receiver that detects a gap simply re-ACKs the last in-order byte.
Timeouts can be long, delaying recovery. Because segments are usually sent back to back, one loss typically causes many duplicate ACKs. On **three duplicate ACKs** for the same data the sender assumes the following segment was lost and performs a **fast retransmit**: it resends the unACKed segment with the smallest sequence number *before its timer expires* [RFC 5681]. SL 3-91: three duplicates mean "3 segments received after a missing segment — lost segment is likely. So retransmit!"

```
event:  ACK received, with ACK field value y
  if (y > SendBase) {
    SendBase = y
    if (any not-yet-acknowledged segments)
      start timer
  }
  else { /* duplicate ACK */
    increment count of dup ACKs for y
    if (count of dup ACKs for y == 3)
      /* TCP fast retransmit */
      resend segment with sequence number y
  }
```

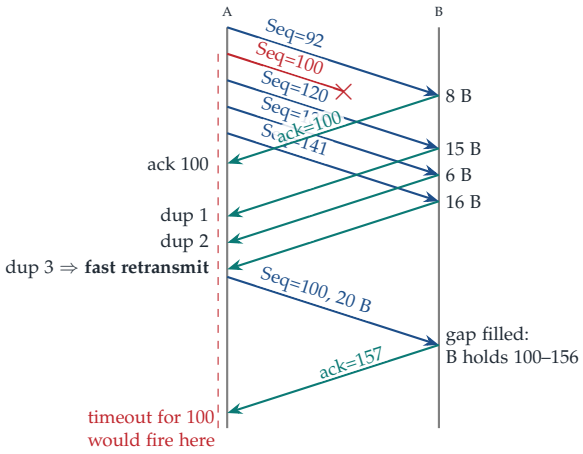


Figure 22: The ACK-received handler with fast retransmit, and its timeline: the three duplicate ACK 100s trigger a resend long before the timer; when the gap fills, B's cumulative ACK jumps to 157. After TB Fig. 3.37 / SL 3-91.

Insight: Why three duplicates and not one?

The network may reorder packets; one out-of-order arrival — one duplicate ACK — is more likely harmless reordering than loss. Three duplicates mean at least three later segments arrived while the gap persisted: strong evidence of loss. A lower threshold recovers sooner but retransmits spuriously more often. Counting convention: TB §3.7.1 speaks of "four acknowledgments for a given segment (one original ACK and then three duplicate ACKs)". The congestion-control consequences of a timeout versus three duplicates are compared in Table 13.

Is TCP Go-Back-N or Selective Repeat?. [TB §3.5.4]

- **GBN-like**: cumulative ACKs; out-of-order segments are not individually ACKed; the sender keeps only `SendBase` and `NextSeqNum`.
- **Not GBN**: many implementations buffer out-of-order segments; and if segments $1..N$ all arrive but the ACK for $n < N$ is lost while the others arrive before their timeouts, GBN would resend $n, \ldots, N$, whereas TCP resends at most segment n — and not even that if the ACK for $n+1$ arrives before n's timeout.
- **SACK** [RFC 2018]: selective acknowledgment of out-of-order segments, combined with selective retransmission, makes TCP look much like SR.

Verdict: TCP's error recovery is best categorised as a **hybrid of GBN and SR**.

Exam Focus: TCP segments, numbers, RTT and reliability

Memorise. Table 8; header fields and sizes; $MSS = \text{MTU} - 40 = 1460$; Seq = first data byte, ACK = next byte expected, SYN/FIN consume one number; the three RTT formulas with $\alpha = 0.125$, $\beta = 0.25$, factor 4, initial 1 s, doubling; no sampling of retransmissions; the three sender events; the four-row ACK table (500 ms); three duplicates $\Rightarrow$ fast retransmit; one timer; GBN/SR hybrid.

Understand. Why bytes are numbered; why ISNs are random; why smoothing and a variability margin; Karn's ambiguity; why no NAKs; why three; why cumulative ACKs make lost ACKs cheap; why only the oldest segment is resent.

Diagrams. Figs. 19 to 22.

Numerical patterns. Seq/ACK for a sequence of segments (with ISN, loss, duplicates); a 4–6-step RTT table; weight of the k-th oldest sample; header length from the 4-bit field.

6.7 TCP flow control

[TB §3.5.5, Fig. 3.38; SL 3-93 to 3-98]

The problem. What if the network delivers data faster than the application removes it from the socket buffer (SL 3-93)? The application may be busy and read long after arrival; a slow reader lets the sender overflow the receive buffer, and data that crossed the Internet successfully is thrown away at the last step.

Definition: Flow control

A **speed-matching service** that eliminates the possibility of the sender overflowing the receiver's buffer, by matching the sending rate to the rate at which the receiving application reads. SL 3-93 (and the class definition): "receiver controls sender, so sender won't overflow receiver's buffer by transmitting too much, too fast."

The test. If the network had infinite capacity, would the mechanism still be needed? Yes $\Rightarrow$ flow control (a slow reader is still slow). If the receiver had an infinite buffer? Yes $\Rightarrow$ congestion control.

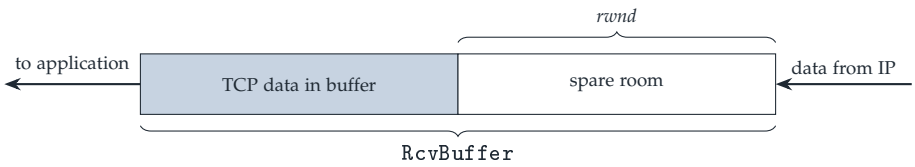


Figure 23: The receive window *rwnd* is the spare room in `RcvBuffer`. After TB Fig. 3.38 / SL 3-97.

For the bookkeeping, TB assumes here that the receiver discards out-of-order segments. Host B allocates `RcvBuffer` and tracks `LastByteRead` (last byte the application has read) and `LastByteRcvd` (last byte placed in the

Table 11: Flow control versus congestion control — "the savvy reader would be wise to distinguish between them" (TB §3.5.5). This is the detailed comparison; other sections refer here.

	Flow control	**Congestion control**
Slogan	"don't overwhelm the receiver"	"don't overwhelm the network"
Scarce resource	the receiver's buffer	link capacity and router buffers
Who sets the limit	the receiver, explicitly	the sender, by inference
Signal	explicit: *rwnd* in every segment	implicit: loss or delay (explicit only with ECN)
Variable	*rwnd*	*cwnd*
Parties affected	this one connection	every flow sharing the bottleneck
Failure if absent	buffer overflow; data dropped at the destination	congestion collapse (§7.2.3)

buffer from the network).

Σ Formula: Receive window and the sender's constraint

$$\text{LastByteRcvd} - \text{LastByteRead} \le \text{RcvBuffer}, \qquad rwnd = \text{RcvBuffer} - \big[\text{LastByteRcvd} - \text{LastByteRead}\big],$$

$$\text{LastByteSent} - \text{LastByteAcked} \le rwnd, \qquad \text{usable window}^{[S]} = rwnd - (\text{LastByteSent} - \text{LastByteAcked}).$$

Meaning. *rwnd* is the free space; it is dynamic, starts at `RcvBuffer`, and B places its current value in the receive-window field of *every* segment to A. A keeps its unACKed ("in-flight") data within *rwnd*, which guarantees no overflow (SL 3-97). `RcvBuffer` is set by socket options (typical default 4096 bytes) and auto-adjusted by many OSs.

Units. Bytes.

Common mistakes. Computing occupied instead of free space; bounding by `RcvBuffer` instead of *rwnd*; counting all bytes sent instead of sent-but-unACKed.

Definition: The zero-window problem and its fix (TB §3.5.5)

If B's buffer fills, it advertises $rwnd = 0$. If B then has nothing to send, it never tells A that space has opened, because TCP sends a segment only when it has data or an ACK to send — A is blocked for ever. **Fix**: the specification requires A to keep sending segments with **one data byte** while *rwnd* is zero; B ACKs them, and once the buffer empties the ACKs carry a non-zero *rwnd*.

Worked example: rwnd and the usable window

Given. RcvBuffer $= 8000$; at B: LastByteRcvd $= 20\,000$, LastByteRead $= 15\,000$; at A: LastByteSent $= 22\,000$, LastByteAcked $= 20\,001$.

Calculation. $rwnd = 8000 - (20\,000 - 15\,000) = 3000$; in flight $= 22\,000 - 20\,001 = 1999$; usable $= 3000 - 1999 = 1001$ bytes. Invariant: $5000 \le 8000$ ✓.

Interpretation. Sending 3000 more would put 4999 bytes in flight against a 3000-byte window; the excess could arrive to a full buffer and be dropped after crossing the whole network.

Worked example: A slow receiver clamps the sender

Given. RcvBuffer $= 10\,000$, initially empty; B's application reads 1000 bytes per RTT; A could send 4000 per RTT; no loss; ignore congestion control.

Calculation.

RTT	*rwnd* known to A	A sends	buffered on arrival	after app reads	*rwnd* advertised
1	10 000	4000	4000	3000	7000
2	7000	4000	7000	6000	4000
3	4000	4000	10 000	9000	1000
4	1000	1000	10 000	9000	1000
5	1000	1000	10 000	9000	1000

Answer. From RTT 4 on, A is clamped to 1000 bytes per RTT — the application's read rate: speed matching made concrete. Had the application stopped reading, *rwnd* would reach 0 and the one-byte probes would keep the connection alive.

Worked example: Which limit binds? (preview of $\min\{cwnd, rwnd\}$, §8.1.1)

Given. $rwnd = 64\,000$ bytes, $cwnd = 6000$ bytes, $RTT = 50$ ms.

Calculation. In flight $\leq \min\{6000, 64\,000\} = 6000$ bytes; rate $\approx 6000 \times 8/0.05 = 0.96$ Mbps.

Interpretation. Congestion control binds. Swap the numbers and the rate is the same but the reason reverses: flow control binds (slow application). Four cases: fast receiver and network (neither, until a window fills); slow receiver (*rwnd*); congested network (*cwnd*); both (the smaller).

UDP has no flow control. UDP appends segments to a finite buffer "preceding" the socket; the process reads one whole segment at a time; if it reads too slowly the buffer overflows and segments are dropped.

6.8 TCP connection management

[TB §3.5.6, Figs. 3.39–3.42; SL 3-99 to 3-106, 3-151, 3-152] Connection establishment adds noticeably to perceived delay (e.g. Web browsing), and many common attacks — notably the SYN flood — exploit TCP connection management.

What a handshake must achieve. (SL 3-99) Both sides (1) agree to establish the connection, each knowing the other is willing, and (2) agree on parameters such as the starting sequence numbers. Afterwards both are in ESTAB and hold the sequence numbers in each direction and the peer's `rcvBuffer` size.

Why two messages are not enough. (SL 3-100 to 3-103; class notes) A two-way "Let's talk" / "OK" fails in a network with variable delay, retransmission after loss, reordering, and sides that cannot see each other:



Figure 24: Two-way handshake failure scenarios. After SL 3-101 to 3-103 and the class notes (which conclude: "hence the 3-way handshake").

Insight: The precise reason a third message is needed

After message 1 the server knows the client wants a connection; after message 2 the client knows the server agreed and knows its ISN. But the server still does not know whether the client received message 2 — whether the client is still there, and whether the request was current or an old duplicate. Message 3, by acknowledging server_isn + 1, proves the client received message 2 *in this incarnation*; only then does each side know that the other knows both ISNs. TB's analogy: a climber and belayer use exactly such a three-way exchange before the climb.

6.8.1 The three-way handshake

Definition: Three-way handshake (TB §3.5.6)

Step 1 — SYN. The client sends a segment with no application data, **SYN** = 1, and a randomly chosen `client_isn` in the sequence field.
Step 2 — SYNACK. The server allocates buffers and variables and replies, again without data: **SYN** = 1, ACK field = `client_isn`+1, sequence field = its own `server_isn`. In effect: "I received your SYN with ISN client_isn, I agree, my ISN is server_isn."
Step 3 — ACK. The client allocates buffers and variables and acknowledges with ACK field = `server_isn`+1, **SYN** = 0. This segment **may carry data**. Thereafter SYN is 0 in every segment.
Slide notation (SL 3-104, 3-151, class): SYN(seq = x) $\rightarrow$ SYNACK(seq = y, ACKnum = $x+1$) $\rightarrow$ ACK(ACKnum = $y+1$).

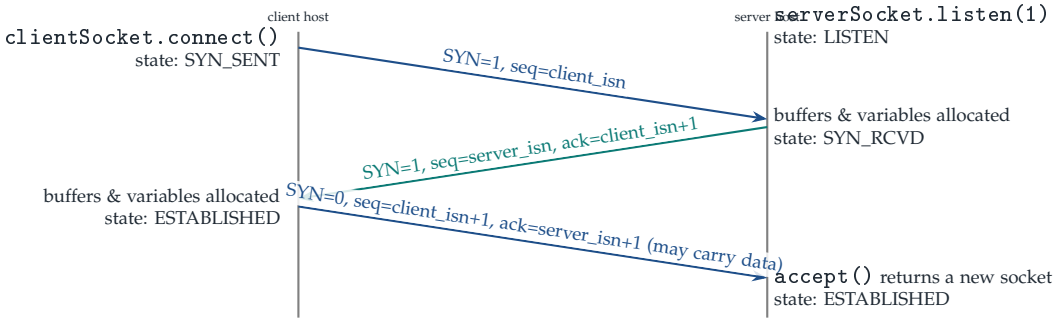


Figure 25: TCP three-way handshake with states and socket calls. After TB Fig. 3.39 / SL 3-104, 3-151.

Worked example: Numbering a handshake plus the first data

Given. `client_isn` = 1500, `server_isn` = 9000; after the handshake the client sends 200 bytes and the server replies with 500.

Rule. SYN consumes one number; otherwise $Seq_{\text{next}} = Seq + \ell$; ACK = next byte expected.

Calculation.

#	Dir.	Flags	Seq	ACK	Reason
1	C→S	SYN	1500	—	ACK bit 0: field not valid
2	S→C	SYN, ACK	9000	1501	SYN consumed 1500
3	C→S	ACK	1501	9001	server's SYN consumed 9000
4	C→S	ACK, 200 B	1501	9001	carries 1501–1700
5	S→C	ACK, 500 B	9001	1701	1501 + 200; carries 9001–9500

The client's next ACK is 9001 + 500 = 9501.

Interpretation. Segments 3 and 4 share Seq 1501 because 3 carries no data; they may be merged, since the third handshake segment may carry data.

6.8.2 Closing a connection and TCP states

Definition: Teardown (TB §3.5.6; SL 3-106)

Either side may close; the hosts then free buffers and variables. Each side closes its own direction: send a segment with **FIN** = 1; respond to a received FIN with an ACK (which may be combined with one's own FIN); simultaneous FINs are handled. A FIN means "I have no more data to send".

Definition: TIME_WAIT

After ACKing the server's FIN the client waits in TIME_WAIT, which lets it **resend the final ACK if that ACK is lost**. The time is implementation-dependent — typically **30 s, 1 min or 2 min** (TB; Fig. 3.41 says 30 s).

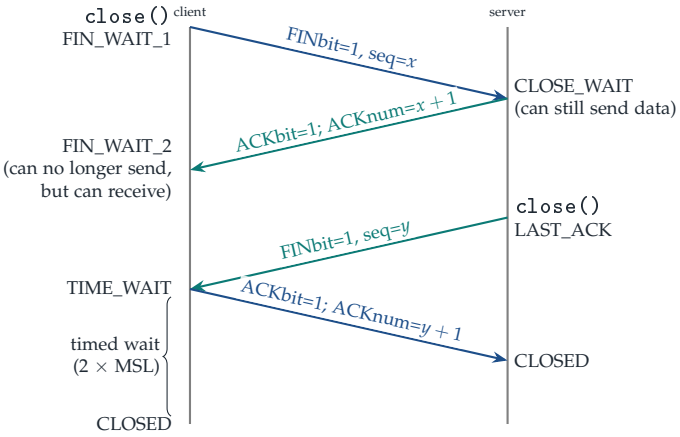


Figure 26: Closing a TCP connection: four segments because each direction of the full-duplex connection is a separate byte stream closed separately; between the 2nd and 3rd segments the connection is *half-closed*. After TB Fig. 3.40 / SL 3-106, 3-152.

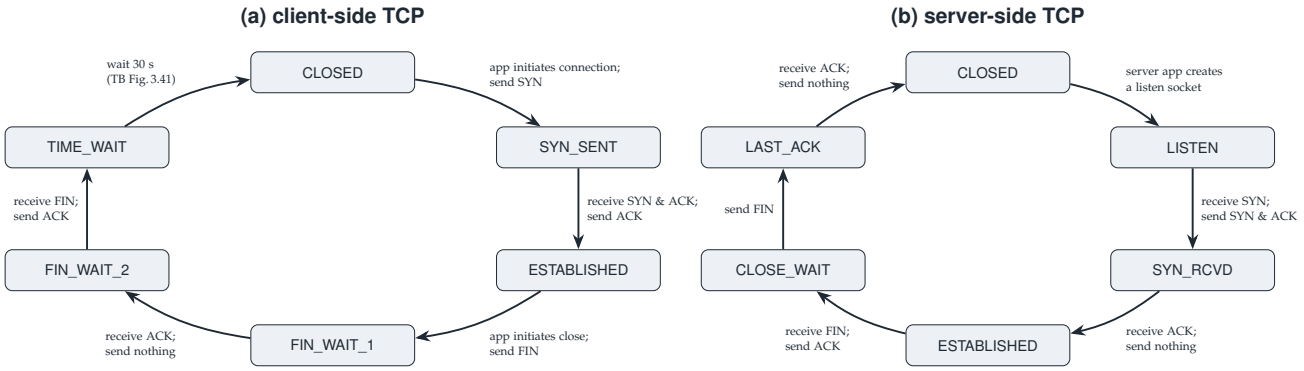


Figure 27: A typical sequence of TCP states, client and server (client initiates the close). After TB Figs. 3.41, 3.42.

Then the connection closes and all client resources, including port numbers, are released. [TB §3.5.6]

Source & convention note: TIME_WAIT duration: book vs slides/class

SL 3-152 and the class notes label the state "timed wait for 2× maximum segment lifetime". Consistent with TB: the slide gives the rule, the book typical results. Quote the one your course uses (class: 2×MSL). The 2×MSL rule also suggests a second purpose: stray segments of the old connection die out before the same 4-tuple can be reused — the hazard that random ISNs and the maximum packet lifetime (§5.5) also guard against. (Interpretation, not a quoted statement.)

6.8.3 No matching socket; nmap; SYN flood

A SYN whose destination port matches no socket (e.g. port 80 with no Web server) is answered with a **reset segment, RST** = 1: "I don't have a socket for that segment. Please do not resend." A UDP segment to an unmatched port triggers a special **ICMP** datagram instead (TB Ch. 5). To probe TCP port 6789, nmap sends a SYN to it:

Response	nmap's conclusion
SYNACK	an application is running on 6789: "open"
RST	the SYN reached the host, no application is on that port, and no firewall blocks it
nothing	the SYN was likely blocked by an intervening firewall

Definition: SYN flood and SYN cookies (TB §3.5.6; RFC 4987)

The server allocates variables and buffers on receiving a SYN; if no final ACK comes it reclaims them only after a minute or more. In a **SYN flood**, a classic DoS attack, the attacker sends many SYNs and never

completes the handshake, exhausting the server's resources on half-open connections.
SYN cookies (now in most major OSs): on a SYN the server creates *no* half-open connection. It sets its ISN to a hash of the SYN's source and destination IPs and ports and a server secret (the cookie), sends the SYNACK, and remembers nothing. A legitimate client returns an ACK; the server recomputes the hash from the ACK's addresses and ports and checks that hash $+1$ equals the ACK number; only then does it create a full connection and socket. A SYN never followed by an ACK has cost nothing.

Exam Focus: Flow control and connection management

Memorise. *rwnd* formula, no-overflow invariant, sender constraint; default `RcvBuffer` 4096 bytes; zero-window one-byte rule; Table 11; the three handshake segments with flags and numbers; four-segment close; client states CLOSED→SYN_SENT→ESTABLISHED→FIN_WAIT_1→FIN_WAIT_2→TIME_WAIT→CLOSED; server CLOSED→LISTEN→SYN_RCVD→ESTABLISHED→CLOSE_WAIT→LAST_ACK→CLOSED; TIME_WAIT purpose and values; RST vs ICMP; nmap's three outcomes; SYN cookies.

Understand. Why *rwnd* is in every segment; why a zero window deadlocks without probes; why UDP loses data at the receiver; why two messages fail; why teardown takes four segments; why SYN cookies defeat the flood.

Diagrams. Figs. 23 and 25 to 27.

Question patterns. "Find *rwnd* / how many more bytes may A send?"; "what rate does the sender converge to?"; "given *cwnd* and *rwnd*, which binds?"; "fill in every Seq/ACK given the ISNs"; "show a two-way handshake failing"; "explain SYN cookies".

7 Principles of Congestion Control

[TB §3.6, Figs. 3.43–3.49; SL 3-108 to 3-120]

7.1 What congestion is

Retransmission treats a *symptom* of congestion (the loss of a particular segment) but not its *cause*: too many sources trying to send at too high a rate. Treating the cause requires mechanisms that throttle senders. Like reliable transfer, congestion control is "high on our 'top-ten' list" (SL 3-108: "a top-10 problem!").

Definition: Congestion (SL 3-108)

Informally: "too many sources sending too much data too fast for network to handle". Manifestations: **long delays** (queueing in router buffers) and **packet loss** (buffer overflow at routers). SL 3-108's own bullet: *different from flow control!* (Table 11).

7.2 Causes and costs: three scenarios

TB §3.6.1 removes one idealisation at a time; each scenario exposes a new cost. Learn the scenarios by their costs.

Scenario	Idealisation removed	New cost exposed	Throughput
1	infinite buffers (kept)	#1 large queueing delay as load nears capacity	$\lambda_{\text{out}} \le R/2$
2	finite buffers; perfect knowledge of loss, then perfect timers	#2 retransmissions needed; #3 unneeded duplicate copies	$R/2 \to R/3 \to R/4$
3	single hop $\to$ multihop	#4 upstream capacity wasted on packets dropped downstream	$\to 0$ (collapse)

7.2.1 Scenario 1: two senders, one router, infinite buffers

Hosts A and B each send original data at λ_{in} bytes/s through one router whose outgoing link has capacity R; no retransmission, flow or congestion control; buffers are infinite.

Σ Formula: Scenario 1: throughput and delay

$$\lambda_{\text{out}} = \begin{cases} \lambda_{\text{in}}, & 0 \le \lambda_{\text{in}} \le R/2 \\ R/2, & \lambda_{\text{in}} > R/2 \end{cases} \qquad \text{average delay} \to \infty \text{ as } \lambda_{\text{in}} \to R/2.$$

Variables. λ_{in} per-connection sending rate, λ_{out} per-connection throughput, R link capacity (bytes/s).

Why $R/2$. Two connections share the link: no connection can receive more than $R/2$ in steady state however fast it sends (SL 3-109: "maximum per-connection throughput: R/2").

Why delay explodes. Approaching $R/2$ from below the queue grows; above $R/2$ the average queue is unbounded and the average delay infinite (for ever-running connections and infinite buffers).

Cost #1: large queueing delays as the arrival rate nears capacity — operating near R is ideal for throughput but "far from ideal from a delay standpoint".

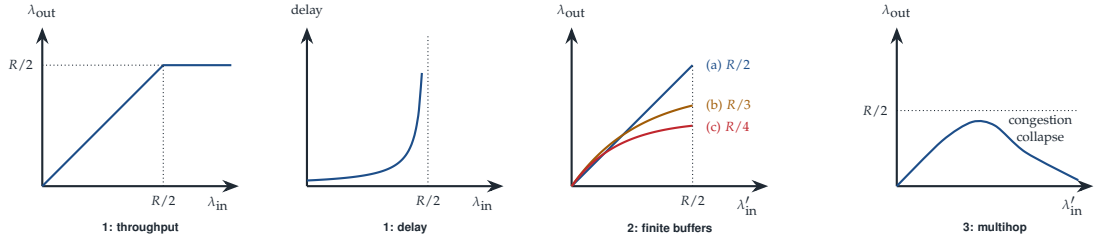


Figure 28: Per-connection throughput and delay in the three scenarios (shapes schematic). After TB Figs. 3.44, 3.46, 3.48.

7.2.2 Scenario 2: finite buffers and retransmission

Now the router buffer is finite (arrivals to a full buffer are dropped) and each connection is reliable (drops are eventually retransmitted).

Definition: Offered load

λ_{in} is the rate at which the application sends *original* data into the socket; λ'_{in} is the rate at which the transport layer sends segments *including retransmissions* into the network — the **offered load**. SL 3-110: application-layer input = output ($\lambda_{in} = \lambda_{out}$); transport-layer input includes retransmissions, $\lambda'_{in} \geq \lambda_{in}$. [TB §3.6.1]

Case	Assumption	Behaviour	Result
(a)	Perfect knowledge: A "magically" sends only when a buffer slot is free	No loss; $\lambda_{in} = \lambda'_{in}$; throughput $= \lambda_{in}$ up to $R/2$	$\lambda_{out} = \lambda'_{in} \leq R/2$
(b)	Retransmit only when a packet is known *for certain* to be lost	At $\lambda'_{in} = R/2$ the receiver gets $R/3$: $0.333R$ original + $0.166R$ retransmitted	$\lambda_{out} = R/3$
(c)	Realistic: premature timeouts resend packets that were only delayed; the receiver discards the duplicate	The router's work on duplicates is wasted; if each packet is forwarded twice on average, throughput tends to $R/4$ as $\lambda'_{in} \to R/2$	$\lambda_{out} \to R/4$

Cost #2: the sender must retransmit to compensate for packets dropped by buffer overflow — "more work (retransmission) for given receiver throughput" (SL 3-115). **Cost #3**: unneeded retransmissions in the face of large delays make routers forward unneeded copies — "link carries multiple copies of a packet, decreasing maximum achievable throughput".

Worked example: Checking $R/3$ and $R/4$

(b) Delivered original $= R/3 \approx 0.333R$; retransmitted $= \lambda'_{in} - \lambda_{out} = 0.5R - 0.333R = 0.166R$ — one third of the offered load.

(c) Only half of what the router forwards is useful: $\lambda_{out} = \frac{1}{2}\lambda'_{in} \to \frac{1}{2} \cdot \frac{R}{2} = \frac{R}{4}$.

Interpretation. $R/2 \to R/3 \to R/4$ tells one story: ideal, then necessary retransmissions, then wasteful duplicates.

7.2.3 Scenario 3: four senders, multihop paths

Four hosts send over overlapping two-hop paths with timeout/retransmission; all have the same λ_{in}; all links have capacity R. The A–C connection crosses R1 then R2, sharing R1 with D–B and R2 with B–D.

- Small λ_{in}: overflows are rare; throughput $\approx$ offered load.
- Large λ_{in}: A–C traffic reaching R2 arrives at most at rate R (the R1–R2 link), whatever λ_{in} is, but B–D traffic can arrive at R2 much faster. Competing for R2's buffer, less and less A–C traffic gets through as B–D's load grows.
- In the limit an empty slot at R2 is immediately taken by a B–D packet: A–C's throughput at R2, and hence end to end, goes to zero (SL 3-116: "blue throughput $\to 0$").

Cost #4: when a packet is dropped along a path, the transmission capacity used at every upstream link to carry it that far is wasted — "the network would have been equally well off (more accurately, equally bad off) if the first router had simply discarded that packet and remained idle." This fall of throughput towards zero is **congestion collapse**; TCP's congestion control was developed in response to observed Internet collapse [Jacobson 1988]. Only cost #4 turns degradation into collapse, because the waste grows with the very load that causes it.

SL 3-118's five insights. (a ready-made short answer) Throughput can never exceed capacity; delay increases as capacity is approached; loss/retransmission decreases effective throughput; unneeded duplicates decrease it further; upstream transmission capacity and buffering are wasted for packets lost downstream.

7.3 Approaches to congestion control

[TB §3.6.2, Fig. 3.49; SL 3-119, 3-120]

Definition: End-to-end congestion control

The network layer gives **no explicit support**; the end systems must infer congestion from observed behaviour (loss, delay). TCP takes this approach because IP need not report congestion: segment loss (a timeout or three duplicate ACKs) is taken as congestion and the window is reduced. Newer proposals use rising RTT as the signal.

Definition: Network-assisted congestion control

Routers give **explicit feedback** about congestion to the sender and/or receiver — a single congestion bit (IBM SNA, DEC DECnet, ATM) or something richer, e.g. ATM ABR, where a router tells the sender the maximum rate it can support. Internet-default IP and TCP are end-to-end, but may optionally be network-assisted (ECN, §8.8). SL 3-120: may indicate the congestion level or explicitly set the sending rate; examples TCP ECN, ATM, DECbit.

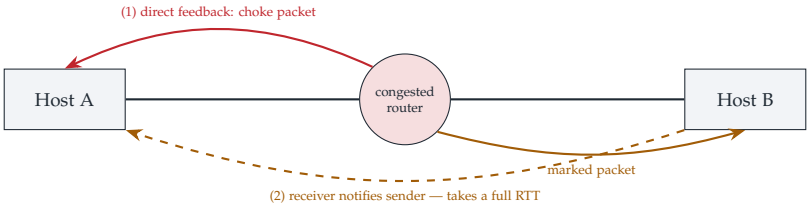


Figure 29: The two feedback pathways for network-indicated congestion: (1) a router sends a choke packet directly to the sender; (2) the more common form — the router marks a packet flowing to the receiver, which then notifies the sender. After TB Fig. 3.49.

	End-to-end	Network-assisted
Who produces the signal	the end systems infer it	routers mark or send it
Signal	implicit: loss (timeout, 3 dup ACKs), delay	explicit: congestion bit(s) or a rate
Router support needed	no	yes
Examples	TCP Reno/CUBIC; delay-based Vegas, BBR	TCP/IP ECN, ATM ABR, DECbit

"End-to-end" does not mean "no feedback": the sender gets plenty of *implicit* feedback — ACK arrivals and timing, missing and duplicate ACKs, RTT samples — which is why TCP is self-clocking (§8.1.3).

Exam Focus

Memorise. Definition and two manifestations; the three scenarios and four costs; $R/2 \to R/3 \to R/4$; λ_{in}, λ'_{in} (offered load), λ_{out}; SL 3-118's five insights; end-to-end vs network-assisted with examples; the two feedback pathways.

Understand. Why delay explodes before loss; why multihop paths collapse; why the marked-packet pathway takes a full RTT; flow vs congestion control (Table 11).

Diagrams. Figs. 28 and 29.

Question patterns. "State and explain four costs of congestion"; "why does throughput fall in scenario 3?"; "split the offered load in scenario 2(b)"; "compare end-to-end and network-assisted control".

8 TCP Congestion Control

[TB §3.7, Figs. 3.50–3.57; SL 3-122 to 3-140, 3-153, 3-154; RFC 5681]

8.1 Three questions, three answers

TCP must use end-to-end control because IP gives no explicit congestion feedback. Each sender limits its rate as a function of *perceived* congestion, which raises three questions: (1) how does a sender limit its rate? (2) how does it perceive congestion? (3) what algorithm changes the rate?

8.1.1 Answer 1: the congestion window

Definition: Congestion window, *cwnd*

A variable kept by the sender's congestion-control mechanism that constrains the rate at which it sends into the network; "dynamically adjusted in response to observed network congestion" (SL 3-124). It is private to the sender — no other host sees it.

Σ Formula: The sending limit and the sending rate

$$\text{LastByteSent} - \text{LastByteAcked} \le \min\{cwnd,\ rwnd\}, \qquad \text{rate} \approx \frac{cwnd}{RTT}\ \text{bytes/s}.$$

Meaning. Unacknowledged data may not exceed the tighter of two independent limits: the network's (estimated) capacity and the receiver's free buffer (§6.7).

Derivation of the rate. With negligible loss and transmission delay, at the start of each RTT the sender may send *cwnd* bytes and receives their ACKs at the end: "send *cwnd* bytes, wait RTT for ACKs, then send more". Adjusting *cwnd* adjusts the rate.

TB's assumptions for §3.7. *rwnd* so large it can be ignored; the sender always has data.

Common mistakes. Adding the windows; treating *cwnd* as advertised by the receiver; applying $cwnd/RTT$ across a whole slow-start phase, where *cwnd* changes every RTT.

8.1.2 Answer 2: the loss event

Definition: Loss event

Either a **timeout** or the receipt of **three duplicate ACKs**. Under excessive congestion a router buffer overflows and drops a datagram; the resulting loss event is taken by the sender as an indication of congestion on the path. [TB §3.7.1]

8.1.3 Answer 3: principles, self-clocking and bandwidth probing

1. A lost segment implies congestion ⇒ decrease the rate on loss.
2. An ACK for a previously unACKed segment means the network is delivering ⇒ increase the rate when ACKs arrive.
3. **Bandwidth probing**: increase on ACKs until a loss event, back off, then probe again — like a child who asks for more and more goodies until told "No!", backs off, and soon starts asking again. There is no explicit network signal; each sender acts on local information, asynchronously.

Definition: Self-clocking

TCP uses ACK arrivals as the signal that all is well and to *trigger* (clock) increases of *cwnd*. If ACKs arrive slowly (long delay or a low-bandwidth link) *cwnd* grows slowly; if quickly, it grows quickly. Hence TCP is **self-clocking**. [TB §3.7.1]

8.2 AIMD

Definition: Additive increase, multiplicative decrease (SL 3-122, 3-123; TB §3.7.1)

Additive increase: raise the rate by 1 MSS every RTT until loss is detected. **Multiplicative decrease**: cut the rate in half at each loss event. The result is the AIMD **sawtooth** — "probing for bandwidth". (TB: ignoring slow start and assuming triple-duplicate-ACK losses, TCP's control is exactly AIMD.)

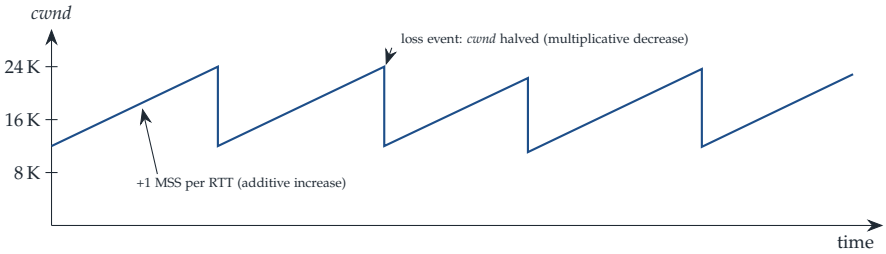


Figure 30: The AIMD sawtooth. After TB Fig. 3.53 / SL 3-122.

Σ Formula: AIMD update rules

$$\text{per new ACK (congestion avoidance):}\ \ cwnd \leftarrow cwnd + MSS \cdot \frac{MSS}{cwnd} \implies +1\ MSS \text{ per RTT;}$$

$$\text{on a loss event:}\ \ cwnd \leftarrow cwnd/2.$$

Why the per-ACK form gives $+1$ MSS per RTT. With $MSS = 1460$ and $cwnd = 14\,600$ bytes, 10 segments are sent per RTT; each of the 10 ACKs adds $1/10$ MSS.

Decrease detail (SL 3-123). Halve on loss detected by triple duplicate ACK (TCP Reno); cut to 1 MSS on loss detected by timeout (TCP Tahoe).

Why AIMD. A distributed, asynchronous algorithm shown to optimise congested flow rates network-wide [Kelly 1998] and to have desirable stability properties; it also converges to fairness (§8.10).

Common mistake. "+1 MSS per ACK" — that is slow start.

8.3 Classic TCP congestion control

The algorithm [Jacobson 1988; RFC 5681] has three components: **slow start** and **congestion avoidance** (mandatory; they differ in how fast *cwnd* grows per ACK — slow start faster, despite its name) and **fast recovery** (recommended, not required).

8.3.1 Slow start

Definition: Slow start (TB §3.7.1; SL 3-125)

A connection starts with $cwnd = 1$ MSS [RFC 3390] — an initial rate of about MSS/RTT (e.g. 500 bytes / 200 ms $\approx$ 20 kbps). *cwnd* grows by **1 MSS for every newly ACKed segment**, which **doubles the rate every RTT**. "Initial rate is slow, but ramps up exponentially fast."

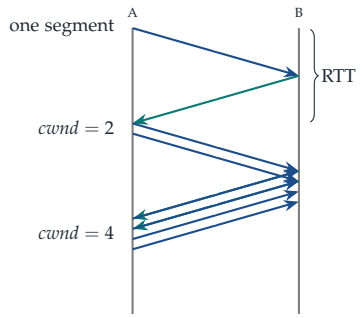


Figure 31: Slow start: the window doubles each RTT. After TB Fig. 3.50 / SL 3-125.

Σ Formula: Slow-start growth

$$\textit{cwnd} \text{ after } n \text{ RTTs} = 2^n \, \textit{MSS} \quad (\textit{cwnd} = 1 \text{ at } n = 0),$$

$$\text{segments sent in rounds } 0..n = \sum_{i=0}^{n} 2^i = 2^{n+1} - 1.$$

Assumptions. One ACK per segment (delayed ACKs slow growth, since *cwnd* grows per ACK); no loss; below *ssthresh*.
Common mistakes. Off-by-one round indexing (state yours); calling slow start slow.

How slow start ends. [TB §3.7.1]

1. **Timeout**: set $\textit{ssthresh} = \textit{cwnd}/2$ (the *slow-start threshold*: half of *cwnd* when congestion was detected), set $\textit{cwnd} = 1$ MSS and restart slow start.
2. *cwnd* **reaches** *ssthresh*: "it might be a bit reckless to keep doubling" near the level where congestion last occurred, so switch to congestion avoidance. SL 3-126: switch from exponential to linear "when *cwnd* gets to 1/2 of its value before timeout".
3. **Three duplicate ACKs**: fast retransmit and enter fast recovery.

8.3.2 Congestion avoidance

On entry *cwnd* is about half its value when congestion was last seen — congestion may be "just around the corner" — so *cwnd* grows by just **1 MSS per RTT** [RFC 5681], usually as $\textit{MSS} \cdot \textit{MSS}/\textit{cwnd}$ per new ACK. It ends on a **timeout** exactly as slow start does ($\textit{ssthresh} = \textit{cwnd}/2$, $\textit{cwnd} = 1$, slow start), or on a **triple duplicate ACK**, where the network is evidently still delivering some segments, so the response is milder: $\textit{ssthresh} = \textit{cwnd}/2$, $\textit{cwnd} = \textit{ssthresh} + 3\,\textit{MSS}$ (the 3 accounts for the three duplicates received), enter fast recovery.

8.3.3 Fast recovery; Tahoe and Reno

Definition: Fast recovery (TB §3.7.1)

cwnd grows by 1 MSS for every further duplicate ACK for the missing segment. When the ACK for the missing segment arrives, TCP **deflates** *cwnd* (to *ssthresh*) and enters congestion avoidance. A timeout in fast recovery acts as elsewhere: $\textit{ssthresh} = \textit{cwnd}/2$, $\textit{cwnd} = 1$, slow start.

TCP Tahoe (early) cut *cwnd* to 1 MSS and entered slow start on *either* kind of loss. **TCP Reno** added fast recovery.

Table 12: Slow start versus congestion avoidance.

	Slow start	Congestion avoidance
Per new ACK	$\textit{cwnd} \mathrel{+}= 1$ MSS	$\textit{cwnd} \mathrel{+}= \textit{MSS}^2/\textit{cwnd}$
Per RTT	doubles (exponential)	+1 MSS (linear, additive increase)
Entered	at connection start, and after a timeout ($\textit{cwnd} = 1$)	when $\textit{cwnd} \geq \textit{ssthresh}$, or after fast recovery (*cwnd* deflated to *ssthresh*)
Rationale	find the available bandwidth quickly	congestion may be near: probe cautiously

8.3.4 The complete FSM

Source & convention note: Conventions for numerical work (source differences resolved)

(1) Switching threshold. TB Fig. 3.51 labels slow start → congestion avoidance "$\textit{cwnd} \geq \textit{ssthresh}$" (text: "when *cwnd* equals *ssthresh*"); SL 3-127 writes "$\textit{cwnd} > \textit{ssthresh}$". **These notes use** $\geq$; the difference moves the switch by at most one round — say which you use. **(2) Reno after a triple duplicate ACK.** TB's text and Fig. 3.52 set $\textit{cwnd} = \textit{ssthresh} + 3$ and let it grow linearly *from there*; the FSM's final step, the class notes

Table 13: Timeout versus triple duplicate ACK — the detailed comparison (the commonest error in *cwnd* numericals is mixing these up).

	Timeout	**Three duplicate ACKs**
Trigger	no ACK for *TimeoutInterval*	ACKs keep arriving, all with the same value
What it suggests	possibly severe congestion: nothing getting through	a single loss; later segments still delivered
Retransmission	oldest unACKed segment; *TimeoutInterval* doubled	fast retransmit of the missing segment, before its timer
Speed of detection	slow (one *TimeoutInterval*, doubling on repeat)	fast (about one RTT)
Tahoe	$ssthresh = cwnd/2$, $cwnd = 1$, slow start	$ssthresh = cwnd/2$, $cwnd = 1$, slow start
Reno	$ssthresh = cwnd/2$, $cwnd = 1$, slow start	$ssthresh = cwnd/2$, $cwnd = ssthresh + 3$, fast recovery; then $cwnd = ssthresh$, congestion avoidance

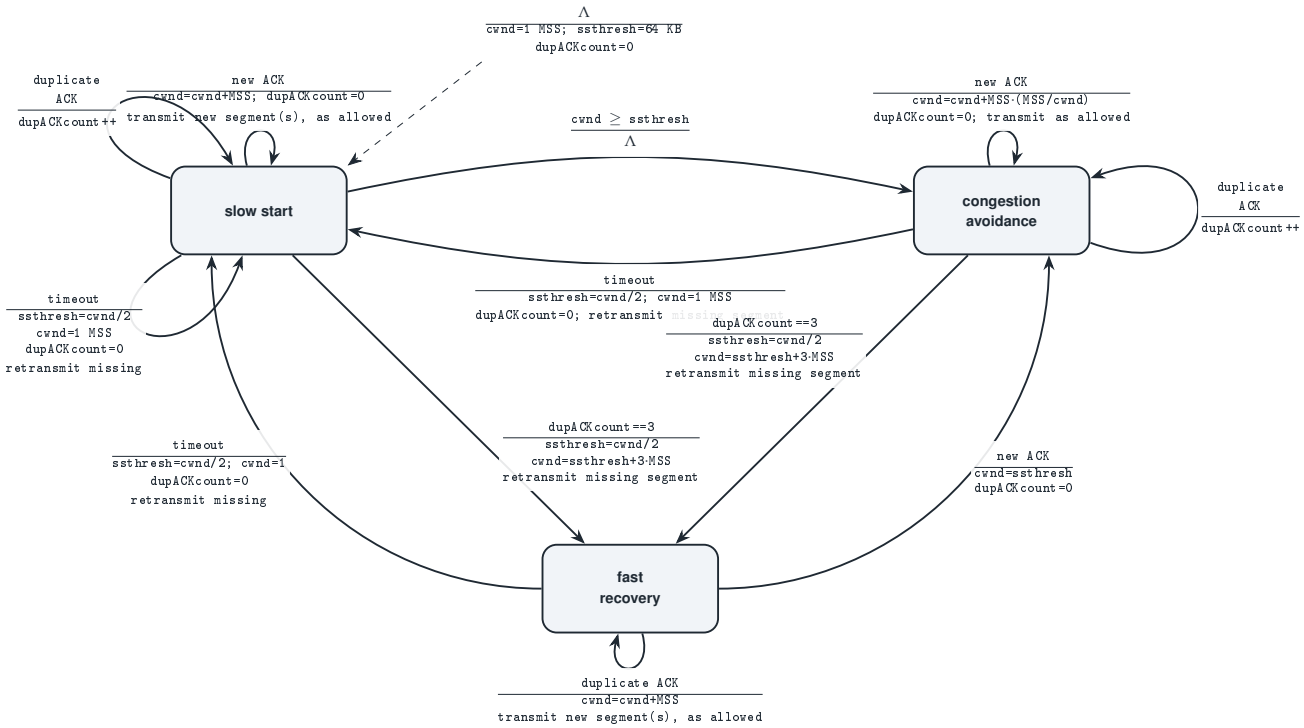


Figure 32: FSM of TCP (Reno) congestion control. After TB Fig. 3.51 / SL 3-127.

and all problems below **deflate to** *ssthresh* when the new ACK ends fast recovery, and grow linearly from *ssthresh*. **Use the class convention unless told otherwise, and state it in one line. (3) Slow-start overshoot.** If doubling would pass *ssthresh*, the window is capped at *ssthresh* (as in TB's Tahoe trace 1, 2, 4, 6, 7, … with $ssthresh = 6$). **(4) Rounds.** Tables list the *cwnd* used *during* each round (transmission round ≈ RTT); a loss in round n changes the window used in round $n + 1$; fast recovery lasts less than a round and is folded into that transition.

8.4 Evolution of the congestion window

Worked example: Reading TB Fig. 3.52 — Tahoe versus Reno (also the class trace)

Given. Initial $ssthresh = 8$ MSS; triple duplicate ACK in round 8, when $cwnd = 12$.

Analysis. Rounds 1–8 are identical: slow start 1, 2, 4, 8 (reaching *ssthresh* in round 4), then congestion avoidance 9, 10, 11, 12. At the loss $ssthresh = 0.5 \times 12 = 6$. *Reno*: fast recovery, then (class convention) linear growth 6, 7, 8, …, 12 in rounds 9–15 (TB's figure: 9, 10, …, 15). *Tahoe*: $cwnd = 1$, slow start 1, 2, 4, capped at 6, then linear 7, 8, 9.

Table 14: The FSM as a transition table — the most useful form for numerical questions.

State	Event	Next state	Action
Slow start	new ACK	SS (CA if $cwnd \geq ssthresh$)	$cwnd \mathrel{+}= MSS$; dupACKcount $= 0$; send as allowed
Slow start	dup ACK	SS	dupACKcount++
Slow start	dupACKcount $= 3$	Fast recovery	$ssthresh = cwnd/2$; $cwnd = ssthresh + 3MSS$; retransmit
Slow start	timeout	SS	$ssthresh = cwnd/2$; $cwnd = 1MSS$; dupACKcount $= 0$; retransmit
Cong. avoid.	new ACK	CA	$cwnd \mathrel{+}= MSS \cdot MSS/cwnd$; dupACKcount $= 0$; send
Cong. avoid.	dup ACK	CA	dupACKcount++
Cong. avoid.	dupACKcount $= 3$	Fast recovery	$ssthresh = cwnd/2$; $cwnd = ssthresh + 3MSS$; retransmit
Cong. avoid.	timeout	SS	$ssthresh = cwnd/2$; $cwnd = 1MSS$; dupACKcount $= 0$; retransmit
Fast recovery	dup ACK	FR	$cwnd \mathrel{+}= MSS$; send as allowed
Fast recovery	new ACK	CA	$cwnd = ssthresh$ (deflate); dupACKcount $= 0$
Fast recovery	timeout	SS	$ssthresh = cwnd/2$; $cwnd = 1MSS$; dupACKcount $= 0$; retransmit

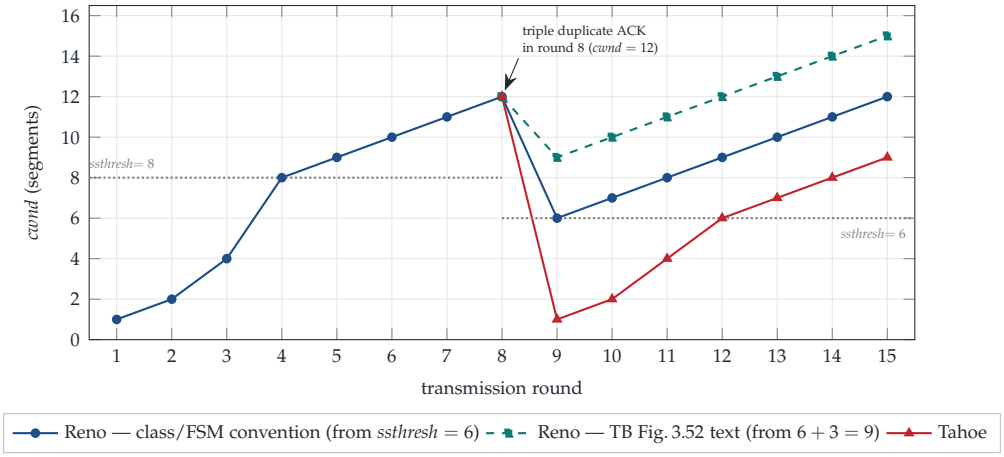


Figure 33: Evolution of *cwnd* for Tahoe and Reno with initial $ssthresh = 8$ and a triple-duplicate-ACK loss in round 8. Drawn after TB Fig. 3.52, with both Reno conventions shown.

Answer. Segments sent in rounds 9–15: Reno $6 + 7 + \cdots + 12 = 63$ (TB's version 84); Tahoe $1 + 2 + 4 + 6 + 7 + 8 + 9 = 37$.

Interpretation. Reno delivers roughly 1.7–2.3 times as much data during recovery — the numerical answer to "why was fast recovery added?".

Worked example: Full evolution with both loss types (Reno)

Given. $cwnd_1 = 1$, $ssthresh = 8$; triple duplicate ACK in round 8; timeout in round 11.

Rules. SS doubles per round (capped at *ssthresh*); CA +1 per round; 3 dup ACKs: $ssthresh = cwnd/2$, $cwnd = ssthresh + 3$, then *ssthresh* (CA); timeout: $ssthresh = cwnd/2$, $cwnd = 1$ (SS).

Calculation.

Round	*cwnd*	*ssthresh*	State / event	Round	*cwnd*	*ssthresh*	State / event
1	1	8	SS	9	6	6	CA
2	2	8	SS	10	7	6	CA
3	4	8	SS	11	8	6	CA; **timeout**
4	8	8	CA (reached *ssthresh*)	12	1	4	SS
5	9	8	CA	13	2	4	SS
6	10	8	CA	14	4	4	CA (reached *ssthresh*)
7	11	8	CA	15	5	4	CA
8	12	8	CA; **3 dup ACKs**	16	6	4	CA

Transitions. Round 8: $ssthresh = 12/2 = 6$, $cwnd = 6 + 3 = 9$ in fast recovery, deflated to 6 by the new ACK. Round 11: $ssthresh = 8/2 = 4$, $cwnd = 1$.

Interpretation. (i) The switch in round 4 is caused by reaching *ssthresh*, not by loss. (ii) The two losses recover differently: CA from 6 after the triple duplicate, slow start from 1 after the timeout. (iii) *ssthresh* falls $8 \to 6 \to 4$: each loss lowers TCP's estimate of the path capacity.

Worked example: Identifying phases from a trace

Given. *cwnd* in rounds 1–17: 1, 2, 4, 8, 9, 10, 11, 12, 13, 14, 15, 16, 1, 2, 4, 8, 9. MSS = 1000 bytes, RTT = 50 ms.

Required. Phases; initial *ssthresh*; type of loss after round 12; *ssthresh* after it; rate in round 11.

Reasoning. Doubling in rounds 1–4 $\Rightarrow$ SS; +1 per round from 8 to 16 $\Rightarrow$ CA, so initial $ssthresh = 8$. Drop from 16 to 1 $\Rightarrow$ **timeout** (Reno would have gone to $8 + 3$, then 8). New $ssthresh = 16/2 = 8$: SS 1, 2, 4, 8 again, then CA. Round 11: $cwnd = 15$ segments $= 15\,000$ bytes $\Rightarrow 15\,000 \times 8/0.05 = 2.4$ Mbps.

Diagnostic rule. Drop to 1 $\Rightarrow$ timeout (or Tahoe); drop to about half $\Rightarrow$ triple duplicate ACK (Reno); doubling $\Rightarrow$ slow start; +1 per round $\Rightarrow$ congestion avoidance.

Worked example: Time and data to reach a target window; TCP splitting

Given. $MSS = 1460$ bytes, $RTT = 100$ ms, *cwnd* starts at 1, $ssthresh = 32$, no loss.

Calculation. (a) $2^n = 32 \Rightarrow 5$ RTTs of slow start. (b) $40 - 32 = 8$ more RTTs to reach 40: 13 RTTs = 1.3 s. (c) SS rounds send $1 + 2 + 4 + 8 + 16 = 31$ segments; CA rounds $32 + \cdots + 39 = 284$; total $315 \times 1460 = 459\,900$ bytes. (d) At $cwnd = 40$: $40 \times 1460 \times 8/0.1 \approx 4.67$ Mbps.

Interpretation. Five rounds take *cwnd* from 1 to 32; eight take it only from 32 to 40 — slow start is the *fast* phase. The same RTT counting underlies TB's *TCP splitting* sidebar: a search reply needing three windows in slow start costs about $4 \cdot RTT$ (handshake + three windows); terminating the connection at a nearby front-end server cuts this to about $4 \cdot RTT_{\text{FE}} + RTT_{\text{BE}}+$ processing, i.e. from about $4 \cdot RTT$ towards RTT for distant users.

8.5 Macroscopic throughput and "long, fat pipes"

Σ Formula: Average throughput of a long-lived Reno connection

$$\text{average throughput} = \frac{0.75\,W}{RTT}.$$

Derivation (TB §3.7.1; SL 3-153). Let W be the window (bytes) when loss occurs. The window sweeps linearly from $W/2$ to W, so the rate sweeps from $\frac{W}{2RTT}$ to $\frac{W}{RTT}$ and its mean is $\frac{1}{2}\left(\frac{W}{2RTT} + \frac{W}{RTT}\right) = \frac{3W}{4RTT}$. SL 3-153: average in-flight bytes $= \frac{3}{4}W$.

Assumptions. Slow start ignored (after timeouts it is short — the sender grows out of it exponentially fast); losses are triple duplicates (halving, not reset); W and RTT constant; always data to send.

Common mistake. Quoting W/RTT (peak) or $W/(2RTT)$ (trough).

Worked example: Applying the macroscopic formula

Given. $MSS = 1460$, loss at $W = 40$ MSS, $RTT = 80$ ms.

Calculation. $W = 58\,400$ bytes $= 467\,200$ bits. Average $= 0.75 \times 467\,200/0.08 = 4.38$ Mbps; peak $= 5.84$; trough $= 2.92$; check $(5.84 + 2.92)/2 = 4.38$ ✓.

Interpretation. The connection never actually sends at 4.38 Mbps; it sweeps the sawtooth.

Σ Formula: Throughput in terms of loss probability (Mathis relation)

$$\text{TCP throughput} = \frac{1.22\,MSS}{RTT\sqrt{p}} \qquad (p = \text{segment loss probability; SL 3-154 and TB write } L).$$

Source. SL 3-154 [Mathis 1997]; TB §3.7.1 cites the relation and asks for it in problem P47.

Derivation (P47 outline). In one sawtooth cycle the window (in segments) grows from $W/2$ to W by one per RTT, and one packet is lost at the end. Packets sent per cycle: $\sum_{k=W/2}^{W} k = \frac{3}{8}W^2 + \frac{3}{4}W$, so $p = 1/(\frac{3}{8}W^2 + \frac{3}{4}W) \approx \frac{8}{3W^2}$ for large W. Hence $W \approx \sqrt{8/(3p)}$, and substituting into $0.75\,W \cdot MSS/RTT$ gives $0.75\sqrt{8/3}\,MSS/(RTT\sqrt{p}) = 1.22\,MSS/(RTT\sqrt{p})$.

Meaning. Throughput falls as $1/\sqrt{p}$ and as $1/RTT$.

Units. MSS in bits for bit/s; RTT in s.

Worked example: TCP over long, fat pipes (SL 3-154; TB P55)

Given. 1500-byte segments, $RTT = 100$ ms, target 10 Gbps.

Calculation. (a) Window $= \frac{10^{10} \times 0.1}{1500 \times 8} \approx 83\,333$ segments in flight. (b) $\sqrt{p} = \frac{1.22 \times 12\,000}{0.1 \times 10^{10}} = 1.464 \times 10^{-5} \Rightarrow p \approx 2 \times 10^{-10}$.

Answer. About 83 333 segments in flight; tolerable loss rate $\approx 2 \cdot 10^{-10}$ — one loss per 5 000 000 000 segments.

Interpretation. "A very small loss rate!" — why new TCP versions for long, high-speed paths were needed (Table 16).

8.6 TCP CUBIC

[TB §3.7.1, Fig. 3.54; SL 3-128, 3-129]

The insight. Halving the rate (or, in Tahoe, cutting to one packet per RTT) and then creeping back may be overly cautious. If $W_{\max}$ is the rate at which loss was last detected, the bottleneck's congestion state "probably (?) hasn't changed much", so after the cut *ramp up to $W_{\max}$ quickly, then approach it slowly* [Ha 2008; RFC 8312].

Definition: CUBIC rules (TB §3.7.1; SL 3-129)

CUBIC differs only slightly from Reno: the window still grows only on ACKs, slow start and fast recovery are unchanged; **only congestion avoidance changes**. $W_{\max}$ is the window at the last loss; K is the future time at which the window will again reach $W_{\max}$ if no loss occurs (K is tunable). CUBIC increases the window as a function of the **cube of the distance between the current time** t **and** K: large increases far from K, small cautious ones near K; beyond K increases are small at first (good if nothing changed) and then grow rapidly (to find a new operating point quickly if congestion has eased). CUBIC tends to achieve higher throughput than Reno.

Source & convention note: CUBIC deployment

TB: around 2000 nearly all popular Web servers ran some Reno; more recently nearly 50% of the 5000 most popular ran CUBIC [Yang 2014], the Linux default. SL 3-129 (9th ed.): default in Linux and "most popular TCP for popular Web servers (until ∼2024)", reflecting BBR's rise. Quote the figure that matches the source examined.

8.7 The bottleneck link: keep the pipe just full, but no fuller

[SL 3-130 to 3-136 (9th-edition deck); TB §3.7.2 has only the slogan and a short BBR paragraph]

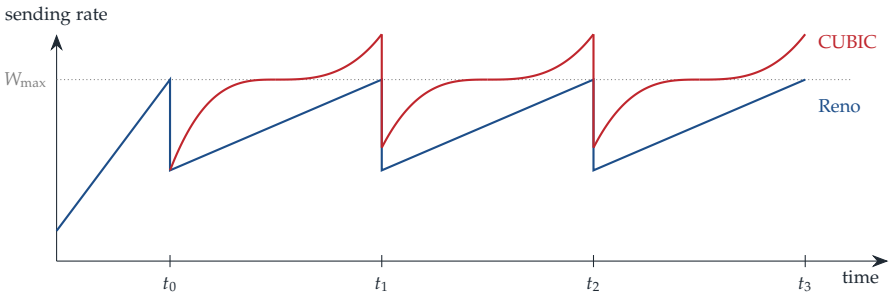


Figure 34: Congestion-avoidance sending rates, Reno (linear) and CUBIC (concave then convex around W_{max}); slow start ends at t_0, losses at t_1, t_2, t_3. Schematic, after TB Fig. 3.54 and the class sketch.

Definition: Bottleneck link (SL 3-130 to 3-132)

Loss-based TCP (classic, CUBIC) raises its rate until packet loss occurs at some router's output: the **bottleneck link**, which is almost always busy. Understanding congestion means focusing on it. When N flows share a bottleneck of rate R, each should ideally get R/N. Goal: "keep the end-end pipe just full, but not fuller" [Kleinrock 2018].

Σ Formula: Measured throughput and ideal in-flight data (SL 3-133)

$$\text{measured throughput} = \frac{\#\text{ bytes sent (in flight) in an RTT interval}}{RTT_{\text{measured}}},$$

$$n_{\text{inflight,ideal}} = \text{connection's share of bottleneck bandwidth} \times \text{min_RTT}.$$

Meaning. The ideal in-flight amount is the connection's **bandwidth–delay product** — the same BDP that sets the window for full utilisation in pipelining (§4.8): keep the bottleneck busy but avoid queueing delay.

Units. bytes; bytes/s; s.

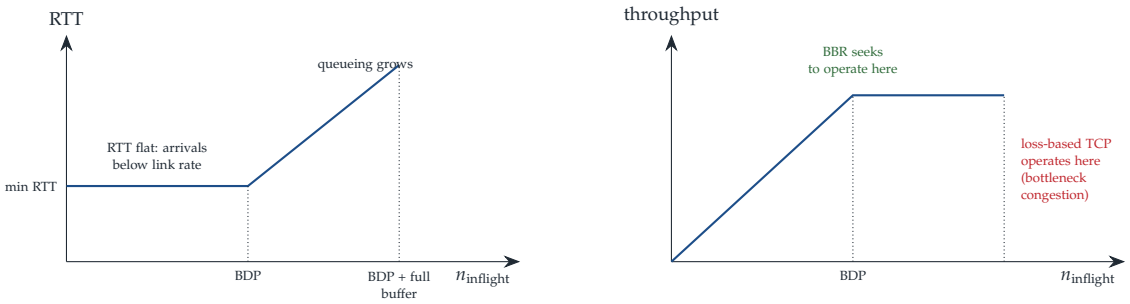


Figure 35: RTT and throughput versus in-flight data: below the BDP more data raises throughput but not RTT; above it, RTT rises but throughput does not; at BDP + a full buffer, loss. After SL 3-134, 3-135.

Definition: BBR (Bottleneck Bandwidth and RTT) (SL 3-136; TB §3.7.2)

BBR regulates n_{inflight}. *Shorter intervals*: **acceleration** (raise rate and n_{inflight} until throughput plateaus), **cruising** (send at the rate the network delivers, as evidenced by ACKs), **deceleration** (deliberately reduce n_{inflight} to relieve the queue and look for a lower min_RTT). *Longer intervals*: drain the pipe to measure a new min_RTT. TB: BBR [Cardwell 2017] builds on TCP Vegas, includes mechanisms to compete fairly with non-BBR senders, replaced CUBIC on Google's private B4 network in 2016 and is being deployed on Google and YouTube servers. Class summary: uncongested rate $= cwnd/RTT_{\text{min}}$ — the basis of delay-based control; BBR keeps re-measuring a new RTT_{min}.

Loss-based TCP learns of congestion only from loss, so it drives the bottleneck to the *congestion* point (full buffer); BBR aims at the BDP — "the pipe is kept just full, but no fuller" (SL 3-135).

8.8 Network-assisted control: ECN

[TB §3.7.2, Fig. 3.55; SL 3-137; RFC 3168]

Definition: Explicit Congestion Notification

The Internet's form of network-assisted control, involving both IP and TCP.

- **IP**: two bits in the IP header's Type-of-Service field. One setting is used by a *router* to indicate congestion; it travels in the marked datagram to the destination. When a router counts as congested is not defined by RFC 3168: the vendor makes it configurable and the operator decides. The intuition: signal the *onset* of congestion before loss. Another setting is used by the *sender* to tell routers that sender and receiver are ECN-capable.
- **Receiver**: on a congestion mark, the receiving TCP sets the **ECE** (ECN-Echo) bit in an ACK to the sender.
- **Sender**: reacts by **halving** *cwnd*, as for a fast-retransmit loss, and sets **CWR** (Congestion Window Reduced) in its next segment.

SL 3-137: "TCP deployments often implement network-assisted congestion control"; ECN involves IP (header bit marking) and TCP (C, E bits).

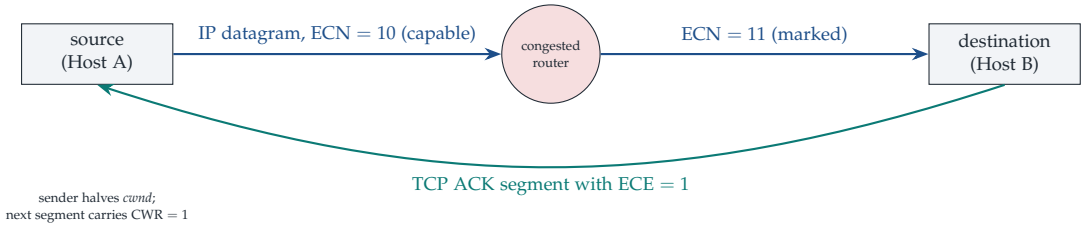


Figure 36: ECN message flow. After TB Fig. 3.55 / SL 3-137.

ECN bits (IP)	Meaning (class notes)	Set by
00	not ECN-capable	source
10 or 01	ECN-capable transport	source
11	congestion encountered	router, based on its buffer

Supplementary: RFC 3168 names these Not-ECT (00), ECT(1) (01), ECT(0) (10) and CE (11).

Other ECN users [TB §3.7.2]: DCCP [RFC 4340] (a low-overhead, congestion-controlled, UDP-like unreliable service), and data-centre protocols DCTCP [RFC 8257] and DCQCN; ECN deployment in servers and routers is increasing.

Insight: Advantages and limitations of ECN

Advantages: congestion is signalled before buffers overflow, so senders slow down without any loss — no retransmission and no wasted upstream capacity (cost #4); the signal is unambiguous (a wireless bit error cannot set it); no extra packets. *Limitations*: needs router support and ECN-capable endpoints; marking policy is operator-defined; the signal still takes a full RTT (pathway 2 of Fig. 29); it says *that* there is congestion, not how much (contrast ATM ABR's explicit rate). (Synthesis of the sources.)

8.9 Delay-based congestion control

[TB §3.7.2]

Definition: TCP Vegas (Brakmo 1995)

Detect congestion onset *before* loss from delay. The sender measures the RTT of all ACKed packets; $RTT_{\min}$, the smallest, is the uncongested path. With window *cwnd* the uncongested throughput is $cwnd/RTT_{\min}$. If the measured throughput is close to it, increase the rate; if significantly less, the path is congested: decrease. Vegas embodies "keep the pipe just full, but no fuller".

Σ Formula: The Vegas test

$$\text{uncongested} = \frac{cwnd}{RTT_{\min}}, \qquad \text{actual} \approx \frac{cwnd}{RTT_{\text{measured}}};$$

$$\text{actual} \approx \text{uncongested} \Rightarrow \text{increase}; \qquad \text{actual} \ll \text{uncongested} \Rightarrow \text{decrease}.$$

Why delay reveals congestion. RTT = propagation + transmission + queueing; the first two are essentially fixed, so any rise above $RTT_{\min}$ is queueing — a buffer filling. Queues fill before they overflow, so delay detects congestion earlier than loss.

Instance[S]. $cwnd = 30$ segments of 1000 bytes; $RTT_{\min} = 50\,\text{ms}$; measured 75 ms. Uncongested $= 240\,000/0.05 = 4.8\,\text{Mbps}$; actual $= 240\,000/0.075 = 3.2\,\text{Mbps}$ — a third lower: $\approx 25\,\text{ms}$ of queueing, so Vegas decreases.

Limitations. $RTT_{\min}$ may be overestimated if the path is never idle, and route changes invalidate it; a delay-based flow backs off while loss-based competitors keep pushing (hence BBR's fairness mechanisms).

Other delay-based protocols named by TB: TIMELY (data centres), Compound TCP (CTCP) and FAST (high-speed, long-distance).

Table 15: Three families of congestion control.

	Loss-based (Reno, CUBIC)	Network-assisted (ECN)	Delay-based (Vegas, BBR)
Signal	timeout or 3 dup ACKs	router-set IP bits, echoed by ECE	RTT rising above $RTT_{\min}$
Router support	no	yes	no
Detects congestion	after buffers overflow	at onset, before loss	as queues build, before loss
Cost of detection	a lost packet and a retransmission	none	none
Main weakness	wasted capacity; treats wireless loss as congestion	needs routers and both ends; operator-defined policy	baseline may be wrong; loses to aggressive competitors

8.10 TCP fairness

[TB §3.7.3, Figs. 3.56, 3.57; SL 3-138 to 3-140]

Definition: Fairness

K TCP connections, each with its own path, all pass through a bottleneck of rate R (all other links uncongested); each transfers a large file; no UDP. Congestion control is **fair** if each connection's average rate is approximately R/K. [TB §3.7.3; SL 3-138]

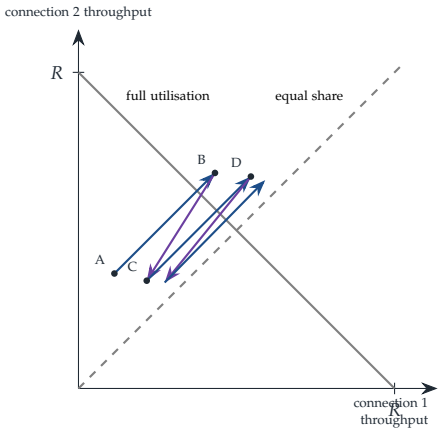


Figure 37: Two AIMD connections converge to the equal-share line. After TB Fig. 3.57 / SL 3-139.

The argument (idealised: same MSS and RTT, large data, no other traffic, slow start ignored so both are always in AIMD). Throughputs on the 45° line are equal; on the line $x_1 + x_2 = R$ the link is fully used. From A the total is below R, no loss: both windows grow by 1 MSS per RTT, so the point moves at 45° (additive increase, "slope of 1"). At B the total exceeds R: loss, both halve — the point moves halfway towards the origin, to C (multiplicative decrease, "decreases throughput proportionally"). Repeat to D, … The bandwidth eventually fluctuates along the equal-share line, whatever the start. SL 3-139: "Is TCP fair? Yes, under idealized assumptions: same RTT; fixed number of sessions only in congestion avoidance."

Supplementary information: Why AIMD — and not another combination — converges

Let the throughputs be (x_1, x_2) and the unfairness $d = |x_1 - x_2|$. Additive increase adds the same amount to both: d unchanged. Multiplicative decrease halves both: d halves. So $d \to 0$. Additive increase with additive decrease leaves d constant for ever; multiplicative with multiplicative scales it without converging. (Algebraic form of the geometric argument, after [Chiu 1989].)

When fairness fails.

- **Different RTTs.** Sessions with smaller RTT open their windows faster when bandwidth frees up and so get higher throughput [Lakshman 1997].
- **UDP.** Internet phone and video conferencing often use UDP precisely so their rate is not throttled, pumping at a constant rate and accepting occasional loss. From TCP's perspective they are not fair and can crowd out TCP; SL 3-140: "there is no 'Internet police' policing use of congestion control".
- **Parallel TCP connections.** An application using several connections (browsers often do) gets a larger share of a congested link.

Worked example: How strongly RTT biases the share[S]

Given. Two long-lived Reno flows at a bottleneck with the same loss rate p and MSS; $RTT_1 = 20\,\text{ms}$, $RTT_2 = 100\,\text{ms}$.

Formula. Mathis: throughput $\propto 1/RTT$ at equal p.

Answer. $T_1/T_2 = RTT_2/RTT_1 = 5$: the short-RTT flow gets about five times as much.

Interpretation. Additive increase is "1 MSS per RTT", so the short-RTT window grows five times as often. (Ratio derived here; TB gives only the qualitative statement.)

Worked example: The parallel-connections calculation (TB §3.7.3; SL 3-140)

Given. A link of rate R carries nine applications with one TCP connection each; a new application arrives.

Formula. Each of K connections gets $\approx R/K$; an application holding m gets mR/K.

Calculation. One connection: $K = 10$, share $R/10$. Eleven connections: $K = 20$, share $11R/20 = 0.55R$.

Interpretation. TB: "more than $R/2$"; SL 3-140 rounds to $R/2$. Fairness is *per connection*, not per application or user.

Exam Focus

Memorise. LastByteSent − LastByteAcked $\leq \min\{cwnd, rwnd\}$; rate $\approx cwnd/RTT$; loss event; slow start (1 MSS, +1/ACK, doubling/RTT); CA (+1 MSS/RTT, $+MSS^2/cwnd$ per ACK); fast recovery (+1 per dup ACK, deflate to *ssthresh*); $ssthresh = cwnd/2$ on any loss; Table 13; initial $ssthresh = 64\,\text{KB}$; $0.75W/RTT$; $1.22\,MSS/(RTT\sqrt{p})$; CUBIC changes only CA; ECN bits, ECE, CWR; $cwnd/RTT_{\min}$; BBR's phases; R/K and its failures.

Understand. Why slow start is fast; why a timeout is punished harder; why AIMD converges; why loss-based TCP overfills the pipe and BBR does not; why ECN takes an RTT; why parallel connections defeat fairness.

Diagrams. Figs. 30 to 37.

Numerical patterns. Round-by-round *cwnd* tables with both loss types (Tahoe and Reno); phase identification; time/data to reach a window; average throughput; tolerable loss rate; shares with unequal RTT or parallel connections.

9 Evolution of Transport-Layer Functionality

[TB §3.8–§3.9, Figs. 3.58, 3.59; SL 3-141 to 3-146]

9.1 Why the two "work horses" were not enough

UDP and TCP have been the Internet's transport work horses (SL 3-142: "principal transport protocols for 40 years"), but experience has exposed situations neither suits well, so functionality keeps evolving.

Evolution within TCP. Beyond Tahoe and Reno, versions in significant use include CUBIC, DCTCP, CTCP and BBR; measurements [Yang 2014] show CUBIC (and its predecessor BIC) and CTCP more widely deployed on Web servers than Reno, and BBR runs in Google's B4 network and on many Google servers. TB also lists TCP variants for wireless links, for high-bandwidth large-RTT paths, for reordering, for short intra-data-centre paths, with priorities among connections, sending over several paths in parallel, and with different ACK and session establishment/closure.

Table 16: Scenarios where classic TCP struggles. Columns 1–2 from SL 3-142; column 3 links them to earlier sections.

Scenario	Challenge	Which mechanism struggles, and why
Long, fat pipes (large transfers)	many packets in flight; a loss shuts down the pipeline	AIMD halves a huge window and regrows 1 MSS per RTT; 10 Gbps needs $p \approx 2 \cdot 10^{-10}$ (§8.5); the 16-bit *rwnd* needs window scaling
Wireless networks	loss from noisy links and mobility; TCP treats it as congestion loss	"loss ⇒ congestion" is false: *cwnd* is halved for a bit error
Long-delay links	extremely long RTTs	every mechanism is clocked by the RTT (slow start, +1 MSS/RTT, timeouts)
Data-centre networks	latency sensitive	queue build-up is unacceptable ⇒ ECN-based DCTCP, DCQCN; delay-based TIMELY
Background traffic	low-priority "background" flows	TCP's goal is equal shares; background flows should deliberately yield

Insight: What, then, is "TCP"?

TB §3.8: "it's probably not even correct anymore to refer to 'the' TCP protocol; perhaps the only common features of these protocols is that they use the TCP segment format … and that they should compete 'fairly' amongst themselves". The segment format and the fairness obligation are invariant; the congestion-control algorithm is a replaceable module. (Surveys: [Afanasyev 2010], [Narayan 2018].)

9.2 QUIC: Quick UDP Internet Connections

Definition: Rolling your own transport; QUIC (TB §3.8; SL 3-142, 3-143)

If an application needs more than UDP but not all of TCP (or different services), designers can "roll their own" protocol *at the application layer*. QUIC [Langley 2017; QUIC 2020] is a new **application-layer protocol, running over UDP**, designed from the ground up to improve transport services for *secure HTTP*. It sits above UDP and below HTTP/3 and implements transport functions — reliability, congestion control, connection management — in the application layer (SL 3-142: "moving transport-layer functions to application layer, on top of UDP").

SCTP (Stream Control Transmission Protocol) [RFC 4960, 3286], a reliable, message-oriented protocol, pioneered multiplexing several application-level streams through one connection; it is used in 4G/5G cellular control-plane protocols.



Figure 38: (a) Traditional secure HTTP stack; (b) QUIC-based HTTP/3. After TB Fig. 3.58 / SL 3-143.

Table 17: QUIC's major features (TB §3.8; SL 3-144) and their roots in this chapter.

Feature	What QUIC does	Chapter-3 root
Connection-oriented and secure	A handshake between two endpoints sets up connection state, including source and destination **connection IDs**. All QUIC packets are encrypted, and QUIC **combines** the connection handshake with the authentication and encryption handshakes, so it is faster than TCP followed by TLS, which needs several RTTs. SL 3-144: reliability, congestion control, authentication, encryption and state established in **one (or even zero) RTT**.	three-way handshake (§6.8.1)
Streams	Several application-level **streams** are multiplexed over one QUIC connection and new ones can be added quickly. A stream is an abstraction for reliable, in-order, bidirectional delivery between two endpoints; in HTTP/3 each Web object gets its own stream. Each connection has a connection ID and each stream a **stream ID**, both in the QUIC packet header; data from several streams may share one QUIC segment, carried over UDP.	multiplexing (Section 2), one layer up
Reliable, TCP-friendly, congestion-controlled	Reliable transfer is provided to *each stream separately*, with ACK mechanisms similar to TCP's [RFC 5681]. Congestion control is based on **TCP NewReno** [RFC 6582], a slight modification of Reno (draft: "readers familiar with TCP's loss detection and congestion control will find algorithms here that parallel well-known TCP ones"). SL 3-144: separate reliable transfer and security per stream, **common congestion control**.	rdt (Section 4); TCP reliability; AIMD

9.3 Head-of-line blocking

Definition: Head-of-line (HOL) blocking (TB §3.8)

With HTTP/1.1 sending several requests over one TCP connection, TCP's reliable *in-order* byte delivery means the requests must be delivered in order; if bytes of one request are lost, all later requests wait until those bytes are retransmitted and received — **HOL blocking**. QUIC delivers reliably and in order *per stream*, so a lost UDP segment affects only the streams whose data it carried; other streams' messages are still delivered.

Insight: In-order delivery: TCP's strength and its structural limit

TCP makes the *connection* the unit of ordering, so everything waits behind a gap. QUIC keeps reliability but moves its granularity to the *stream*, while keeping congestion control per *connection* — each property applied where it makes sense.

Source & convention note: Labels and currency

TB Fig. 3.59 contrasts (a) HTTP/1.1 and (b) HTTP/3; SL 3-145 labels (b) "HTTP/2 with QUIC: no HOL blocking". TB describes QUIC below "a simplified but evolved version of HTTP/2", says HTTP/3 "will" incorporate QUIC and that QUIC was still being standardised; the 9th-edition slides already list HTTP/3 as a current UDP application. Differences of labelling and currency, not mechanism.

Deployment and evolvability. Google deployed QUIC on many public-facing servers, in Chrome, the mobile YouTube app and Android's Google Search app; more than 7% of Internet traffic was QUIC [Langley 2017]. Because QUIC is an application-layer protocol it can change at **"application-update timescales"** — far faster than kernel TCP or UDP.

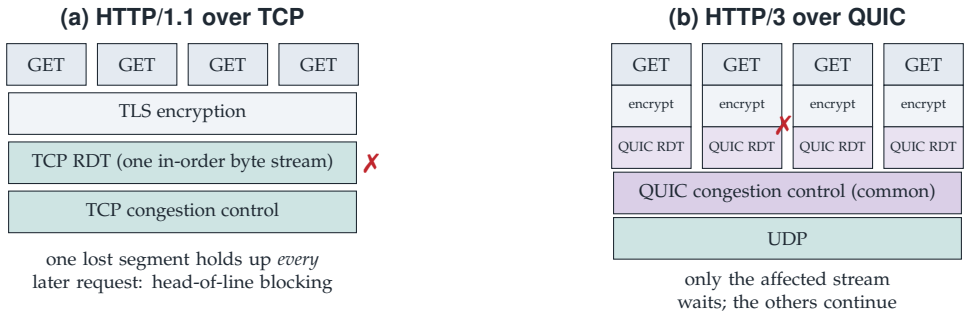


Figure 39: Streams give parallelism without head-of-line blocking: reliability and encryption per stream, congestion control shared. After TB Fig. 3.59 / SL 3-145.

Table 18: UDP, TCP and QUIC.

	UDP	TCP	QUIC
Layer	transport	transport (in the OS kernel)	**application**, over UDP
Connection setup	none	3-way handshake; TLS adds more RTTs	combined transport + crypto handshake: 1 (or 0) RTT
Reliability	none	one in-order byte stream per connection	per stream (in order within a stream)
Multiplexing	ports only	one stream per connection	many streams per connection (stream IDs)
HOL blocking	n/a	yes (across all data of the connection)	only within the affected stream
Congestion control	none	Reno/CUBIC/BBR …	NewReno-based, shared by all streams
Security	none	separate (TLS above)	built in; all packets encrypted
Identification	2-tuple socket	4-tuple	connection ID (plus UDP ports)
Evolution speed	kernel	kernel (slow)	application-update timescales

Core versus newer material. *Core*: mux/demux; UDP and the checksum; rdt; GBN/SR; TCP segments, numbers, RTT, reliability, flow control, connection management; congestion principles; Tahoe/Reno, AIMD, fairness. *Newer*: CUBIC; ECN; delay-based control (Vegas, BBR, TIMELY, CTCP, FAST); DCTCP, DCQCN; the bottleneck-link/n_{inflight} framing; QUIC and HTTP/3. SL 3-146 closes the chapter: leaving the network "edge" (application and transport) for the network "core" (data and control planes).

Exam Focus

Memorise. SL 3-142's five scenarios; QUIC = application layer over UDP; connection ID, stream ID; 1- or 0-RTT establishment; per-stream reliability and security, common congestion control (NewReno-based); SCTP as precursor; $>$ 7% of traffic.

Understand. Why wireless loss misleads TCP; why long RTTs slow every mechanism; how streams remove HOL blocking; why application-layer placement speeds evolution.

Diagrams. Figs. 38 and 39.

Question patterns. "Explain HOL blocking and how QUIC avoids it"; "at which layer does QUIC run and why does it matter?"; "three scenarios where classic TCP performs poorly"; "compare TCP and QUIC".

10 Reading Real Traces: the Wireshark Labs

[Assignment 3: Kurose–Ross Wireshark Lab: TCP (uploading `alice.txt` to `gaia.cs.umass.edu` by HTTP POST) and Wireshark Lab: UDP (DNS traffic from `nslookup`). Trace values are those printed in the lab handout's figures; your own trace will differ.]

The original notes scattered the lab material over a dozen boxes and a 55-question unit; it is gathered here once. Every item applies theory from the earlier sections, referenced in brackets.

10.1 Tools and capture

- **Wireshark** is a packet analyser ("sniffer"): it passively copies every frame sent or received by the host's interface and decodes each layer's headers (Ethernet, IP, TCP/UDP, application). It never sends packets itself.
- **nslookup** queries DNS to resolve a name to an address; query and reply travel in UDP segments to and from port 53 — a convenient, single request/reply UDP exchange (DNS is a core UDP application, Table 3). If the reply is lost, the application (resolver) times out and resends; UDP does not.
- The lab *uploads* with POST rather than downloading with GET so that the *client* is the TCP sender and the trace shows its sequence numbers, window growth, and the server's ACKs and advertised windows.
- A display filter (`tcp`, `udp`) only hides packets from the list; the capture holds everything.
- Wireshark's **"No."** column is a listing index, not a header field: neither UDP nor TCP has a "packet number". (A report such as "TCP packet 4 has sequence number 4" is wrong twice: the index is not in the packet, and TCP counts bytes — the first data segment has relative Seq 1.)

10.2 The UDP capture

Observation	Explanation
Four header fields, 2 bytes each	source port, destination port, length, checksum: 8-byte header (§3)
DNS query with 32-byte payload shows Length = 40	Length counts header + payload
Maximum payload	$2^{16} - 1 - 8 = 65\,527$ bytes; largest port number $2^{16} - 1 = 65\,535$
Query from an ephemeral port to 53; reply from 53 back to it	port inversion: the source port is the return address (Fig. 3) — e.g. query 51413→53, reply 53→51413
IP Protocol field: 17 for UDP, 6 for TCP	tells the receiving host which transport protocol gets the payload, before port demultiplexing (*supplementary*: IP header, TB Ch. 4)
Two clients querying port 53	reach the same UDP socket (2-tuple); two TCP clients to port 80 reach different sockets (4-tuple)
Checksum present	detects only some errors and nothing about loss — DNS retries at the application level

10.3 The TCP capture: setup and segment structure

The connection's 4-tuple is (192.168.86.68, 55639, 128.119.245.12, 80): client IP and ephemeral port, server IP and HTTP port. Every segment carries it, in one order or the other.

No.	Time (s)	Segment	Key fields (relative numbers)
1	0.000000	client→server [SYN]	Seq = 0, Win = 65 535, MSS = 1460, WS = 64, SACK_PERM
2	0.022414	server→client [SYN, ACK]	Seq = 0, Ack = 1, Win = 28 960, MSS = 1460
3	0.022505	client→server [ACK]	Seq = 1, Ack = 1
4	0.024047	client→server data (POST)	Seq = 1, Len = 1448 ⇒ next Seq = 1449

- **Flags.** The SYN shows `Flags: 0x002` (only SYN); the SYNACK `0x012` (SYN and ACK).
- **SYNACK's Ack** = client ISN +1 (relative 1): the SYN consumes one sequence number (§6.4).
- **Raw vs relative sequence numbers.** Wireshark subtracts the ISN so the SYN shows 0 and the first data byte 1; the raw header value is the random ISN (plus offset). Relative numbers are a convenience *because* ISNs are random.

- **Header length.** The SYN shows "Header Length: 44 bytes (11)": the 4-bit field holds 11 words, $11 \times 4 = 44$, because the SYN carries options — MSS, window scale, SACK permitted, timestamps. Data segments carry 32-byte headers (20 + 12 bytes of timestamp option).
- **Frame sizes.** SYN $= 14 + 20 + 44 = 78$ bytes; pure ACK $= 14 + 20 + 32 = 66$; full data segment $= 14$ (Ethernet) $+20$ (IP) $+32$ (TCP) $+1448$ (data) $= 1514$.
- **MSS vs data per segment.** The SYN advertises $MSS = 1460 = 1500 - 40$, yet each full segment carries 1448 data bytes: the 12 bytes of timestamp options come out of the MSS ($1460 - 12 = 1448$). TCP segment length $= 32 + 1448 = 1480$; all segments stay below 1480 bytes because the interface limits datagrams to 1500 bytes and at least 40 bytes are TCP/IP headers.
- **Segmentation.** The ≈153 KB POST is a byte stream split into MSS-sized chunks: Wireshark shows "[106 Reassembled TCP Segments (153425 bytes)]". The server's TCP reassembles them in its buffer by sequence number; the "HTTP" line is attached to the *last* segment. The POST header itself travels in the first data segment (Seq = 1), sent soon after the SYNACK arrives; the third handshake segment *may* carry data.
- **SACK.** SACK_PERM in the SYN, echoed in the SYNACK, means both ends agree to selective acknowledgments — TCP then behaves more like SR (§6.6).

10.4 Sequence numbers, ACKs and retransmissions in the trace

- Consecutive client Seq values 1, 1449, 2897, 4345 $\Rightarrow$ 1448 bytes each; after three in-order segments the server should ACK 4345.
- Server ACKs rising by 1448 ($1 \to 1449 \to 2897$) mean per-segment ACKing; a jump of $2 \times 1448 = 2896$ (e.g. $2897 \to 5793$) means **one cumulative ACK for two segments** — row 2 of the ACK-generation table (Table 10).
- **Retransmissions**: look for a repeated sequence number, duplicate ACKs, or a dot below the rising line in the time–sequence graph; Wireshark labels "TCP Retransmission" and "TCP Dup ACK". The lab trace has none: sequence numbers only increase. Four identical ACKs followed by a resend well before any timeout is **fast retransmit**.
- The server's pure ACKs during the upload carry no data, so their sequence number stays at relative 1.

10.5 RTT, timeout and throughput from a trace

Worked example: SampleRTT and EstimatedRTT from the trace

Given. Segment Seq 1 sent at 0.024047 s, ACKed (ACK 1449) at 0.052671 s; segment Seq 1449 sent at 0.024048 s, ACKed (ACK 2897) at 0.052676 s. Initial $EstimatedRTT$ = first sample (as the lab instructs), $\alpha = 0.125$.

Formula. $SampleRTT = t_{\text{ACK}} - t_{\text{sent}}$; $EstimatedRTT \leftarrow 0.875\, EstimatedRTT + 0.125\, SampleRTT$.

Calculation. $SampleRTT_1 = 28.624$ ms, $SampleRTT_2 = 28.628$ ms; $EstimatedRTT = 0.875(28.624) + 0.125(28.628) = 25.046 + 3.5785 = 28.6245$ ms.

Interpretation. The handshake alone gives an estimate too: SYN at 0, SYNACK at 0.022414 s $\Rightarrow \approx 22.4$ ms, since the server replies at once. Wireshark plots all samples: Statistics $\to$ TCP Stream Graph $\to$ Round Trip Time Graph.

Worked example: Measured throughput of the connection

Formula. throughput = bytes transferred / elapsed time.

Substitution. 153 425 bytes; first data segment at 0.024047 s; server's HTTP 200 OK (sent after the last byte) at 0.192625 s.

Calculation. $153\,425 / 0.168578 \approx 910\,000$ bytes/s ≈ 7.3 Mbit/s.

Interpretation. State the start and end events used (SYN or first segment; last segment, last ACK or server reply): answers differ with that choice. This is the *actual* average of one short transfer, including slow start and application effects — not the idealised long-run Reno average $0.75W/RTT$ (§8.5).

10.6 Flow control and window scaling in the trace

- "Window Size Value" is the raw 16-bit *rwnd* field of the segment's *sender* — its free receive-buffer space (flow control, §6.7).
- With window scaling negotiated in SYN/SYNACK, $rwnd_{\text{actual}} = \text{Window Size Value} \times \text{scaling factor}$. The client offers WS = 64 (shift 6, 2^6); its later segments advertise $2058 \times 64 = 131\,712$ bytes, shown as "[Calculated window size]". SYN and SYNACK windows are never scaled. Scaling exists because 65 535 bytes is too small for high-BDP paths.
- Did flow control throttle the sender? Compare the smallest window the server advertised with the client's unACKed data in flight. In the lab the server's window keeps growing (28 960, 31 872, 34 816, 37 760, ...) while only a few 1448-byte segments are outstanding: the receiver never limited the sender.
- *cwnd* cannot be seen in a trace — it is private to the sender — only inferred from how much the sender keeps in flight per RTT.

10.7 Congestion control in a Stevens time–sequence graph

Statistics → TCP Stream Graph → Time-Sequence-Graph (Stevens) plots the sequence number of each segment against its send time.

- Each dot is a segment; a vertical stack is a **fleet** — one window sent back to back.
- The gap between fleets is about one RTT (self-clocking: the next window waits for ACKs). In the lab trace fleets near $t = 0.025, 0.053, 0.082, 0.1$ s are ≈ 0.028 s apart — the measured RTT.
- Fleet sizes doubling per RTT ⇒ slow start; growing by one segment per RTT ⇒ congestion avoidance; constant ⇒ limited by something else (receiver window or application).
- The slope of the graph (bytes/s) is the throughput.
- A short transfer (150 KB) finishes within a few RTTs, often before any loss or *ssthresh* is reached; initial window, delayed ACKs and application writes blur the doubling. Describe what the fleet sizes do and name the phase they correspond to.

Verifying each TCP service in a trace. Reliability: ACKs and any retransmissions. In-order delivery: contiguous sequence numbers and cumulative ACKs. Flow control: advertised windows versus bytes in flight. Congestion control: fleet sizes growing per RTT and backing off after loss.

Exam Focus

Be able to. find the SYN/SYNACK by flags; give raw vs relative sequence numbers; decode the header-length field; account for every byte of a 1514-byte frame; explain 1448 vs 1460; read ACK increments (per-segment vs delayed); compute SampleRTT, EstimatedRTT and throughput from timestamps; apply the window-scale factor; read fleets, spacing and slope in a time–sequence graph; state UDP field sizes, maximum payload, largest port and protocol numbers 17/6.

11 Common Confusions & Exam Traps

The single consolidated list. Each row states the misconception, the correct distinction, and where the mechanism is developed.

Misconception	Correct distinction	§
Layers and services		
"IP delivers data to the application" (host-to-host = process-to-process)	IP delivers a datagram to a *host* (an interface). Delivery to the right *process* is the transport layer's demultiplexing, using ports.	1.2, 2
"TCP is reliable, so it guarantees throughput or delay"	TCP guarantees eventual, correct, in-order delivery — nothing about when or how fast. No Internet transport can give delay/bandwidth guarantees because IP does not.	1.3
Segment vs datagram used interchangeably	Segment = transport-layer packet (TCP *and* UDP in these notes); datagram = network-layer (IP) packet. The RFCs' "UDP datagram" is the source of the muddle.	1.1
"Connection-oriented" = reliable, or = a reserved circuit	Connection-oriented = state set up by a handshake, held only in the end systems (routers are oblivious). Reliable = no loss, corruption, duplication, reordering. Independent: QUIC is both, over connectionless UDP.	6.1
UDP is only for video; TCP is "better"	Different service models. UDP: mux/demux + checksum, no setup, no state, 8-byte header, no congestion control — DNS, SNMP, HTTP/3. Streaming (DASH) often uses TCP.	3.2
Multiplexing		
Multiplexing and demultiplexing both happen at the receiver	Multiplexing: sender gathers from sockets and adds headers. Demultiplexing: receiver delivers each segment to the right socket. Every host does both.	2.1
"All clients of port 80 share one socket" (UDP 2-tuple vs TCP 4-tuple)	UDP socket = (dst IP, dst port): different sources reach the same socket. TCP connection socket = 4-tuple: each connection its own socket; two clients may even share a source port if their IPs differ.	2.3–2.4
"UDP demultiplexes on the destination port only" vs "two-tuple"	Both true: the destination IP was already used by IP to reach the host; within transport the discriminating field is the port.	2.3
Errors, reliability, acknowledgments		
A checksum makes a protocol reliable, or corrects errors	It only *detects* some errors; cannot correct, cannot detect loss, misses cancelling errors ("weak protection"). Reliability needs sequence numbers, ACKs, timers, retransmission.	3.4
Loss and corruption are the same event	Corruption: the packet arrives damaged, caught by the checksum. Loss: nothing arrives — only a timer (or duplicate ACKs from later packets) reveals it. rdt2.x handles corruption; rdt3.0 adds the timer. The sender responds identically: retransmit.	4.3–4.6
TCP sends NAKs (ACK vs NAK)	rdt2.0/2.1 use NAKs; rdt2.2 and TCP are NAK-free: re-ACK the last good data; a duplicate ACK acts as an implicit NAK.	4.5, 6.6
A duplicate ACK is a glitch to ignore	It is a deliberate signal of a gap. rdt2.2 retransmits on it; rdt3.0 ignores wrong-numbered ACKs (timer handles it); TCP fast-retransmits on three.	4.5, 4.6, 6.6
ACK n acknowledges only packet/byte n	Cumulative: GBN ACK(n) covers all $\leq n$; TCP ACK x covers all bytes $< x$. Selective (SR, TCP SACK) covers individual packets/blocks. Cumulative ACKs make earlier lost ACKs harmless.	5, 6.4
Sequence number and ACK number count the same thing	Seq = first data byte of *this* segment in its own direction; ACK = next byte expected from the *other* direction. Both count bytes; each direction has its own ISN.	6.4
"ACK 536 means I received byte 536"; ACK $= Seq + \ell - 1$	ACK 536 = "I have everything up to 535; send 536." ACK $= Seq + \ell$ when in order; in general the first missing byte.	6.4
SYN/FIN carry no data, so use no sequence numbers	Each consumes exactly one: SYNACK ACKs client_isn+1; the third segment ACKs server_isn+1; the ACK of FIN(x) is $x + 1$.	6.4
Timeout = duplicate-ACK trigger; one duplicate triggers resend	Timeout: no ACK within *TimeoutInterval* (possibly severe congestion). Fast retransmit: *three* duplicates (four identical ACKs); later segments still arriving. Different congestion responses (Table 13).	6.6, 8.3

Misconception	Correct distinction	§
TCP keeps a timer per segment and resends the window on timeout	One retransmission timer (oldest unACKed); a timeout resends only that segment. GBN resends the window; SR keeps a logical timer per packet.	6.6
Timeout = latest RTT, or = EstimatedRTT	$TimeoutInterval = EstimatedRTT + 4DevRTT$ ($\alpha = 0.125$, $\beta = 0.25$), initial 1 s, doubled after each timeout; never sample retransmitted segments.	6.5
Pipelining		
SR is always better; any window up to the sequence space	SR retransmits less but needs buffering, per-packet timers and $N \leq 2^{k-1}$; GBN needs only $N \leq 2^k - 1$. $k = 2$, $N = 3$ breaks SR.	5.4
"TCP is GBN because ACKs are cumulative"	Hybrid: GBN-like cumulative ACKs and SendBase/NextSeqNum; SR-like buffering of out-of-order data and single-segment retransmission; SACK makes it closer to SR.	6.6
MSS = segment size; MSS = window	MSS = maximum application data per segment (MTU $-$ 40); the window is bytes in flight.	6.2
Flow and congestion control		
Flow control = congestion control	Flow: don't overwhelm the *receiver*; explicit *rwnd*; protects one buffer. Congestion: don't overwhelm the *network*; *cwnd* inferred by the sender; protects shared links. Test: still needed with an infinitely fast network? $\Rightarrow$ flow control.	6.7
The sender uses *cwnd or rwnd*, or their sum	Always unACKed $\leq \min\{cwnd, rwnd\}$. *rwnd* is advertised in every segment; *cwnd* is private to the sender. Either may bind.	8.1
Flow control bounds by RcvBuffer	The bound is the free space *rwnd*, not the whole buffer, and it limits *unACKed* data, not all data sent.	6.7
Slow start grows slowly	It doubles *cwnd* every RTT (+1 MSS per ACK); CA adds 1 MSS per RTT. "Slow" only in starting at 1 MSS.	8.3
"After a loss *ssthresh* is halved"	*ssthresh* is set to half the current *cwnd* at the moment of loss, whatever its old value.	8.3
Reno resets to 1 MSS on three duplicate ACKs	That is Tahoe (or any timeout). Reno: $ssthresh = cwnd/2$, $cwnd = ssthresh + 3$, fast recovery, then $cwnd = ssthresh$.	8.3
Reliability and congestion control serve the same party	Reliability serves this application; congestion control serves everyone else and may slow this connection.	1.3
End-to-end congestion control means no feedback	No *explicit* router feedback; plenty of implicit feedback (ACKs, duplicates, timeouts, delay). ECN is the network-assisted alternative.	7.3
Every loss means congestion	On wireless links loss is often noise/mobility, yet TCP halves *cwnd* — a motivation for ECN and delay-based control.	9.1
TCP gives each application an equal share	Fairness (R/K) is per *connection*, under idealised conditions. Short-RTT flows get more; UDP need not back off; parallel connections win (11 of 20 $\Rightarrow 0.55R$).	8.10
Evolution		
QUIC is a new transport-layer protocol replacing TCP in the kernel (TCP vs QUIC)	QUIC is an *application-layer* protocol over UDP providing secure, reliable, congestion-controlled *streams*; per-stream ordering removes HOL blocking; evolves at application-update timescales.	9.2

12 Formula Sheet

One authoritative sheet. Each entry: the formula, a one-line meaning, and its conditions. Derivations are where the formula is first developed (section in the last column). [S] = derived in these notes from source statements.

Formula	Meaning · conditions · trap	§
A. UDP		
$\text{Length} = 8 + D$	Length field counts header + data (D payload bytes).	3.3
$D_{\max} = 2^{16} - 1 - 8 = 65\,527$ bytes[S]	Largest payload the 16-bit Length allows (IP's limit is tighter).	3.3
overhead $= H/(H+D)$; $H_{\text{UDP}} = 8$, $H_{\text{TCP}} = 20$[S]	Why small messages favour UDP.	3.3
$S_k = (S_{k-1} + w_k) \bmod 2^{16} + \lfloor (S_{k-1} + w_k)/2^{16} \rfloor$	One's-complement sum of 16-bit words with wraparound (header, data, IP addresses). Trap: dropping the carry.	3.4
$\text{checksum} = \overline{S_n} = (2^{16} - 1) - S_n$	Sender sends the complement (checksum field $= 0$ while summing).	3.4
receiver: $\sum_1 w_i \oplus_1 \text{checksum} = 1111111111111111$	All 1s $\Rightarrow$ no error *detected*; any 0 $\Rightarrow$ error.	3.4
ports $\in [0, 65\,535]$; well-known 0–1023	16-bit fields; 2^{16} values.	2.2
B. Reliable data transfer		
$d_{\text{trans}} = L/R$	Time to push a packet onto the link (L bits, R bit/s). Trap: bytes vs bits.	4.7
$U = \dfrac{L/R}{RTT + L/R}$	Stop-and-wait utilisation; tiny ACKs, no loss/processing/queueing. 1 Gbps, 30 ms, 8000 bits $\Rightarrow$ 0.00027.	4.7
$\text{throughput} = \dfrac{L}{RTT + L/R}$	Stop-and-wait delivered rate ($\approx$ 267 kbps in the example).	4.7
$U = \min\left\{1, \dfrac{N\,L/R}{RTT + L/R}\right\}$[S]	Pipelined, window N packets. Trap: $U > 1$.	4.8
$\text{BDP} = R \cdot RTT; \quad N \geq 1 + \dfrac{R \cdot RTT}{L}$[S]	Bits in the pipe; window for $U = 1$ (round up).	4.8
C. Go-Back-N / Selective Repeat		
space $[0, 2^k - 1]$, arithmetic mod 2^k	k-bit sequence field; a ring.	5.1
$\text{nextseqnum} - \text{base} \leq N$	GBN invariant; four intervals of the sender's view.	5.2
$\text{ACK}(n) \Rightarrow \text{base} \leftarrow n + 1$	GBN cumulative ACK. Trap: base $\leftarrow n$.	5.2
$N_{\text{SR}} \leq 2^{k-1}$	SR: old and new receiver windows must not overlap.	5.4
$N_{\text{GBN}} \leq 2^k - 1$[S]	GBN bound. $k = 3$: GBN 7, SR 4.	5.4
one loss: GBN resends up to N; SR exactly 1[S]	Why SR wins with large windows.	5.6
D. TCP sequence and acknowledgment numbers		
bytes carried $= [Seq, Seq + \ell - 1]$; $Seq_{\text{next}} = Seq + \ell$	ℓ = data bytes only; a pure ACK does not advance Seq.	6.4
ACK $= Seq + \ell$ (in order); generally the first missing byte	Next byte expected; cumulative. Trap: $Seq + \ell - 1$.	6.4
SYN, FIN consume 1 number	SYNACK ACK = client_isn+1; 3rd segment ACK = server_isn+1; ACK of FIN(x) $= x + 1$.	6.4
32-bit fields; arithmetic mod 2^{32}	Counts bytes; random ISNs.	6.4
$MSS = \text{MTU} - 40$ (typical); segments $= \lceil F/MSS \rceil$	MTU 1500 $\Rightarrow$ MSS 1460; MSS = data only.	6.2
header bytes $= 4\times$ header-length field (20–60)	Field counts 32-bit words.	6.3
E. RTT estimation		
$EstimatedRTT \leftarrow (1 - \alpha)EstimatedRTT + \alpha\, SampleRTT$, $\alpha = 0.125$	EWMA; update about once per RTT; never from a retransmitted segment (Karn). Trap: swapped weights.	6.5

Formula	Meaning · conditions · trap	§
$DevRTT \leftarrow (1-\beta)DevRTT + \beta\,\lvert SampleRTT - EstimatedRTT\rvert$, $\beta = 0.25$	EWMA of absolute deviation (not a variance); here computed with the new *EstimatedRTT*.	6.5
weight of sample k updates old $= \alpha(1-\alpha)^k$ [S]	Exponential decay; weights sum to 1.	6.5
F. Timeout		
$TimeoutInterval = EstimatedRTT + 4\,DevRTT$	Estimate plus variability margin. Trap: coefficient $\neq 4$.	6.5
initial $TimeoutInterval = 1$ s	RFC 6298.	6.5
on timeout: $TimeoutInterval \leftarrow 2\,TimeoutInterval$	Exponential backoff ($0.75 \rightarrow 1.5 \rightarrow 3.0$ s); recomputed when the timer is next started by data or an ACK.	6.5
G. Flow control		
LastByteRcvd − LastByteRead ≤ RcvBuffer	Receiver's no-overflow invariant.	6.7
$rwnd$ = RcvBuffer − (LastByteRcvd − LastByteRead)	Free space, advertised in every segment; default RcvBuffer often 4096 bytes.	6.7
LastByteSent − LastByteAcked ≤ $rwnd$	Sender's constraint on unACKed data.	6.7
usable = $rwnd$ − (LastByteSent − LastByteAcked) [S]	New bytes that may be sent now.	6.7
$rwnd_{\max} = 2^{16} - 1 = 65\,535$; actual = value × scale factor	16-bit field; window-scale option for high-BDP paths.	6.3, 10
$rwnd = 0 \Rightarrow$ sender keeps sending 1-byte segments	Breaks the zero-window deadlock.	6.7
H. Congestion control		
LastByteSent − LastByteAcked ≤ $\min\{cwnd, rwnd\}$	The real sending limit. Trap: adding the windows.	8.1
rate $\approx cwnd/RTT$	One window per RTT; not across a slow-start phase.	8.1
SS: $cwnd \mathrel{+}= MSS$ per new ACK; $cwnd = 2^n$ MSS after n RTTs; $\sum_0^n 2^i = 2^{n+1} - 1$	Starts at 1 MSS; ends at timeout, $cwnd \geq ssthresh$, or 3 dup ACKs.	8.3
CA: $cwnd \mathrel{+}= MSS \cdot MSS/cwnd$ per ACK (+1 MSS/RTT)	Additive increase.	8.3
timeout: $ssthresh = cwnd/2$, $cwnd = 1$ MSS, slow start	Tahoe and Reno.	8.3
3 dup ACKs (Reno): $ssthresh = cwnd/2$, $cwnd = ssthresh + 3$, FR; new ACK: $cwnd = ssthresh$	Tahoe: $cwnd = 1$. FR: +1 MSS per further dup ACK. Trap: forgetting +3 or the deflation.	8.3
initial $cwnd = 1$ MSS, $ssthresh = 64$ KB; SS→CA at $cwnd \geq ssthresh$	TB Fig. 3.51 (">" on SL 3-127).	8.3
CUBIC: growth $\propto (t-K)^3$ around $W_{\max}$	Changes congestion avoidance only.	8.6
Vegas: compare $cwnd/RTT_{\min}$ with $cwnd/RTT_{\text{measured}}$	Close ⇒ increase; much lower ⇒ decrease; the gap is queueing.	8.9
ECN: router 10→11; receiver ECE = 1; sender halves $cwnd$, sets CWR	Network-assisted; costs one RTT.	8.8
I. Throughput		
average $= 0.75\,W/RTT$	Reno sawtooth $W/2 \rightarrow W$ (W bytes at loss); no slow start, triple-dup losses. Trap: W/RTT.	8.5
throughput $= \dfrac{1.22\,MSS}{RTT\sqrt{p}}$	Mathis; p = loss probability.	8.5
$p = 1/(\frac{3}{8}W^2 + \frac{3}{4}W) \approx 8/(3W^2)$	Loss rate of the idealised sawtooth, W in segments (P47).	8.5
10 Gbps, 100 ms, 1500 B ⇒ $W \approx 83\,333$, $p \approx 2 \cdot 10^{-10}$	Long, fat pipe (SL 3-154).	8.5

Formula	Meaning · conditions · trap	§
$n_{\text{inflight,ideal}} =$ share of bottleneck bw $\times$ min_RTT	The connection's BDP; BBR's target.	8.7
measured throughput = bytes in an RTT $/RTT_{\text{measured}}$; trace: bytes / elapsed time	SL 3-133; Wireshark measurement.	8.7, 10
fair share $= R/K$; m of K connections $\Rightarrow mR/K$	1 of 10 $\Rightarrow R/10$; 11 of 20 $\Rightarrow 0.55R$.	8.10
$T_1/T_2 = RTT_2/RTT_1$ at equal p[S]	RTT unfairness.	8.10
J. Other relationships		
Scenario 1: $\lambda_{\text{out}} = \min\{\lambda_{\text{in}}, R/2\}$; delay $\to \infty$ as $\lambda_{\text{in}} \to R/2$	Cost #1.	7.2
Scenario 2 at $\lambda'_{\text{in}} = R/2$: (a) $R/2$, (b) $R/3$ ($0.333R$ + $0.166R$ retx), (c) $\to R/4$	Costs #2, #3.	7.2
Scenario 3: $\lambda_{\text{out}} \to 0$ as $\lambda'_{\text{in}} \to \infty$	Congestion collapse; cost #4.	7.2
fast retransmit after 3 duplicate ACKs (4 identical)	Trap: "two".	6.6
delayed ACK $\leq 500\,\text{ms}$	TB Table 3.2.	6.6
max packet lifetime $\approx 3\,\text{min}$; TIME_WAIT $30\,\text{s}/1\,\text{min}/2\,\text{min}$ (TB), $2\times$MSL (SL, class)	Sequence reuse; closing.	5.5, 6.8
IP Protocol: UDP = 17, TCP = 6	Selects the transport protocol (supplementary, TB Ch. 4).	10
frame $= 14 + 20 +$ TCP hdr $+$ data	Lab: $14 + 20 + 32 + 1448 = 1514$; SYN $14 + 20 + 44 = 78$.	10

13 Curated Question Bank

Representative questions only: where the source had several questions testing the same procedure, the strongest is kept (the worked examples in the chapters already cover the others). Try each before reading the answer. Answers are compact; the section reference points to the full development.

13.1 Level 1 — Basic recall

Question	Answer
Define the transport layer using "logical communication".	Provides logical communication between application processes running on different hosts.
What identifies a UDP socket? A TCP socket?	(dst IP, dst port); (src IP, src port, dst IP, dst port).
Which port numbers are well-known?	0–1023.
Header size of UDP; of TCP without options?	8 bytes; 20 bytes.
What does ARQ stand for, and what three capabilities does it need?	Automatic Repeat reQuest: error detection, receiver feedback, retransmission.
Why is rdt3.0 the alternating-bit protocol?	Its sequence numbers alternate 0, 1, 0, 1.
What does rdt2.1 fix in rdt2.0, and how?	Corrupted ACK/NAKs — by numbering packets (0/1) so duplicates are recognised.
What does TCP's sequence number of a segment denote? The ACK number?	The byte-stream number of its first data byte; the next byte expected from the other side.
MSS: definition and typical value on Ethernet.	Maximum application data per segment; $1500 - 40 = 1460$ bytes.
Recommended α, β; safety factor; initial timeout?	0.125, 0.25, 4, 1 s.
How many duplicate ACKs trigger fast retransmit?	Three (four identical ACKs).
What is *rwnd* and who sets it?	Free space in the receive buffer; set by the receiver, in every segment.
Purpose of TIME_WAIT?	To resend the final ACK if lost (and let old segments die out: $2\times$MSL).
Reply to a SYN for a port with no socket? To a UDP segment?	TCP RST; an ICMP datagram.
Define a loss event.	A timeout or three duplicate ACKs.
What is *ssthresh* set to after a loss event?	Half of *cwnd* at the time of the loss.
What does ECE indicate; what does CWR indicate?	Receiver echoes an ECN congestion mark; sender has reduced its window.
At which layer does QUIC operate?	Application layer, over UDP.

13.2 Level 2 — Conceptual understanding

C1. Give four reasons an application might choose UDP.

Answer. Finer control of what is sent and when; no connection-setup delay; no connection state (more clients per server); 8- vs 20-byte header. §3.2

C2. Why does UDP carry a checksum when Ethernet already checks errors?

Answer. Not every link on the path is guaranteed to check, and bits can flip in router memory; error detection needed end to end must be done end to end (end-end principle). §3.4

C3. Why do two sequence numbers suffice for stop-and-wait?

Answer. One packet outstanding: the receiver only asks "same packet or next?"; a retransmission always carries the same bit as its original, a new packet the other bit.

C4. Why does the rdt3.0 sender ignore an ACK with the wrong number instead of retransmitting?

Answer. The timer already handles loss; retransmitting on a stale ACK would turn every premature timeout into a cascade of needless retransmissions.

C5. Why must an SR receiver re-ACK packets below its window?

Answer. Its ACK may have been lost; the sender's window cannot advance until that packet is ACKed, so the receiver must acknowledge the retransmission even though it already has the data.

C6. Why does TCP never take a SampleRTT from a retransmitted segment?

Answer. The ACK is ambiguous (original or retransmission?); a wrong guess corrupts *EstimatedRTT*, typically shrinking the timeout when the network is congested (Karn).

C7. Why does TCP wait for three duplicate ACKs, not one?

Answer. A single duplicate is often harmless reordering; three mean three later segments arrived while the gap persisted — likely loss.

C8. Explain the zero-window problem.

Answer. B advertises $rwnd = 0$ and has nothing to send, so it never announces freed space; A would block for ever. A must keep sending one-byte segments, whose ACKs eventually carry a non-zero *rwnd*.

C9. Why is a two-way handshake insufficient?

Answer. With delay, loss and retransmission a stale request can open a half-open connection to a departed client, or old data can be accepted twice; the third message proves the client received the server's reply in this incarnation.

C10. How do SYN cookies defeat a SYN flood?

Answer. The server keeps no state for a SYN: its ISN is a keyed hash of the addresses and ports; a legitimate ACK is verified by recomputing it (hash +1 = ACK number) and only then is a connection created.

C11. Why is TCP's response to three duplicate ACKs milder than to a timeout?

Answer. Duplicates prove later segments are still getting through — mild congestion; a timeout suggests nothing is getting through.

C12. Explain self-clocking.

Answer. ACK arrivals trigger window growth and new transmissions, so the sending rate automatically follows the rate at which the path delivers.

C13. How can rising RTT indicate congestion?

Answer. Propagation and transmission delays are fixed; any RTT above $RTT_{\min}$ is queueing, which grows before buffers overflow (Vegas, BBR).

13.3 Level 3 — Numerical

N1. Checksum. Compute the checksum of `1000011001011110`, `1010110001100000`, `0111000100101010` and show the receiver's check.

Answer. $w_1 + w_2 =$ `1 0011001010111110` $\rightarrow$ wrap $\rightarrow$ `0011001010111111`; $+w_3 =$ `1010001111101001` (no carry) = sum. Checksum = `0101110000010110`. Receiver: sum + checksum = `1111111111111111`. The first wraparound is the step most often missed.

N2. Stop-and-wait. $R = 100$ Mbps, $RTT = 40$ ms, 1500-byte packets. Find L/R, U, throughput and the window for full utilisation.

Answer. $L = 12\,000$ bits; $L/R = 0.12$ ms; $U = 0.12/40.12 = 0.0030$; throughput $= 12\,000/0.04012 \approx 299$ kbps; $N \geq 1 + 10^8 \times 0.04/12\,000 = 334.3 \Rightarrow 335$ packets.

N3. Sequence numbers. client_isn $= 2000$; after the handshake the client sends 500, 300, 700 bytes. (a) Seq of each; (b) ACKs if all arrive; (c) ACKs if the second is lost; (d) client's next Seq.

Answer. The SYN consumed 2000, so data starts at 2001. (a) 2001, 2501, 2801. (b) 2501, 2801, 3501. (c) 2501, then duplicate 2501. (d) 3501. Forgetting the SYN shifts every answer by one.

N4. RTT and backoff. $EstimatedRTT = 80$ ms, $DevRTT = 5$ ms; a sample of 90 ms arrives. (a) New values and *TimeoutInterval*. (b) The next segment's timer then expires twice: what intervals are used? (c) A later, never-retransmitted segment gives a 100 ms sample: new *TimeoutInterval*?

Answer. (a) $EstimatedRTT = 0.875(80) + 0.125(90) = 81.25$; $|90 - 81.25| = 8.75$; $DevRTT = 0.75(5) + 0.25(8.75) = 5.94$; $TimeoutInterval = 81.25 + 4(5.94) = 105.0$ ms. (b) $105 \to 210 \to 420$ ms (doubling; no samples from the retransmitted segment). (c) $EstimatedRTT = 83.59$, $DevRTT = 8.55$, $TimeoutInterval \approx 117.8$ ms — recomputed from the estimators once the timer is restarted by new data/ACK.

N5. Segmentation. A 1 000 000-byte file over a path with MTU 1500 (40 bytes of TCP/IP headers), first byte numbered 0. MSS, number of segments, size and Seq of the last?

Answer. MSS 1460; $\lceil 10^6/1460 \rceil = 685$; the first 684 carry 998 640 bytes, so the last carries 1360 bytes with Seq 998 640.

N6. Congestion window. Reno, $cwnd = 1$, $ssthresh = 16$. A triple duplicate ACK occurs in round 7 and a timeout in round 15. Tabulate rounds 1–20; which rounds are slow start; segments sent in rounds 1–7?

Answer. *cwnd*: 1, 2, 4, 8, 16 (reaches *ssthresh*: CA), 17, 18 | 3 dup ACKs: $ssthresh = 9$, $cwnd = 12 \to 9$ | 9, 10, ..., 16 (rounds 8–15) | timeout at 16: $ssthresh = 8$, $cwnd = 1$ | 1, 2, 4, 8 (CA), 9 (rounds 16–20). Slow start: rounds 1–4 and 16–18. Rounds 1–7 send $1 + 2 + 4 + 8 + 16 + 17 + 18 = 66$ segments.

N7. Macroscopic throughput. Reno with MSS 1000 bytes, RTT 200 ms, loss whenever the window reaches 100 segments.

Answer. $W = 800\,000$ bits; $0.75 \times 800\,000/0.2 = 3$ Mbps (sawtooth between 2 and 4 Mbps).

N8. Mathis. MSS 1460 bytes, RTT 50 ms: (a) throughput at $p = 10^{-4}$; (b) p for 1 Gbps.

Answer. $1.22 \times 11\,680 = 14\,249.6$. (a) $14\,249.6/(0.05 \times 0.01) \approx 28.5$ Mbps. (b) $\sqrt{p} = 14\,249.6/(0.05 \times 10^9) = 2.85 \times 10^{-4} \Rightarrow p \approx 8 \times 10^{-8}$. Throughput scales as $1/\sqrt{p}$: $100\times$ less loss buys only $10\times$ more rate.

N9. Fairness. Four applications with one connection each share a 100 Mbps bottleneck; a fifth opens six. Shares?

Answer. $K = 10$ connections of 10 Mbps: 10, 10, 10, 10 and 60 Mbps. Fairness is per connection.

N10. Slow start. From $cwnd = 1$, no loss or $ssthresh$ limit: RTTs to send 63 segments?

Answer. $1 + 2 + 4 + 8 + 16 + 32 = 63 = 2^6 - 1$: 6 RTTs.

N11. TCP header. Header-length field $= 8$; IP datagram of 1500 bytes with a 20-byte IP header. Options and data length?

Answer. Header $= 32$ bytes $\Rightarrow$ 12 bytes of options; data $= 1500 - 20 - 32 = 1448$ bytes.

13.4 Level 4 — Protocol tracing

T1. rdt3.0: pkt0 and ACK0 succeed; pkt1 arrives but ACK1 is lost. Trace to the delivery of the next pkt0.

Answer. Sender in "Wait for ACK 1" times out (knows only that no ACK1 came), resends pkt1, restarts timer. Receiver, expecting 0, sees seq 1: duplicate — not delivered, ACK1 resent. ACK1 arrives: stop timer, "Wait for call 0"; next `rdt_send` sends pkt0, which is delivered and ACKed.

T2. GBN and SR, $N = 4$, packets 0–6 sent as the window allows, pkt3 lost. What happens at the receiver on 4, 5, 6 and at the timeout?

Answer. *GBN*: 4, 5, 6 discarded, ACK2 re-sent each time (ignored); window stuck at base 3; timeout resends 3, 4, 5, 6; delivered in order. *SR*: 4, 5, 6 buffered and ACKed individually; only pkt3 resent on its own timer; on arrival 3–6 are delivered together, `rcv_base`$\to 7$; ACK3 moves `send_base` to 7 and packets 7–10 become sendable.

T3. TCP: A sends Seq 100, 150, 200 (50 bytes each); all arrive; the first ACK is lost; ACK 200 and ACK 250 arrive before any timeout. What is resent; final SendBase?

Answer. Nothing: ACK 200 cumulatively covers 100–199. SendBase $= 250$; timer stopped.

T4. client_isn $= 3000$, server_isn $= 7000$. After the handshake the client sends 100 bytes, the server ACKs, the client closes. Every segment until its FIN is ACKed?

Answer. (1) C→S SYN, Seq 3000. (2) S→C SYNACK, Seq 7000, ACK 3001. (3) C→S ACK, Seq 3001, ACK 7001. (4) C→S 100 bytes, Seq 3001, ACK 7001 (may merge with 3). (5) S→C ACK, Seq 7001, ACK 3101. (6) C→S FIN, Seq 3101, ACK 7001. (7) S→C ACK, Seq 7001, ACK 3102. Client in FIN_WAIT_2, server in CLOSE_WAIT.

T5. A sends Seq 300, 400, 500, 600, 700 (100 bytes each); 400 is lost. B's ACKs and A's action?

Answer. ACK 400, then three duplicate ACK 400s (for 500, 600, 700). On the third duplicate A fast-retransmits Seq 400; when it arrives B ACKs 800.

T6. B advertises $rwnd = 0$ and has no data. What happens?

Answer. A stops normal sending but transmits one-byte probe segments; B ACKs each with its current *rwnd*; once B's application reads, an ACK carries $rwnd > 0$ and A resumes.

13.5 Level 5 — Analytical: compare, justify, debug

A1. Is TCP a GBN or an SR protocol? Justify both ways.

Answer. GBN-like: cumulative ACKs, SendBase/NextSeqNum only. SR-like: out-of-order buffering, only one segment resent (not even that if a later cumulative ACK arrives in time); with SACK closer still to SR. Verdict: hybrid. §6.6

A2. A 3-bit sequence field is available and the path needs 6 packets in flight. Which protocol, and what is the trade-off?

Answer. GBN ($N \leq 7$) can fill the pipe; SR ($N \leq 4$) cannot unless $k \geq 4$. SR saves retransmissions but costs a bit of sequence space and receiver buffering.

A3. A transfer is slow. How do you decide whether flow control or congestion control is the limit?

Answer. Compare in-flight data with the advertised *rwnd*: if the sender sits at *rwnd*, the receiver binds; if well below *rwnd* with loss/ACK-clocked growth, *cwnd* binds. Conceptual test: would it still happen with an infinitely fast network? Then it is flow control.

A4. Why does ECN not simply replace loss-based detection?

Answer. It needs router support and ECN-capable endpoints, the marking policy is operator-defined, the signal still costs an RTT, and it says only *that* there is congestion; loss remains the universal fallback.

Spot the error. (explanations in Section 11)

Claim	Correction
"Seq 5000 with 200 bytes arrives in order, so ACK 5199."	ACK 5200 (next byte expected).
"Our SR protocol uses a 3-bit sequence number and a window of 5."	$N \leq 2^2 = 4$; old and new windows would overlap.
An rdt2.1 receiver in "Wait for 0" gets an uncorrupted seq-1 packet, delivers it and moves to "Wait for 1".	It is a duplicate: do not deliver, stay, send an ACK.
"The sender keeps LastByteSent − LastByteAcked ≤ RcvBuffer."	The bound is *rwnd* (free space), not the whole buffer.
"Our UDP server creates a socket per client, since clients are demultiplexed by source address."	UDP demultiplexes on (dst IP, dst port); per-client sockets are TCP's 4-tuple.
A student adds 16-bit words and discards the carry.	The carry must be wrapped around into the LSB.
"In the SYNACK the ACK number is client_isn."	client_isn+1: the SYN consumes one number.
"Routers store the connection's sequence numbers."	Routers keep no TCP state; only the end systems do.

Essay questions with answer skeletons.

E1. *Trace rdt1.0 → rdt3.0.* Per version: channel assumption, new mechanism, sender/receiver states, remaining weakness (Table 4); FSM sketches; end with stop-and-wait utilisation motivating pipelining.

E2. *Pipelining; GBN vs SR.* Utilisation argument and BDP; windows, ACK types, timers, buffering, behaviour on loss/reordering (Table 5); window limits and the SR dilemma; timelines.

E3. *How TCP achieves reliability.* Byte-stream Seq/ACK, cumulative ACKs, single timer and its estimation (EWMA, $4DevRTT$, Karn, backoff), ACK-generation rules, fast retransmit; GBN/SR hybrid verdict.

E4. *Connection management.* Handshake with numbers; why not two messages; four-segment close and half-close; state sequences; TIME_WAIT; RST/ICMP/nmap; SYN flood and cookies.

E5. *Causes and costs of congestion.* Scenarios 1–3 mapped to costs #1–#4; $R/2 \to R/3 \to R/4 \to$ collapse; SL 3-118's five insights; end-to-end vs network-assisted.

E6. *Classic TCP congestion control. cwnd* and $\min\{cwnd, rwnd\}$; loss events; slow start, CA, fast recovery (FSM); Tahoe vs Reno; AIMD; a *cwnd* graph.

E7. *Loss-based vs ECN vs delay-based.* Table 15; bottleneck, BDP, "just full, no fuller"; BBR's phases.
E8. *Is TCP fair?* Definition; two-connection convergence (geometry, and why AIMD); failures: RTT bias, UDP, parallel connections.
E9. *Why transport evolved.* SL 3-142 scenarios (Table 16); QUIC's placement, features, HOL blocking; application-update timescales.

13.6 Level 6 — Challenging (connecting several ideas)

H1. $R = 1$ Gbps, $RTT = 80$ ms. (a) BDP in bytes; (b) maximum throughput with an unscaled receive window; (c) implication?

Answer. (a) $10^9 \times 0.08 = 8 \times 10^7$ bits $= 10$ MB. (b) $65\,535 \times 8/0.08 \approx 6.55$ Mbps (0.66% of the link). (c) The 16-bit field cannot express the needed window: the window-scaling option exists for this; congestion control would further need a tiny loss rate to hold such a window.

H2. At 10 Gbps, how long until TCP's 32-bit sequence space wraps? Compare with the maximum packet lifetime and explain the danger.

Answer. $2^{32}/(1.25 \times 10^9$ bytes/s$) \approx 3.4$ s — far below the ≈ 3 min lifetime assumed by RFC 7323. A delayed old segment could reappear while its numbers are reused and be accepted as new data (the reuse hazard of §5.5). *Supplementary*: RFC 7323 guards against this with the timestamp option.

H3. A wireless path loses $p = 10^{-3}$ of segments to bit errors, not congestion. With MSS 1460 bytes and RTT 100 ms, estimate TCP's throughput on a 100 Mbps link.

Answer. $14\,249.6/(0.1 \times \sqrt{10^{-3}}) \approx 4.5$ Mbps: TCP reads noise as congestion and uses under 5% of capacity — a motivation for ECN, delay-based control and wireless-specific TCP variants.

H4. A 15-segment object is fetched over a new connection, RTT 100 ms, *cwnd* from 1, no loss, negligible transmission time. Latency from SYN to the last segment? How does TCP splitting help?

Answer. 1 RTT handshake + slow-start rounds of 1, 2, 4, 8 (= 15) $\approx$ 5 RTT $= 0.5$ s. Short transfers are RTT-bound, not bandwidth-bound; terminating the connection at a nearby front end makes the handshake and slow-start rounds cost the short client–front-end RTT.

H5. Explain how premature timeouts (scenario 2c) and multihop drops (scenario 3) interact with TCP's timer and window rules to prevent collapse.

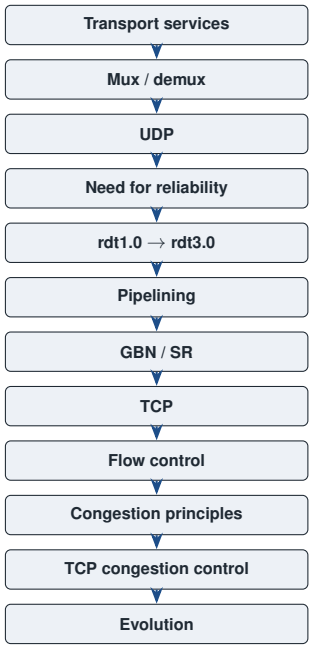
Answer. Duplicates (cost #3) and wasted upstream work (cost #4) grow with load; if senders kept retransmitting at the same pace, offered load would rise as goodput fell. TCP's defences: $EstimatedRTT + 4DevRTT$ reduces premature timeouts; Karn keeps samples honest; exponential backoff doubles the timeout after each expiry; a timeout drops *cwnd* to 1 MSS — so offered load falls exactly when the network is overloaded.

H6. Four Web objects are fetched over one TCP connection and, alternatively, over four QUIC streams; one segment carrying part of object 1 is lost. Compare what the application sees, and what is still shared.

Answer. TCP: in-order delivery for the whole connection — objects 2–4 wait behind the gap (HOL blocking). QUIC: only stream 1 waits; streams 2–4 are delivered. Still shared: one QUIC congestion controller, so the loss slows the whole connection's sending rate.

14 Final Revision

14.1 The chapter on one page



IP delivers to *hosts*, best effort; applications need *processes*, often reliably.

Ports and sockets turn host delivery into process delivery (UDP 2-tuple; TCP 4-tuple).

Mux/demux + checksum and nothing else: the thinnest useful transport.

The channel corrupts, loses, delays; a checksum only detects.

Checksum + feedback + retransmission → sequence numbers → NAK-free → timer: one fix per channel problem.

rdt3.0 is correct but stop-and-wait wastes the pipe ($U = 0.00027$); fill it with $N \geq 1 + R \cdot RTT/L$.

Many packets in flight need a recovery policy: resend the window, or only what is missing.

Byte-stream sliding window: cumulative ACKs, one timer, $EstimatedRTT + 4DevRTT$, fast retransmit — a GBN/SR hybrid.

A reliable sender can still overrun a slow reader: advertise *rwnd*.

Senders together overrun the network: delay, retransmission, duplicates, wasted upstream work → collapse.

Infer congestion from loss; limit with *cwnd*; slow start, AIMD, fast recovery; fair shares R/K.

Better probing (CUBIC), explicit (ECN) and delay signals (Vegas, BBR); transport moved into the application (QUIC).

Problem	Mechanism	Where in TCP
Which process?	ports, sockets	4-tuple demultiplexing
Bits flipped	checksum	Internet checksum
Did it arrive intact?	ACK (and NAK in rdt2.0/2.1)	cumulative ACK number
Duplicates after retransmission	sequence numbers	byte-stream sequence numbers
Packet or ACK lost	timer, retransmission	one timer; $EstimatedRTT + 4DevRTT$; doubling
Idle sender (stop-and-wait)	pipelining, windows	$\min\{cwnd, rwnd\}$ window
Slow loss recovery	duplicate ACKs as implicit NAKs	fast retransmit after 3 duplicates
Receiver overrun	receiver-advertised window	*rwnd*; 1-byte zero-window probes
Network overrun	sender-side congestion window	slow start, AIMD, fast recovery
Agree on initial state / release it	handshake / teardown	SYN, SYNACK, ACK / FIN–ACK each way, TIME_WAIT
Unequal shares	AIMD convergence	fairness under idealised conditions

14.2 The formulas you must be able to write from memory

The full sheet with conditions is Section 12; these are the ones exams actually use.

Stop-and-wait / pipelined	$U = \frac{L/R}{RTT+L/R}$; $U = \min\{1, \frac{NL/R}{RTT+L/R}\}$; $N \geq 1 + \frac{R\,RTT}{L}$
Window limits	$N_{\text{GBN}} \leq 2^k - 1$, $N_{\text{SR}} \leq 2^{k-1}$
TCP numbers	ACK $= Seq + \ell$ (first missing byte); SYN/FIN consume 1; $MSS = \text{MTU} - 40$
RTT	$EstimatedRTT \leftarrow 0.875\,EstimatedRTT + 0.125\,SampleRTT$
	$DevRTT \leftarrow 0.75\,DevRTT + 0.25\,\lvert SampleRTT - EstimatedRTT\rvert$
Timeout	$TimeoutInterval = EstimatedRTT + 4DevRTT$; initial 1 s; doubled on each timeout
Flow control	$rwnd = \text{RcvBuffer} - (\text{LastByteRcvd} - \text{LastByteRead})$
Sending limit, rate	$\text{LastByteSent} - \text{LastByteAcked} \leq \min\{cwnd, rwnd\}$; rate $\approx cwnd/RTT$
Loss responses	timeout: $ssthresh = cwnd/2$, $cwnd = 1$ MSS, slow start
	3 dup ACKs (Reno): $ssthresh = cwnd/2$, $cwnd = ssthresh + 3 \rightarrow ssthresh$, then CA
Throughput	$0.75\,W/RTT$; $1.22\,MSS/(RTT\sqrt{p})$; fair share R/K
Checksum	one's-complement sum with wraparound, then complement; receiver gets all 1s

14.3 Protocol comparison

Table 19: UDP, TCP, GBN, SR and QUIC side by side (details: Tables 2, 6 and 18).

	UDP	TCP	GBN	SR	QUIC
Kind	Internet protocol	Internet protocol	rdt principle	rdt principle	app-layer protocol over UDP
Connection	none	3-way handshake	—	—	1-/0-RTT, with crypto
Reliability	none	byte stream, in order	packets, in order	packets, in order	per stream
ACKs	—	cumulative (+SACK)	cumulative	individual	TCP-like, per stream
Timers	—	one	one	one per packet	(TCP-like)
On loss	nothing	resend oldest; fast retransmit	resend window	resend that packet	resend in that stream
Receiver buffering	socket buffer only	out-of-order kept (in practice)	none	up to N	per stream
Window limit	—	$\min\{cwnd, rwnd\}$	$2^k - 1$	2^{k-1}	flow + congestion
Flow / congestion control	no / no	yes / yes	—	—	yes / yes (shared)
Header	8 bytes	20–60 bytes	k-bit seq	k-bit seq	encrypted; conn. + stream IDs
Typical use	DNS, SNMP, HTTP/3	HTTP, SMTP, SSH, FTP, DASH	teaching model	teaching model	HTTP/3

14.4 Diagrams to be able to draw

1. Five-layer stack with the logical end-to-end arrow (Fig. 1); mux/demux and port inversion (Figs. 2 and 3); three segments to port 80 (Fig. 4).
2. UDP header (Fig. 5) and TCP header (Fig. 19).
3. FSMs: rdt2.0 sender/receiver, rdt2.1 sender and receiver, rdt2.2 receiver, rdt3.0 sender (Figs. 8 to 11); the four rdt3.0 timelines (Fig. 12).
4. Stop-and-wait vs pipelined timing (Fig. 13); GBN/SR windows (Fig. 14); GBN and SR in action (Fig. 16); the SR dilemma (Fig. 17).

5. Telnet Seq/ACK (Fig. 20); retransmission scenarios and fast retransmit (Figs. 21 and 22); receive buffer (Fig. 23).
6. Handshake, close and state sequences (Figs. 25 to 27).
7. Congestion scenario curves and feedback pathways (Figs. 28 and 29).
8. Slow start, AIMD sawtooth, congestion-control FSM, Tahoe/Reno evolution (Figs. 30 to 33); CUBIC, in-flight graphs, ECN, fairness (Figs. 34 to 37).
9. QUIC stacks and HOL blocking (Figs. 38 and 39).

14.5 High-value definitions

Term	Exam-ready definition
Transport layer	Logical communication between application processes on different hosts.
Demultiplexing	Delivering a received segment's data to the correct socket (multiplexing: the sender-side gathering and header-adding).
Best-effort service	No guarantee of delivery, order or integrity (IP).
End-end principle	Functions that must be end to end may be redundant or of little value at lower levels.
ARQ	Reliability through error detection, receiver feedback and retransmission.
Stop-and-wait	Send one packet and wait for its acknowledgment before the next.
Utilisation	Fraction of time the sender is busy sending bits into the channel.
Pipelining	Several transmitted-but-unacknowledged packets in flight.
Cumulative ACK	Acknowledges everything up to a point (GBN: $\leq n$; TCP: all bytes before the ACK number).
Duplicate ACK	Re-acknowledgment of already-ACKed data; signals a gap (implicit NAK).
MSS / MTU	Max application data in a segment / largest link-layer frame.
SampleRTT, EWMA	Time from sending a segment to its ACK; average whose weights decay exponentially.
Fast retransmit	Resend after three duplicate ACKs, before the timer expires.
Flow control	Speed matching so the sender cannot overflow the receiver's buffer.
Congestion	Too many sources sending too much data too fast for the network.
Offered load	λ'_{in}: transport sending rate including retransmissions.
Congestion collapse	Throughput falling towards zero as offered load rises.
Loss event	A timeout or three duplicate ACKs.
Self-clocking	ACK arrivals trigger window increases and new sends.
ssthresh	Half of *cwnd* at the last loss; switch point from slow start to CA.
AIMD	+1 MSS per RTT; halve on loss.
Bottleneck link	The congested, almost always busy link where loss occurs.
Fairness	K connections at a bottleneck of rate R each average $\approx R/K$.
HOL blocking	A lost segment delaying all later data of an in-order stream.

14.6 Last-minute checklist

☐ Transport vs network; host-to-host vs process-to-process; services UDP/TCP do and do not provide.
☐ Which socket receives a given UDP or TCP segment; reply ports.
☐ Compute and verify an Internet checksum with wraparound; give an error it misses.
☐ Why each rdt version exists; draw its FSM; trace rdt3.0 through loss, ACK loss and premature timeout.
☐ Stop-and-wait and pipelined utilisation; window for full utilisation.
☐ Trace GBN and SR after a loss; window limits and the SR dilemma.
☐ Draw and label the TCP header; Seq/ACK for any exchange including SYN and FIN.
☐ A multi-step *EstimatedRTT*/*DevRTT*/*TimeoutInterval* table; exponential backoff; Karn's rule.
☐ The TCP sender's three events; the ACK-generation table; fast retransmit and why three.
☐ *rwnd*, usable window, zero-window probes; flow vs congestion control in one sentence each.
☐ Handshake and teardown with numbers; state names; TIME_WAIT; SYN cookies; nmap outcomes.
☐ Four costs of congestion from the three scenarios.
☐ A round-by-round *cwnd* table with both loss types, Tahoe and Reno (state the Reno convention); identify phases from a graph.
☐ $0.75W/RTT$ (derive it) and the Mathis relation; long-fat-pipe numbers.

- ☐ CUBIC, ECN (bits, ECE, CWR), Vegas, BBR in two sentences each; fairness and its failures.
- ☐ QUIC's layer, features and HOL blocking.
- ☐ Read a Wireshark trace: flags, raw vs relative Seq, header length, 1448 vs 1460, window scaling, RTT, throughput, fleets.
- ☐ Class conventions: Tahoe/Reno trace ($ssthresh = 8$, loss at 12, Reno continuing from 6); ECN code points 00 / 01, 10 / 11; TIME_WAIT $2\times$MSL.

Source & convention note: Sources and supplementary material

Definitions, mechanisms, figures and most examples follow [TB]; quoted wording is from [TB] or [SL] as attributed; [SL] supplied lecture emphasis and 9th-edition-only material (bottleneck link, n_{inflight}, BBR phases, TIME_WAIT as $2\times$MSL, the long-fat-pipe example); [CN] set the Reno and TIME_WAIT conventions and the ECN code-point table; [AS] supplied Section 10. Worked examples with new numbers use only the sources' formulas. Material from outside the sources is limited to, and labelled where it appears: the RFC 6298 update-order remark; the RFC 1071 byte-order remark; separate TCP/UDP port spaces and IP protocol numbers (TB Ch. 4); RFC 3168 code-point names; the algebraic AIMD convergence argument (Chiu–Jain); the timestamp remark in H2.